

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT
SPECIALIZED PROJECT
SEMESTER 251 ACADEMIC YEAR 2025-2026**

**DOMAIN GENERALIZATION FOR
ADVERSARIAL ROBUSTNESS**

MAJOR: COMPUTER SCIENCE
COUNCIL: 10CC Specialized Projects
SUPERVISOR(s): Dr. Nguyen An Khuong
Dr. Van Minh Hao
REVIEWER: Dr. Truong Vinh Lan

—o0o—

STUDENT 1: Nguyen Anh Kiet - 2252403
STUDENT 2: Le Van Duc Anh - 2252027
STUDENT 3: Nguyen Tien Thanh - 2053437

HO CHI MINH CITY, December 2025

Instructor's Signature

_____ Date: _____

NGUYEN AN KHUONG (Instructor)

...

Faculty of Computer Science and Engineering

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of Dr. Nguyen An Khuong at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology and Dr. Van Minh Hao at University of Arkansas.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2025

Authors

Acknowledgement

We would like to express my appreciation to Dr. Nguyen An Khuong for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from Dr. Van Minh Hao for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on adversarial attack so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to our family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in our potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

This report addresses the critical vulnerability of deep learning models to adversarial perturbations, particularly the limitation where standard defense mechanisms fail to generalize across different threat models. Our research proposes a novel perspective that treats distinct families of adversarial attacks as separate "domains," aiming to identify and quantify the "cross-attack robustness gap" inherent in current training paradigms.

We establish a comprehensive experimental framework to evaluate the transferability of robustness across diverse adversarial environments. By subjecting models trained on specific perturbation norms to a wide spectrum of unseen attack algorithms, the study systematically measures the performance degradation that occurs when shifting between these domains.

Ultimately, this quantitative analysis serves as the empirical foundation for the subsequent phase of research. By characterizing the domain shifts between attack families, we pave the way for integrating Domain Generalization principles into adversarial training. This progression aims to directly mitigate the observed robustness gaps, ensuring that future models can effectively generalize to unforeseen and heterogeneous adversarial scenarios.

Contents

1	Introduction	11
1.1	Motivation	11
1.2	Problem Statement	12
1.3	Objectives	12
1.4	Contributions	13
1.5	Report Organization	14
2	Background	15
2.1	Deep Neural Networks and Classification	15
2.2	Adversarial Examples and Attacks	17
2.2.1	Adversarial Examples	17
2.2.2	Fast Gradient Sign Method (FGSM)	17
2.2.3	Projected Gradient Descent (PGD)	21
2.2.4	Carlini–Wagner (CW) Attack	23
2.2.5	DeepFool	26
2.2.6	AutoAttack	30
2.3	Adversarial Training	37
2.3.1	Min–Max Formulation	37
2.3.2	Madry’s PGD Adversarial Training	39
2.3.3	Variants of Adversarial Training	42
2.4	Domain Shift and Domain Generalization	43
2.4.1	Problem Setup and Taxonomy	43
2.4.2	Data Manipulation for Domain Generalization	44
2.4.3	Representation Learning	45
2.4.4	Learning Strategy	46
2.4.5	From Classical DG to Attacks-as-Domains	48
2.5	Chapter Summary	48
3	Related Work	49
3.1	Adversarial Robustness and Adversarial Training	49
3.2	Domain Generalization	50

3.3	Bridging Adversarial Robustness and Domain Generalization	51
3.4	Research Gap: Attacks as Domains	53
4	Proposed Framework	55
4.1	Attacks as Domains	56
4.2	Research Questions and Hypotheses	56
4.3	Framework Overview	58
4.3.1	Two-phase structure.	58
4.3.2	Pipeline overview.	59
4.4	Design Space and Prioritization of DG Objectives	61
4.5	Evaluation Protocol	63
4.6	Validation Scenarios	66
4.7	Expected Trade-offs	68
5	Initial Implementation	70
5.1	Baseline Setup	70
5.1.1	Dataset	70
5.1.2	Model Architecture	71
5.1.3	Attack Suite	73
5.1.4	Threat Model and Attack Configuration	74
5.2	Adversarial Training and Evaluation Workflow	74
5.2.1	PGD-AT Training Pipeline	74
5.2.2	Metrics and Logging	75
5.3	Robustness Results	76
5.3.1	Summary Table of Robust Accuracies	76
5.3.2	Key Findings	77
5.3.3	PGD Step-Sweep Analysis	78
5.3.4	PGD ℓ_∞ ϵ -Sweep	79
5.3.5	Qualitative Attack Diagnostics	80
5.3.6	Explained Insights	82
5.4	Main Limitations and Takeaways	83
5.5	Summary and Transition to Stage 2	84
6	Conclusion and Future Work	85
6.1	Summary of Stage 1 Outcomes	85
6.2	Implications Under the Attacks-as-Domains View	86
6.3	Future Work: Stage 2 Plan	86
6.4	Closing Remarks	87

A	Pseudo-code and Flowcharts	92
A.1	FGSM Attack	92
A.2	PGD Attack	93
A.3	Carlini & Wagner Attack	93
A.4	DeepFool Attack	94
A.5	Auto-PGD attack	95
A.6	Training Pipeline Flowcharts - ERM vs Adversarial Training	97
A.7	Compute Environment (Reproducibility)	99
B	Notation Tables	100
B.1	Notation Summary	100
C	Additional Tables and Visualizations	103
C.1	PGD Step-Sweep Sensitivity	103

List of Figures

2.1	Schematic of a deep neural network classifier	15
4.1	High-level training and evaluation pipeline.	59
5.1	CIFAR-10 samples	71
5.2	BasicBlock of residual learning used in ResNet-18	71
5.3	ResNet-18 with CIFAR stem	72
5.4	PGD- ℓ_∞ step-sweep	78
5.5	PGD- ℓ_2 step-sweep	79
5.6	PGD- ℓ_∞ ε -sweep	80
5.7	Qualitative norm comparison	81
5.8	Qualitative attack-family comparison	82
A.1	APGD workflow	95
A.2	ERM training pipeline	97
A.3	Adversarial training pipeline	98
C.1	Per-model PGD step-sweep	105
C.2	PGD- ℓ_∞ qualitative grid	106
C.3	PGD- ℓ_2 qualitative grid	107
C.4	CW- ℓ_2 qualitative grid	108
C.5	AutoAttack APGD-CE examples	109
C.6	AutoAttack APGD-DLR examples	110
C.7	AutoAttack FAB examples	111
C.8	AutoAttack Square examples	112

List of Tables

4.1	Qualitative comparison of candidate DG objectives	63
5.1	Stage 1 robust accuracies on CIFAR-10	77
B.1	Notation for data and models	100
B.2	Notation for losses	101
B.3	Notation for adversarial examples	101
B.4	Notation for hyperparameters	101
B.5	Notation for robustness metrics	102
B.6	Notation for domain generalization	102
C.1	PGD step-sweep (baseline ERM)	104
C.2	PGD step-sweep (TA-100)	104
C.3	PGD step-sweep (TA-50)	104

List of Abbreviations

Abbreviation	Definitions
AA	AutoAttack
AT	Adversarial Training
CW	Carlini–Wagner
DG	Domain Generalization
DNN	Deep Neural Network
FGSM	Fast Gradient Sign Method
IRM	Invariant Risk Minimization
PGD	Projected Gradient Descent
REx	Risk Extrapolation

Chapter 1

Introduction

1.1 Motivation

Deep neural networks (DNNs) have achieved state-of-the-art performance on widely used image classification benchmarks such as MNIST, CIFAR-10, CIFAR-100, and ImageNet, exhibiting high accuracy under standard test conditions. However, it is now well established that these models are highly vulnerable to *adversarial examples*—inputs perturbed by visually imperceptible yet carefully designed modifications that lead to incorrect predictions. This susceptibility exposes a fundamental gap between standard generalization and reliable behavior under adversarial conditions, raising serious concerns for the deployment of DNNs in safety-critical settings and motivating the study of *adversarial robustness*.

A large body of work has shown that adversarial training — in particular, the Projected Gradient Descent (PGD)-based adversarial training procedure introduced by Madry et al. [1] — can substantially improve robustness against a given threat model, typically defined by a norm constraint and a perturbation budget. Nevertheless, robustness gains are often highly specific to the training attack. Models trained with a single PGD configuration can remain vulnerable to other attack algorithms, norms, budgets, or even different implementations of the same attack. This phenomenon suggests a form of *attack-specific overfitting*—the model learns to withstand the particular optimization procedure used during training, yet fails to generalize to other adversarial scenarios.

In this project we investigate adversarial robustness through the lens of *domain generalization* (DG). Instead of viewing robustness as resistance to a single adversary, we interpret each attack family as a distinct *domain*. This perspective allows us to use tools from domain generalization to analyze and potentially reduce the gap between robustness to the training attack and robustness to other attacks. We use robust accuracy under AutoAttack as the primary robustness metric; in the DG view, AutoAttack can be interpreted as a target-domain evaluation.

1.2 Problem Statement

We consider a standard supervised classification setting with inputs $x \in \mathbb{R}^d$, labels $y \in \{1, \dots, C\}$, a classifier $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^C$ parameterized by θ , and a clean data distribution P_{XY} . Adversarial examples are generated by perturbing a clean input x within a threat set

$$\mathcal{S}_p(\varepsilon) = \{\delta \in \mathbb{R}^d : \|\delta\|_p \leq \varepsilon\},$$

where $p \in \{2, \infty\}$ and $\varepsilon > 0$ is a budget chosen to keep perturbations visually imperceptible. An attack algorithm $\mathcal{A}$ (e.g., FGSM, PGD, Carlini–Wagner, DeepFool, AutoAttack) produces an adversarial example

$$x' = \mathcal{A}(x, y, f_\theta)$$

that attempts to maximize loss under the threat model.

Adversarial training (AT) aims to learn parameters θ that minimize the robust risk

$$\min_{\theta} \mathbb{E}_{(x,y) \sim P_{XY}} \left[\max_{\delta \in \mathcal{S}_p(\varepsilon)} \ell(f_\theta(x + \delta), y) \right],$$

where $\ell(\cdot, y)$ is typically the cross-entropy loss. In practice, the inner maximization is approximated using an attack algorithm $\mathcal{A}$, giving rise to a domain-specific risk

$$R_e(\theta) = \mathbb{E}_{(x,y) \sim P_{XY}} [\ell(f_\theta(\mathcal{A}_e(x, y, f_\theta)), y)]$$

for each attack $e \in \mathcal{E} = \{1, \dots, E\}$.

The fundamental problem addressed in this project is the limited generalization of adversarial robustness learned through standard PGD-based training. Although such models are typically robust to the specific attack used during training, they often exhibit substantial robustness gaps when evaluated against other attacks. This project investigates whether ideas from domain generalization can reduce these cross-attack gaps and improve worst-case and average robustness across diverse attack domains under realistic computational constraints on CIFAR-10.

1.3 Objectives

To address this problem, we structure the work into two stages with the following objectives:

- **Stage 1: Baseline implementation and analysis.**
 - Implement a unified training and evaluation pipeline for adversarial robustness on CIFAR-10, using a ResNet-18 backbone and a suite of attacks

- including FGSM, PGD under ℓ_∞ and ℓ_2 , Carlini–Wagner (CW) under ℓ_2 , DeepFool under ℓ_2 , and AutoAttack.
- Reproduce strong PGD-based adversarial training baselines (PGD-AT) under an ℓ_∞ threat model, including variants differing in training epochs and attack implementations.
- Quantify cross-attack robustness gaps, clean-versus-robust accuracy trade-offs, and compute cost across these baselines, using AutoAttack robust accuracy as the primary validation metric.
- **Stage 2: Domain-generalization-enhanced adversarial training (planned).**
 - Formalize the perspective of *attacks as domains* and connect adversarial robustness to domain generalization.
 - Design multi-attack adversarial training objectives that integrate DG-inspired methods such as Risk Extrapolation (REx), Group Distributionally Robust Optimization (GroupDRO), cross-domain Mixup, RAMP-style logit pairing, and optionally IRM-style penalties.
 - Propose an evaluation plan to compare these DG-enhanced models against Stage-1 baselines in terms of cross-attack robustness, worst-case and average robustness, and computational efficiency.

1.4 Contributions

The main contributions of this project can be summarized as follows:

- **Unified Stage-1 adversarial robustness pipeline.** We implement a unified training and attack harness on CIFAR-10 with ResNet-18, covering gradient-based (FGSM, PGD- ℓ_∞ , PGD- ℓ_2), optimization-based (CW- ℓ_2), boundary-based (DeepFool- ℓ_2), and ensemble (AutoAttack) attacks. Stage 1 includes cross-library comparison using both TorchAttacks and Adversarial Robustness Toolbox (ART), while Stage-2 experiments use TorchAttacks exclusively.
- **Empirical analysis of cross-attack robustness gaps.** Using this pipeline, we train multiple PGD-AT models and evaluate them across the attack suite. We report clean and robust accuracies, single-source cross-attack transfer profiles (evaluating PGD-trained models across the attack suite), and robustness gaps, demonstrating that models can be substantially more robust to the training attack than to other attacks, including AutoAttack.
- **Attacks-as-domains formulation and DG-inspired objectives.** We propose to treat each attack family as a domain and define domain-specific risks $R_e(\theta)$. Building on this formulation, we introduce DG-inspired adversarial training objectives including multi-attack ERM, REx-AT, GroupDRO-AT, Mixup-AT, RAMP-AT, and an optional IRM-AT variant.

- **Stage-2 evaluation plan for DG-enhanced robustness.** We outline a detailed evaluation plan for Stage 2 that specifies model variants, attack domains, evaluation scenarios (including leave-one-attack-out and held-out AutoAttack), metrics, and analyses. This plan is intended to guide future experiments on DG-enhanced adversarial training.

1.5 Report Organization

The remainder of this report is organized as follows.

- Chapter 2 reviews the theoretical background on deep neural networks, adversarial examples and attacks, adversarial training, domain shift, and domain generalization, and defines the robustness metrics used throughout the report.
- Chapter 3 surveys related work on adversarial robustness and domain generalization, and identifies a gap in existing approaches with respect to treating attack families as distinct domains.
- Chapter 4 presents the proposed framework, formalizing the attacks-as-domains perspective and introducing domain-generalization-inspired adversarial training objectives and the evaluation methodology planned for Stage 2.
- Chapter 5 details the Stage 1 implementation, including the dataset, model architecture, training configurations, and attack harness, and reports empirical results and cross-attack robustness analyses.
- Chapter 6 summarizes the findings, discusses their implications, and outlines directions for future research.

Chapter 2

Background

This chapter establishes the theoretical and methodological foundations for the project. We first review deep neural networks for classification and fix the notation used throughout. We then formalize adversarial examples and threat models, describe the adversarial attacks that constitute our evaluation suite, and present the adversarial training framework used for Stage 1. Finally, we introduce domain shift and domain generalization, and define evaluation metrics that treat each attack as a separate domain.

2.1 Deep Neural Networks and Classification

Deep neural networks (DNNs) have become the dominant paradigm for modern classification tasks due to their ability to learn hierarchical feature representations directly from data. A DNN consists of multiple layers of nonlinear transformations, where each layer maps an input vector to a higher-level abstraction. Formally, a network with parameters θ defines a function

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}^K, \quad (2.1)$$

which maps an input $x \in \mathbb{R}^d$ to a vector of K real-valued outputs, typically referred to as *logits*. These logits represent the model’s unnormalized scores for each class.

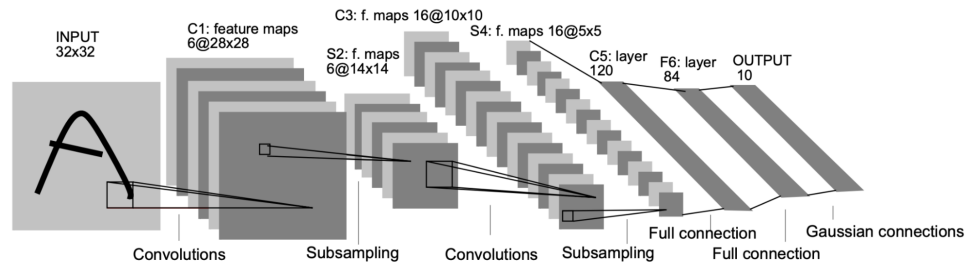


Figure 2.1: Schematic of a deep neural network classifier: stacked nonlinear layers map input x to logits $f_{\theta}(x)$ before the softmax.

Softmax Classifier. For classification, the logits are converted to a probability distribution over the K classes using the softmax function:

$$p_{\theta}(y = k \mid x) = \frac{\exp(f_{\theta}(x)_k)}{\sum_{j=1}^K \exp(f_{\theta}(x)_j)}, \quad (2.2)$$

where $f_{\theta}(x)_k$ denotes the logit corresponding to class k . The predicted label is typically chosen as the class with maximum probability:

$$\hat{y} = \arg \max_k p_{\theta}(y = k \mid x). \quad (2.3)$$

Cross-Entropy Loss. To train the network, we minimize the cross-entropy loss between the predicted distribution $p_{\theta}(\cdot \mid x)$ and the ground-truth label y . For a single training example (x, y) , the loss is defined as

$$\ell(f_{\theta}(x), y) = -\log p_{\theta}(y \mid x) = -f_{\theta}(x)_y + \log \sum_{j=1}^K \exp(f_{\theta}(x)_j). \quad (2.4)$$

The cross-entropy objective encourages the model to assign high probability to the correct class, and it is optimized via gradient-based methods such as stochastic gradient descent (SGD) or Adam.

Backpropagation and Training. Model parameters θ are updated by computing gradients of the loss with respect to θ using backpropagation. For a mini-batch $\mathcal{B}$, the update rule typically takes the form

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \left(\frac{1}{|\mathcal{B}|} \sum_{(x,y) \in \mathcal{B}} \ell(f_{\theta}(x), y) \right), \quad (2.5)$$

where η is the learning rate. Through repeated updates over many epochs, the network learns decision boundaries that separate the classes in the feature space.

Decision Boundaries and Robustness Considerations. Although high-dimensional and nonlinear, DNN classifiers often behave locally linearly in the input space, especially due to activation functions. This property has strong implications for adversarial robustness: even small, imperceptible perturbations to the input can significantly alter the logits $f_{\theta}(x)$, causing misclassification. These vulnerabilities motivate the study of adversarial attacks and defenses, which are discussed in subsequent sections.

2.2 Adversarial Examples and Attacks

2.2.1 Adversarial Examples

Adversarial examples are inputs intentionally modified to cause a neural network to make incorrect predictions despite the perturbations being small or even imperceptible to humans. Given an original input x , an adversarial example is constructed as

$$x' = x + \delta, \quad (2.6)$$

where δ is a carefully designed perturbation computed by an adversarial attack method. Although δ is typically very small in magnitude, it can significantly alter the model's output.

Threat model. We assume a white-box, untargeted adversary constrained by an ℓ_p norm. Perturbations satisfy

$$\|\delta\|_p \leq \varepsilon, \quad (2.7)$$

with $\varepsilon > 0$ the budget (typically $p \in \{\infty, 2\}$). The adversary solves an inner maximization to find $x_{\text{adv}} = x + \delta$ and we project/clip x_{adv} back to the valid input range.

Label Preservation. A key property of adversarial examples is label preservation, which ensures that the true semantic class of the input does not change under the perturbation. In other words, humans still perceive x and x' as belonging to the same class, even though the model may assign x' to a different label. This condition guarantees that the model is genuinely being fooled, rather than simply reacting to an input that has been visually or semantically altered.

Adversarial examples pose a significant challenge to the reliability of modern deep learning systems. Understanding their structure, the norms that constrain them, and the unified threat model above provides the foundation for analyzing attacks and designing robust defenses.

We focus on five attacks, organized by taxonomy, to cover single-step and iterative gradient-based methods (FGSM, PGD), optimization-based attacks (CW), decision-boundary attacks (DeepFool), and an ensemble stress test (AutoAttack).

2.2.2 Fast Gradient Sign Method (FGSM)

The Fast Gradient Sign Method (FGSM) is one of the earliest and most influential adversarial attacks in deep learning security. Introduced by Goodfellow et al. [2], FGSM demonstrated that a single gradient-based step is sufficient to produce small,

human-imperceptible perturbations that cause a neural network to misclassify the input. Despite being weaker than iterative attacks such as PGD [1], FGSM remains a foundational technique for understanding adversarial vulnerabilities and serves as a fast baseline for robustness evaluation.

Motivation. Before FGSM, adversarial example generation methods such as L-BFGS [3] were computationally expensive and difficult to scale to modern deep networks. Goodfellow et al. [2] observed that neural networks behave approximately linearly in high-dimensional input space, which allows small perturbations aligned with the gradient sign direction to induce large changes in the loss.

FGSM exploits this linearity by taking a single gradient ascent (or descent) step in the direction that increases the loss most rapidly. Although FGSM does not optimize the perturbation iteratively, it is extremely efficient and exposes the core mechanism of gradient-based adversarial attacks. It also played a crucial role in identifying gradient masking and motivated the development of stronger iterative methods like PGD.

Threat Model. FGSM operates under a white-box threat model, where the adversary has full access to the classifier’s parameters and gradients. The perturbation δ is constrained by the ℓ_∞ norm

$$\mathcal{S}_\infty(\varepsilon) = \{\delta \in \mathbb{R}^d : \|\delta\|_\infty \leq \varepsilon\},$$

where $\varepsilon > 0$ controls the maximum allowable distortion. The goal is to produce an adversarial example

$$x' = x + \delta$$

that changes the model’s prediction while keeping the perturbation imperceptibly small.

Objective. The objective of the Fast Gradient Sign Method (FGSM) is to construct an adversarial example that maximizes the classification loss under an ℓ_∞ -bounded perturbation constraint. Given a trained model f_θ , an input x with label y , and a loss function ℓ , FGSM seeks to approximate the solution of the following optimization problem

$$\max_{\delta} \ell(f_\theta(x + \delta), y) \quad \text{subject to} \quad \|\delta\|_\infty \leq \epsilon.$$

Instead of solving this problem iteratively, FGSM uses a first-order approximation of the loss and applies a single-step perturbation in the direction of the sign of the input gradient, providing a computationally efficient approximation of the inner maximization.

Untargeted FGSM. The untargeted attack seeks a perturbation δ that *maximizes* the loss with respect to the true label y

$$x' = \arg \max_{\|\delta\|_\infty \leq \varepsilon} \ell(f_\theta(x + \delta), y),$$

where $\varepsilon > 0$ controls the perturbation budget, $\|\cdot\|_\infty$ denotes the ℓ_∞ norm, and $\ell(\cdot, y)$ is the training loss evaluated with respect to the correct label y .

This maximization objective can be approximated by a single gradient ascent step

$$x' = x + \varepsilon \cdot \text{sign}(\nabla_x \ell(f_\theta(x), y)),$$

where $\nabla_x \ell(f_\theta(x), y)$ is the gradient of the loss with respect to the input.

Targeted FGSM. For the targeted attack, the goal is to *minimize* the loss toward a chosen target label y_{target}

$$x' = \arg \min_{\|\delta\|_\infty \leq \varepsilon} \ell(f_\theta(x + \delta), y_{\text{target}}),$$

where y_{target} denotes the attacker's desired misclassification target.

This leads to a gradient descent perturbation

$$x' = x - \varepsilon \cdot \text{sign}(\nabla_x \ell(f_\theta(x), y_{\text{target}})),$$

where the negative sign reverses the gradient direction to reduce the loss toward y_{target} .

Algorithm Description. FGSM performs a *single-step perturbation* consisting of:

1. **Gradient computation:** Compute the gradient of the loss with respect to the input

$$g = \nabla_x \ell(f_\theta(x), y).$$

2. **Sign extraction:** Use the element-wise sign of the gradient to determine the direction of maximal influence.
3. **Perturbation update:** Apply the perturbation by scaling the sign by ε . For the untargeted case, add the perturbation

$$x'_{\text{untargeted}} = x + \varepsilon \cdot \text{sign}(g),$$

for the targeted case, subtract it

$$x'_{\text{targeted}} = x - \varepsilon \cdot \text{sign}(g).$$

4. **Projection (clipping):** Ensure the adversarial example remains within valid pixel bounds

$$x' \leftarrow \text{clip}(x', 0, 1),$$

where the clipping operator projects x' onto the domain $[0, 1]^d$.

Key characteristics.

- Extremely fast (one forward pass and one backward pass).
- Often used as a sanity check for gradient quality.
- Commonly used to initialize stronger attacks (e.g., FGSM-random, PGD).
- Susceptible to gradient masking defenses.

Pseudo-code. Algorithm 1 summarizes the untargeted FGSM procedure. Inputs specify the model, loss, clean example and label, and the ℓ_∞ budget; the output is a clipped adversarial example produced by a single signed-gradient step.

Algorithm 1: FGSM ℓ_∞ Attack (Untargeted)

Input: classifier f_θ , loss ℓ , input x , label y , perturbation budget ε

Output: adversarial example x'

- 1 $g \leftarrow \nabla_x \ell(f_\theta(x), y)$
 - 2 $\delta \leftarrow \varepsilon \cdot \text{sign}(g)$
 - 3 $x' \leftarrow \text{clip}(x + \delta, 0, 1)$
 - 4 **return** x'
-

Role in this project. In this project, FGSM is used *exclusively for evaluation* as a fast and lightweight first-order baseline. Its roles include:

- Assessing the basic gradient sensitivity of the model.
- Demonstrating the difference between single-step and multi-step attacks.
- Providing a reference point for robustness comparison against PGD, C&W, DeepFool, and AutoAttack.
- Highlighting cases where models overfit to iterative attacks but still fail under simpler gradient-based perturbations.
- Supporting transfer-gap analysis by serving as one of the attack domains in robustness evaluation.

2.2.3 Projected Gradient Descent (PGD)

Projected Gradient Descent (PGD) [1] is a powerful first-order iterative adversarial method formulated within the robust optimization framework. It extends the single-step Fast Gradient Sign Method (FGSM) by applying multiple gradient-based updates while projecting each perturbed point back into the allowed perturbation region. PGD is often regarded as a strong first-order adversary since it explores the local loss landscape more thoroughly than single-step methods and can provide a practical approximation to worst-case perturbations under norm constraints.

Motivation. Traditional robustness evaluation using single-step attacks can underestimate the true worst-case vulnerability due to gradient masking or the highly non-linear loss landscape of modern neural networks. PGD mitigates these issues by applying an iterative first-order optimization procedure within a norm-bounded constraint set, progressively refining the perturbation over multiple steps to search for more damaging adversarial examples. Because it can be instantiated under different threat models (e.g., ℓ_∞ or ℓ_2) and objectives (untargeted or targeted), PGD serves as a standard baseline for both robustness evaluation and adversarial training.

Threat model. The PGD attack operates in a **white-box** setting, where the adversary has full access to the model’s parameters, architecture, and gradients. Under this threat model, the attacker seeks a small perturbation δ bounded by an ℓ_p -norm constraint that causes the model to misclassify the input. Formally, given a classifier f_θ and a threat set

$$\mathcal{S}_p(\varepsilon) = \{\delta : \|\delta\|_p \leq \varepsilon\},$$

adversarial examples take the form $x' = x + \delta$ with $\delta \in \mathcal{S}_p(\varepsilon)$. In addition, we enforce valid input constraints (e.g., pixel bounds) by restricting to

$$S = (x + \mathcal{S}_p(\varepsilon)) \cap [0, 1]^d,$$

and denote by $\Pi_S(\cdot)$ the projection operator onto the feasible set S .

Objective. Projected Gradient Descent (PGD) formulates adversarial example generation as an iterative inner optimization problem. Given an input x with label y and a loss function ℓ , the *untargeted* objective maximizes the classification loss within the threat set:

$$x' = \arg \max_{\|\delta\|_p \leq \varepsilon} \ell(f_\theta(x + \delta), y). \quad (2.8)$$

The *targeted* objective instead minimizes the loss with respect to a chosen target label $y_{\text{target}} \neq y$:

$$x' = \arg \min_{\|\delta\|_p \leq \varepsilon} \ell(f_\theta(x + \delta), y_{\text{target}}).$$

Untargeted PGD. To approximate Eq. (2.8), PGD performs iterative **gradient ascent** steps on the input (increasing the loss), followed by **projection** back to the feasible set to enforce the perturbation budget and valid input bounds. Under the ℓ_∞ threat model, the standard update takes the form

$$x^{(t+1)} = \Pi_S \left(x^{(t)} + \alpha \cdot \text{sign}(\nabla_x \ell(f_\theta(x^{(t)}), y)) \right), \quad (2.9)$$

where $x^{(t)}$ is the current adversarial iterate, $\alpha > 0$ is the step size, and $\text{sign}(\cdot)$ denotes the element-wise sign operator (appropriate for ℓ_∞ updates). The procedure runs for N iterations, returning $x^{(N)}$.

Algorithm. Algorithm 2 provides a concrete procedural realization of the projected update in Eq. (2.9). In practice, the projection operator $\Pi_S(\cdot)$ is implemented in two steps: (i) clipping the perturbation $\delta^{(t+1)} = \hat{x}^{(t+1)} - x$ to the ℓ_∞ ball of radius ε (budget enforcement), and (ii) clipping the resulting adversarial example to the valid input range $[0, 1]^d$. The initialization uses a *random start* by sampling $\delta^{(0)}$ uniformly from the ℓ_∞ threat set.

Algorithm 2: PGD ℓ_∞ Attack (Untargeted)

Input: classifier f_θ , loss $\ell(\cdot, y)$, clean input x , true label y , budget $\varepsilon > 0$, step size $\alpha > 0$, iterations N

Output: adversarial example x'

```

1 Initialize  $x^{(0)} \leftarrow x + \delta^{(0)}$  where  $\delta^{(0)} \sim \text{Uniform}(\mathcal{S}_\infty(\varepsilon))$ 
2 for  $t = 0$  to  $N - 1$  do
3    $g^{(t)} \leftarrow \nabla_x \ell(f_\theta(x^{(t)}), y)$ 
4    $\hat{x}^{(t+1)} \leftarrow x^{(t)} + \alpha \cdot \text{sign}(g^{(t)})$ 
5    $\delta^{(t+1)} \leftarrow \text{clip}(\hat{x}^{(t+1)} - x, -\varepsilon, +\varepsilon)$ 
6    $x^{(t+1)} \leftarrow \text{clip}(x + \delta^{(t+1)}, 0, 1)$ 
7 end
8 return  $x' \leftarrow x^{(N)}$ 

```

ℓ_2 PGD update. Under an ℓ_2 threat model, PGD uses the *normalized* gradient direction instead of the sign operator. The untargeted update becomes

$$x^{(t+1)} = \Pi_S \left(x^{(t)} + \alpha \cdot \frac{\nabla_x \ell(f_\theta(x^{(t)}), y)}{\|\nabla_x \ell(f_\theta(x^{(t)}), y)\|_2} \right), \quad (2.10)$$

where α is the step size and $\Pi_S(\cdot)$ enforces both the ℓ_2 budget $\|x^{(t)} - x\|_2 \leq \varepsilon$ and pixel bounds in $[0, 1]^d$.

Targeted PGD. The targeted variant follows the same iterative structure and feasibility constraints as the untargeted case, but replaces gradient ascent on the true-label loss by gradient **descent** on the target-label loss (i.e., reversing the sign of the update). Under the ℓ_∞ threat model, the update rule is

$$x^{(t+1)} = \Pi_S\left(x^{(t)} - \alpha \cdot \text{sign}(\nabla_x \ell(f_\theta(x^{(t)}), y_{\text{target}}))\right), \quad (2.11)$$

where the negative sign ensures the step decreases the target-label loss and encourages the model to predict y_{target} .

Algorithmic difference from untargeted PGD. Relative to Eq. (2.9), the targeted update in Eq. (2.11) modifies only two components: (i) the loss is evaluated with respect to y_{target} , and (ii) the gradient step uses descent instead of ascent. The projection operator $\Pi_S(\cdot)$ and the feasibility constraints remain unchanged.

Role in this project. In this project, PGD plays a central role. PGD-based adversarial training (PGD-AT) serves as the primary baseline in Stage 1, providing a strong and widely accepted reference for robustness under a fixed threat model. In addition, PGD is treated as a representative source attack domain in the attacks-as-domains formulation, enabling analysis of how robustness learned against a specific attack generalizes to other attack families. PGD is also used as a controlled evaluation attack for sensitivity analyses such as step-sweep experiments.

2.2.4 Carlini–Wagner (CW) Attack

Neural networks achieve state-of-the-art results across diverse tasks, yet they remain highly vulnerable to adversarial examples—subtle perturbations that induce misclassification while being imperceptible to humans [2, 3]. Defensive distillation emerged as a promising defense, reportedly slashing attack success rates from over 95% to just 0.5% on distilled models [4]. However, prior attacks like L-BFGS and FGSM often failed to fully probe this robustness, either due to inefficiency or suboptimal perturbations.

Motivation. The Carlini & Wagner (C&W) attacks [5] were developed to rigorously evaluate neural network robustness, particularly against defenses like distillation. Existing methods, such as L-BFGS (slow and non-scalable) or FGSM (fast but yielding large distortions), provided unreliable upper bounds on vulnerability, often failing on

hardened models. C&W introduces three tailored attacks for ℓ_0 , ℓ_2 , and ℓ_∞ norms that reliably find closer adversarial examples, demonstrating that distillation offers minimal protection and serving as a stronger benchmark for future defenses.

By reformulating adversarial generation as an optimization problem amenable to gradient descent, C&W achieves 100% success on both standard and distilled networks, outperforming predecessors in distortion metrics. Unlike gradient-sign methods that prioritize speed, C&W emphasizes minimality, enabling targeted misclassification even on large-scale datasets like ImageNet with imperceptible changes (e.g., flipping lowest-order bits).

Threat model. C&W operates in a white-box setting with full access to the model’s architecture, parameters, and gradients. Perturbations are constrained under ℓ_0 (pixel changes), ℓ_2 (Euclidean distance), or ℓ_∞ (maximum change) norms, without a fixed budget—focusing instead on minimal distortion. Attacks are targeted, selecting classes via average-case (random non-true label), best-case (easiest target), or worst-case (hardest target) strategies.

Objective. Given input x and target $t \neq C^*(x)$, C&W solves:

$$\min_{\delta} \|\delta\|_p + c \cdot f(x + \delta) \quad \text{s.t.} \quad x + \delta \in [0, 1]^d, \quad (2.12)$$

where $f(x') = \max(\max_{i \neq t} Z(x')_i - Z(x')_t, -\kappa)$ uses logits Z to encourage high-confidence misclassification (controlled by $\kappa \geq 0$), and $c > 0$ is tuned via binary search. Box constraints are handled via the change of variables $\delta_i = \frac{1}{2}(\tanh(w_i) + 1) - x_i$. Optimization employs Adam for ℓ_2/ℓ_∞ and iterative pixel fixing for ℓ_0 .

Algorithm description. The C&W attack family consists of seven variants ($\ell_0, \ell_2, \ell_\infty \times \text{targeted/untargeted}$), all sharing the same core principle: transform the constrained optimization problem of Eq. (2.12) into an unconstrained one that can be efficiently solved with first-order methods. The ℓ_2 attack is the flagship version — it is the fastest, most reliable, and produces the smallest Euclidean distortions; the original paper uses it for almost all experiments, including breaking defensive distillation on ImageNet.

The ℓ_2 attack proceeds as follows:

1. Introduce a new variable $w \in \mathbb{R}^d$ and enforce the box constraint $x + \delta \in [0, 1]^d$ via the differentiable change-of-variables

$$\delta_i = \frac{1}{2}(\tanh(w_i) + 1) - x_i.$$

This maps $w_i \in \mathbb{R}$ to $\delta_i \in [-x_i, 1 - x_i]$, guaranteeing validity.

2. For a chosen target class $t \neq y$, optimize the unconstrained Lagrangian

$$\min_w \left\| \frac{1}{2}(\tanh(w) + 1) - x \right\|_2^2 + c \cdot \ell(x + \delta(w), t),$$

where $\ell(x', t) = \max(\max_{i \neq t} Z(x')_i - Z(x')_t, -\kappa)$ and $Z(\cdot)$ are the pre-softmax logits.

3. Perform binary search over the constant $c > 0$ (typically 7–10 iterations) to find the smallest distortion that achieves misclassification.
4. Within each binary-search iteration, run Adam (or another gradient-based optimizer) on w for a fixed number of steps (usually 1000–10000) until $\ell(x + \delta, t) \leq 0$.
5. Optionally increase the confidence parameter κ (default $\kappa = 0$; $\kappa = 10$ –50 for high-confidence/transferrable examples).

The result is an adversarial example $x' = x + \delta$ that is misclassified as t with minimal ℓ_2 distortion.

Differences with ℓ_0 and ℓ_∞ variants.

- **ℓ_2 attack:** Uses the squared Euclidean distance $\|\delta\|_2^2$ directly in the objective (differentiable) and optimizes with Adam on the unconstrained variable w . Converges reliably and yields the smallest Euclidean perturbations.
- **ℓ_∞ attack:** Replaces $\|\delta\|_2^2$ with $c \cdot \|\delta\|_\infty$. Because the ℓ_∞ norm is non-differentiable, the authors instead optimize the same unconstrained Lagrangian but replace the distortion term with $c \cdot \max_i |\delta_i|$ and use the same tanh change-of-variables. Adam is still applied directly.
- **ℓ_0 attack:** The only non-differentiable and non-convex variant. Instead of optimizing a smooth norm, it iteratively
 1. Solves an ℓ_2 attack while penalizing all pixels,
 2. Identifies pixels with the smallest $|\delta_i|$ (least contributing),
 3. Fixes them to their original value (removes them from optimization),
 4. Repeats until the desired sparsity is achieved or misclassification occurs.

This greedy reduction yields the first published targeted ℓ_0 attack capable of fooling ImageNet models.

The ℓ_2 version is by far the most widely used and cited; it remains one of the strongest single-step white-box attacks even in 2025 when evaluating non- ℓ_∞ -robust models.

Role in this project. The Carlini & Wagner (C&W) ℓ_2 attack is used exclusively for evaluation, never for training. It serves as:

- A strong white-box baseline for minimal- ℓ_2 -distortion robustness, complementing the fixed- ε ℓ_∞ evaluations performed with AutoAttack.
- **One of the key attacks in our evaluation suite:** we systematically report $\text{Acc}_{\text{robust}}^{\text{C\&W}}$ alongside $\text{Acc}_{\text{robust}}^{\text{AA}}$, $\text{Acc}_{\text{robust}}^{\text{PGD-20}}$, and $\text{Acc}_{\text{robust}}^{\text{DeepFool}}$.
- The reference method for measuring average ℓ_2 distortion on non- ℓ_∞ -robust models and for cross-attack transfer analysis: a large gap $\Delta_{\text{transfer}}^{\text{PGD-20} \rightarrow \text{C\&W}}$ reveals that ℓ_∞ -based training does not generalize to ℓ_2 threats.
- The standard tool for historical reproduction: we use C&W to replicate the original 2017 results that broke defensive distillation [4] and several gradient-masking defenses [6].

By including C&W in our metric set, we ensure that robustness claims are not limited to the ℓ_∞ ball used during training. Models that achieve high $\text{Acc}_{\text{robust}}^{\text{AA}}$ but collapse under C&W (or require only tiny ℓ_2 perturbations) are immediately identified as having overly close or irregular decision boundaries — a core vulnerability that our Stage 2 domain-generalization approach explicitly targets.

2.2.5 DeepFool

Traditional methods for generating adversarial examples before 2016 suffered from major drawbacks: L-BFGS-based attacks [3] were extremely slow and did not scale to large datasets, while the Fast Gradient Sign Method (FGSM) [2] produced overly large perturbations that were far from minimal. As a consequence, neither approach provided a reliable way to quantify the *true* robustness of modern deep classifiers. Moreover, the lack of an efficient and accurate tool made it difficult to systematically compare different architectures or training strategies with respect to adversarial fragility.

Motivation. The key insight behind DeepFool is that measuring robustness requires computing **minimal** adversarial perturbations, not just any perturbation that fools the model. Prior attacks either sacrificed minimality for speed (FGSM) or sacrificed speed for accuracy (L-BFGS). DeepFool [7] bridges this gap by introducing a simple, iterative linearization procedure that rapidly converges to near-optimal perturbations while remaining computationally lightweight. By providing a reliable estimate of the distance to the decision boundary for every test sample, DeepFool enables the first large-scale, quantitative assessment of classifier robustness and becomes a foundational tool for understanding and improving adversarial robustness.

Threat model. DeepFool operates in a white-box setting with full access to the classifier’s architecture, parameters, and gradients. The original formulation [7] primarily targets ℓ_2 -norm bounded (or unbounded) perturbations, although extensions to

ℓ_∞ and ℓ_1 norms exist in later works. Unlike budget-constrained attacks, DeepFool does *not* fix a perturbation radius ε a priori. Instead, it explicitly searches for the *smallest* perturbation that crosses the decision boundary (untargeted attack) under the implicit threat set

$$\mathcal{S}_2 = \{\delta : \hat{k}(x + \delta) \neq \hat{k}(x)\},$$

where the admissible set is defined by the success condition itself rather than a pre-defined norm ball.

Objective. For a given correctly classified input x with predicted label $\hat{k}(x)$, DeepFool solves the following minimal-norm optimization problem

$$\Delta(x; \hat{k}) := \min_r \|r\|_2 \quad \text{subject to} \quad \hat{k}(x + r) \neq \hat{k}(x).$$

The resulting perturbation $r^*(x)$ is used both as an adversarial example and as a direct measure of local robustness. The average robustness of the classifier over a dataset is then defined as

$$\rho_{\text{adv}}(\hat{k}) = \mathbb{E}_x \left[\frac{\Delta(x; \hat{k})}{\|x\|_2} \right] = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \frac{\|r^*(x_i)\|_2}{\|x_i\|_2},$$

where lower values indicate higher vulnerability. This metric was the de facto standard for quantifying adversarial fragility from 2016 to 2019.

Key properties.

- **Near-optimality:** Provably optimal for affine classifiers; empirically yields perturbations within a few percent of the global minimum for deep networks.
- **Parameter-free:** No step size, number of iterations, or regularization hyperparameters — convergence is determined automatically by label change.
- **High efficiency:** Typically converges in 3–10 iterations (each requiring one forward + backward pass), orders of magnitude faster than L-BFGS or early C&W variants.
- **Iterative linearization:** Core mechanism (linearize $\rightarrow$ project $\rightarrow$ step $\rightarrow$ repeat) inspired numerous subsequent attacks (e.g., C&W, FAB, projective gradient methods).
- **Direct interpretability:** The magnitude of the computed perturbation directly reflects distance to the decision boundary in input space.

Assumptions.

1. **White-box access:** Full knowledge of and differentiability of the classifier f_θ (including gradients of logits or class probabilities).
2. **Differentiability:** The classification function must be locally approximable by its first-order Taylor expansion at each iteration point.
3. **Bounded inputs:** Inputs $x \in [0, 1]^d$ (or $[-1, 1]^d$ depending on preprocessing); during iteration, DeepFool optionally clips perturbations to remain in the valid pixel range (though not strictly enforced in the original version).
4. **No fixed perturbation budget:** Unlike PGD or AutoAttack, DeepFool has no predefined ε ; it stops as soon as misclassification occurs, naturally producing the smallest possible ℓ_2 perturbation.
5. **Untargeted attack:** The goal is to cause any misclassification, not to a specific target class (though targeted extensions exist).
6. **Label preservation under small noise:** The true label y remains semantically meaningful for perturbations of magnitude comparable to $\Delta(x; \hat{k})$, ensuring that misclassification reflects model fragility rather than domain shift.

Notation. Let $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^C$ be a differentiable classifier that outputs C logits (or class scores) $f_\theta(x) = [f_1(x), \dots, f_C(x)]$. The predicted label is $\hat{k}(x) = \arg \max_k f_k(x)$. At any point x_i during the attack, we denote $\ell(x_i) := \hat{k}(x_i)$ the current predicted class, and $\nabla f_k(x_i) \in \mathbb{R}^d$ the gradient of the k -th logit. All perturbations are computed in the original input space (no projection onto a fixed ε -ball is required).

DeepFool is built on a single, elegant mechanism—iterative local linearization of the classifier—applied in two settings:

Component 1: DeepFool for Binary Classifiers (theoretical foundation). For a binary classifier expressed as $\hat{k}(x) = \text{sign}(f(x))$ where $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is differentiable, the decision boundary is the zero-level set $\mathcal{F} = \{x : f(x) = 0\}$.

1. **Analytical solution for affine $f(x) = w^\top x + b$:** The minimal ℓ_2 -perturbation is the orthogonal projection onto the hyperplane

$$r^*(x_0) = -\frac{f(x_0)}{\|w\|_2^2} w. \quad (2.13)$$

2. **General differentiable case:** At iteration i , linearize f around the current estimate x_i

$$f(x) \approx f(x_i) + \nabla f(x_i)^\top (x - x_i). \quad (2.14)$$

The minimal perturbation for this surrogate affine classifier is exactly Eq. (2.13)

with $w = \nabla f(x_i)$

$$r_i = -\frac{f(x_i)}{\|\nabla f(x_i)\|_2^2} \nabla f(x_i).$$

Update $x_{i+1} \leftarrow x_i + (1 + \eta)r_i$ with a tiny overshoot $\eta \approx 0.02$ to ensure crossing the true nonlinear boundary.

The loop terminates as soon as $\text{sign}(f(x_i)) \neq \text{sign}(f(x_0))$.

Component 2: Extension to Multiclass Classifiers (practical algorithm).

Modern classifiers are multiclass and use the one-vs-all (max-logit) decision rule. DeepFool treats the multiclass problem as a sequence of binary sub-problems against the closest competing class.

At each iteration i :

1. Compute gradients $\nabla f_k(x_i)$ for all $k = 1, \dots, C$.
2. Let $\ell_i = \hat{k}(x_i)$ be the current predicted class.
3. For every class $k \neq \ell_i$, consider the affine surrogate function $g_k(x) := f_k(x) - f_{\ell_i}(x)$. The minimal perturbation to reach the hyperplane $g_k(x) = 0$ is

$$r_i^{(k)} = -\frac{f_k(x_i) - f_{\ell_i}(x_i)}{\|\nabla f_k(x_i) - \nabla f_{\ell_i}(x_i)\|_2^2} (\nabla f_k(x_i) - \nabla f_{\ell_i}(x_i)). \quad (2.15)$$

4. Choose the closest hyperplane (smallest perturbation)

$$k^* = \arg \min_{k \neq \ell_i} \frac{|f_k(x_i) - f_{\ell_i}(x_i)|}{\|\nabla f_k(x_i) - \nabla f_{\ell_i}(x_i)\|_2}. \quad (2.16)$$

5. Apply the step in the direction of k^* with slight overshoot

$$x_{i+1} \leftarrow x_i + (1 + \eta)r_i^{(k^*)}, \quad \eta = 0.02. \quad (2.17)$$

The algorithm stops when $\hat{k}(x_i) \neq \hat{k}(x_0)$. The final perturbation is $\hat{r} = x_{\text{final}} - x_0$.

Geometric interpretation. Each iteration moves to the closest point on the locally linearized decision boundary (orange line/hyperplane in Figures 2–3 of the original paper). Repeating this process traces an almost straight trajectory toward the true nonlinear boundary, explaining both the near-optimality and the extremely low number of required iterations (typically 3–10 for ImageNet models).

Convergence and overshoot. Empirically, the iterate often stops exactly on the true boundary. The small overshoot factor $(1 + \eta)$ guarantees that the final point lies on

the wrong side, making the attack 100% successful on correctly classified inputs while keeping the perturbation minimal.

Role in this project. DeepFool is used exclusively for evaluation and analysis, never for training our models. It serves as:

- The primary minimal- ℓ_2 -distortion attack and the original source of the average robustness metric ρ_{adv} — a standard complement to fixed- ε ℓ_∞ evaluations.
- A fast, near-optimal white-box baseline that reliably estimates the true Euclidean distance to the decision boundary for any correctly classified example.
- The reference method for geometric studies of classifier boundaries: its iterative trajectory approximates the shortest path in input space, revealing boundary curvature and linearity violations that fixed-budget attacks cannot expose.

By reporting both AutoAttack robust accuracy and DeepFool average distortion, we clearly separate bounded-threat robustness from minimal-distortion robustness. Models that survive strong ℓ_∞ attacks yet collapse under tiny DeepFool perturbations have extremely close or highly non-linear boundaries — a key vulnerability that motivates our focus on boundary regularization and geometry-aware training objectives.

2.2.6 AutoAttack

In many adversarial robustness evaluations, relying on a single attack (e.g., PGD) can yield overly optimistic robustness estimates due to suboptimal hyperparameters, gradient masking, or unsuitable loss functions [6]. To address this issue, AutoAttack [8] combines four complementary attacks with fixed hyperparameters: APGD-CE (white-box, cross-entropy loss), APGD-T (white-box, targeted attack using a DLR margin-based objective), FAB-T (boundary-based, targeted), and Square Attack (score-based, black-box). This parameter-free suite eliminates manual tuning and provides a reliable lower bound on robust accuracy, and it is widely used on RobustBench [9].

Motivation. A key insight underlying AutoAttack is that *no single attack reliably breaks all defenses*. In practice, gradient-based methods can fail under gradient masking, cross-entropy loss may saturate on highly confident models, and first-order methods may miss vulnerabilities near the decision boundary. Consequently, each attack mechanism can exhibit blind spots—regions where it fails to find adversarial examples. Importantly, even when different attacks achieve similar robust accuracy on the same model, they may succeed on different subsets of test examples. AutoAttack aggregates these complementary failure cases by marking an example as non-robust if *any* attack succeeds, which corresponds to taking the worst case over the set of attacks and yields a more reliable robustness assessment.

Threat model. AutoAttack evaluates robustness against norm-bounded perturbations with budget $\varepsilon > 0$ under the threat set

$$\mathcal{S}_p(\varepsilon) = \{\delta : \|\delta\|_p \leq \varepsilon\}, \quad p \in \{2, \infty\}.$$

The ensemble includes both **white-box** attacks (APGD-CE, APGD-T, FAB-T with full gradient access) and **black-box** attacks (Square Attack with score-only access). This combination covers complementary attack assumptions (gradient-based and score-based) commonly used in standardized robustness evaluation.

Objective. Given a test example (x, y) and a threat set $S = (x + \mathcal{S}_p(\varepsilon)) \cap [0, 1]^d$, AutoAttack aims to determine whether there exists an adversarial example $x' \in S$ such that $f_\theta(x') \neq y$. Rather than optimizing a single attack objective, AutoAttack applies a *fixed, parameter-free* suite of m attacks $\{\mathcal{A}_j\}_{j=1}^m$ sequentially and records an adversarial example as soon as misclassification is achieved.

Formally, each component produces a candidate $x'_j = \mathcal{A}_j(x, y) \in S$. AutoAttack then selects the stored adversarial example as

$$x'_{\text{AA}} = \begin{cases} x'_k, & \text{where } k = \min\{j \in \{1, \dots, m\} : f_\theta(x'_j) \neq y\}, \\ x, & \text{if } f_\theta(x'_j) = y \text{ for all } j \in \{1, \dots, m\}. \end{cases}$$

In other words, once a component succeeds on an example, that example is removed from subsequent attacks and the first successful adversarial candidate is retained; if none succeeds, the original input is kept.

Robust accuracy under AutoAttack is then computed as

$$\text{Acc}_{\text{robust}}^{\text{AA}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbb{1}[f_\theta(x'_{\text{AA},i}) = y_i],$$

i.e., the fraction of test examples that remain correctly classified after the AutoAttack procedure.

Assumptions.

1. **White-box access:** APGD-CE, APGD-T, and FAB-T have full access to model parameters θ and gradients $\nabla_x f_\theta(x)$.
2. **Black-box (score-based) access:** Square Attack requires only model outputs $z(x)$ without gradient information.
3. **Bounded inputs:** All inputs satisfy $x \in [0, 1]^d$, and adversarial examples are projected to this domain.

4. **Fixed perturbation budget:** All components use the same ε (e.g., $\varepsilon = 8/255$ for ℓ_∞ on CIFAR-10), ensuring fair comparison across attack types.
5. **Label preservation:** The true label y remains semantically correct for all $x' = x + \delta$ with $\|\delta\|_p \leq \varepsilon$, justifying that misclassification represents a genuine attack success rather than an invalid input.

Key properties.

- **Diversity:** APGD searches for any adversarial example within the ℓ_p -ball; FAB aims at (near) minimal-norm misclassification and can remain effective even under gradient masking; Square Attack is a score-based black-box method based on random search without any gradient approximation.
- **Worst-case over a set of attacks:** Diversity improves reliability because (i) certain defenses can cause some attacks to fail dramatically while others still succeed, and (ii) different attacks may succeed on different subsets of test examples even when achieving similar robust accuracy individually. AutoAttack therefore reports robustness under the worst case over the considered attacks, aggregating successful outcomes across the set.
- **Fixed hyperparameters (parameter-free in practice):** All attack hyperparameters are kept fixed across datasets, models, and norms, enabling a fully automatic evaluation without manual tuning.
- **Reliable and cost-efficient evaluation:** Despite a limited computational/query budget, AutoAttack provides a robust and reproducible robustness estimate in an automatic manner.

Algorithm. AutoAttack consists of four complementary components applied sequentially.

Component 1 & 2: APGD (Auto Project Gradient Descent). with 2 variants: APGD-CE and APGD-T. APGD is a refined PGD variant with three key innovations: (1) adaptive step-size scheduling that reduces η during optimization stagnation, (2) momentum in the input space to accelerate convergence, and (3) best-iterate tracking to avoid overshooting near decision boundaries. These design choices make APGD significantly more robust to hyperparameter choices than standard PGD, enabling AutoAttack’s parameter-free configuration.

APGD-CE (Cross-Entropy Loss). Uses standard cross-entropy

$$\ell_{\text{CE}}(x, y) = -\log p_\theta(y | x) = -z_y(x) + \log \sum_{j=1}^C \exp(z_j(x)),$$

which will be effective on most models but may saturate on adversarially trained models where $p_\theta(y | x)$ becomes extremely small, causing gradients $\nabla_x \ell_{\text{CE}}$ to become uninformative.

APGD-T (DLR Margin-Based Loss). Aiming to address gradient saturation, the Difference-of-Logits-Ratio (DLR) loss operates directly on logits

$$\ell_{\text{DLR}}(x, y) = -\frac{z_y(x) - z_{\max \neq y}(x)}{z_{(1)}(x) - z_{(3)}(x) + \varepsilon_{\text{stab}}},$$

where $z_{\max \neq y}(x) = \max_{i \neq y} z_i(x)$ and $z_{(i)}$ denotes the i -th largest logit. The numerator measures the margin between the true class and its strongest competitor while the denominator $z_{(1)}(x) - z_{(3)}(x)$ normalizes this margin by the spread of the top logits, preventing the loss from vanishing when all logits are large. Maximizing ℓ_{DLR} (i.e., making it less negative) pushes the model toward misclassification by shrinking or inverting the margin.

Algorithm overview. The APGD attack follows four main phases:

1. **Initialization:** Start from random $x^{(0)} \in S$ and perform one gradient step to obtain $x^{(1)}$. Track best iterate $(x_{\max}, f_{\max})$.
2. **Momentum update:** At each iteration k , compute gradient $g^{(k)} = \nabla_x \ell(x^{(k)}, y)$ and perform projected gradient step $z^{(k+1)}$. Combine with previous displacement using momentum parameter α to obtain $x^{(k+1)}$.
3. **Best-iterate tracking:** Update $(x_{\max}, f_{\max})$ whenever a new iterate achieves higher loss.
4. **Adaptive step-size:** At checkpoints $k \in W$, analyze recent loss trajectory. Define ϕ_k as the fraction of iterations in which loss increased. If $\phi_k < \rho$ (too few successful steps) or $f_{\max}$ has not improved since the last η reduction (stagnation), halve step size η and restart from $x_{\max}$.

Key properties:

- **White-box, first-order attack:** APGD assumes full access to model parameters and gradients, and iteratively updates the input via projected gradient steps within the ℓ_p -ball.
- **Projection-based feasibility:** Each iterate is explicitly projected onto the feasible set $S = (x + \mathcal{S}_p(\varepsilon)) \cap [0, 1]^d$, ensuring the final adversarial candidate respects both the norm budget and valid input range constraints.
- **Adaptive step-size schedule:** Unlike standard PGD with a fixed step size, APGD monitors optimization progress and reduces the step size when the loss stagnates, improving stability and reducing sensitivity to hyperparameter choices.
- **Momentum in input space:** APGD incorporates a momentum mechanism in

the update rule to accelerate convergence and help escape oscillations, which is particularly beneficial near decision boundaries.

- **Loss variants in AutoAttack:** AutoAttack uses multiple APGD instantiations (e.g., cross-entropy and DLR/margin-based losses, and optionally targeted variants) to reduce failure modes such as loss saturation and to increase attack diversity within the ensemble.

Component 3: FAB-T (Fast Adaptive Boundary). FAB [10] is a boundary-based attack that finds minimal-norm perturbations via iterative projection onto linearized decision boundaries. AutoAttack uses the **targeted variant FAB-T**, which considers only $K_{\text{targ}} = 9$ target classes (the top-9 non-true logits) rather than attacking all C boundaries, improving scalability.

Algorithm overview. For each target class $t \in \{1, \dots, K_{\text{targ}}\}$ (top-9 non-true classes), FAB-T performs the following:

1. **Initialization:** Start from a random point near x or from the best adversarial example found in previous restarts.
2. **Linearize boundary:** At current point $x^{(k)}$, compute:
 - Logit difference: $\Delta z = z_y(x^{(k)}) - z_t(x^{(k)})$
 - Gradient difference: $\Delta g = \nabla_x z_y(x^{(k)}) - \nabla_x z_t(x^{(k)})$
 where y is the true label and t is the target class. This defines a linear approximation of the decision boundary between classes y and t .
3. **Minimal-norm projection:** Compute the minimal ℓ_p -norm perturbation δ^* to reach the linearized boundary (closed-form solution for $p \in \{2, \infty\}$). This gives direction towards the boundary.
4. **Forward step with overshoot:** Move beyond the boundary using

$$x^{(k+1)} = x + \eta \cdot \delta^*, \quad \eta > 1$$

where $\eta = 1.05$ is the overshoot parameter. Overshoot ensures crossing the boundary even with linearization errors.

5. **Projection to feasible set:** Project back to the threat set

$$x^{(k+1)} \leftarrow \Pi_S(x^{(k+1)}), \quad S = x + \mathcal{S}_p(\varepsilon) \cap [0, 1]^d$$

6. **Update best adversarial:** If $f_\theta(x^{(k+1)}) \neq y$ and $\|x^{(k+1)} - x\|_p < \|x_{\text{best}} - x\|_p$, update $x_{\text{best}} \leftarrow x^{(k+1)}$.

7. **Backward step (optional):** To reduce perturbation norm, apply

$$x^{(k+1)} \leftarrow x + \beta \cdot (x^{(k+1)} - x), \quad \beta = 0.9$$

This ”pulls back” towards the original image while staying adversarial.

8. **Iterate:** Repeat for 100 iterations per target.

Typical configuration. For CIFAR-10 with ℓ_∞ ($\epsilon = 8/255$):

- $K_{\text{targ}} = 9$ target classes (top-9 non-true logits)
- 100 iterations per target $\rightarrow$ 900 total iterations
- $\alpha_{\text{max}} = 0.1$ (maximum step size)
- $\eta = 1.05$ (overshoot parameter)
- $\beta = 0.9$ (backward step parameter)
- Single restart (to save queries)

Key properties:

- **Boundary-focused:** Directly explores decision boundaries, complementing gradient-based attacks (APGD) that optimize loss.
- **Minimal perturbations:** By construction, FAB finds near-minimal-norm adversarial examples.
- **Targeted efficiency:** Considering only top-9 targets (instead of all $C - 1 = 9$ for CIFAR-10) reduces attack time while maintaining effectiveness.
- **Overshoot robustness:** $\eta > 1$ ensures crossing boundaries despite linearization errors.

FAB-T complements APGD by exploring decision boundaries more directly, capturing vulnerabilities that gradient ascent on loss functions might miss.

Component 4: Square Attack. Square Attack [11] is a gradient-free, score-based black-box attack using random local search over patches. It does not require gradients, making it immune to gradient masking and effective on obfuscated defenses.

Algorithm overview. Square Attack maintains the current best adversarial example (or equivalently perturbation) and iteratively proposes local square updates:

1. **Initialize:** Start from a random perturbation (e.g., random vertical stripes for ℓ_∞), and set the current best candidate x_{best} .
2. **Patch size schedule:** At query k , select a fraction p_k according to a piecewise schedule (large to small). In the official AutoAttack implementation, the query index can be *rescaled* to a $[0, 10000]$ axis (`resc_schedule`), so thresholds such as ”8000” refer to the rescaled progress rather than raw query count. The square side

length is then chosen as

$$s_k = \min\left(\max(\lfloor \sqrt{p_k d/c} \rfloor, 1), \min(h, w)\right),$$

where d is the input dimension and c is the number of channels.

3. **Propose modification:** Randomly select a square region of side s_k and apply a random update inside the patch (e.g., for ℓ_∞ set the patch perturbation to $\pm 2\varepsilon$), followed by clipping to the threat set and to $[0, 1]^d$.
4. **Query model and compute objective:** Evaluate logits $z(x)$ and compute an attack objective $J(x)$. In the implementation, J is defined so that *smaller is better* (e.g., for untargeted attacks, minimizing the logit margin $z_y - \max_{i \neq y} z_i$, or minimizing $-\text{CE}$ which is equivalent to maximizing cross-entropy).
5. **Accept/reject:** Accept the proposal if it improves the objective $J(x_{\text{new}}) < J(x_{\text{best}})$ or achieves misclassification $f_\theta(x_{\text{new}}) \neq y$ (equivalently margin ≤ 0 for the untargeted margin objective). Otherwise, keep the current best x_{best} .

Typical configuration. For CIFAR-10 with $\varepsilon = 8/255$ (ℓ_∞), Square Attack uses $Q = 5000$ queries per image. With `resc_schedule` enabled, the schedule thresholds are defined on a rescaled $[0, 10000]$ axis; thus, a line such as “8000+” corresponds to late-stage refinement (roughly after $\sim 0.8Q$ queries), not to more than Q queries. This gradual schedule enables coarse-to-fine exploration while maintaining query efficiency.

Key properties:

- **Gradient-free:** Only requires model outputs (logits/scores), not gradients.
- **Norm-agnostic:** Supports ℓ_∞ , ℓ_2 , and ℓ_1 perturbations in the AutoAttack codebase.
- **Robust to obfuscation:** Not affected by gradient masking.
- **Query-efficient:** Achieves strong attack success within a fixed query budget (e.g., $Q = 5000$ on CIFAR-10).

Role in this project. AutoAttack is used **exclusively for evaluation**, not training. It serves as:

- The **strongest benchmark** in our attack suite for final robustness assessment.
- **Gold-standard metric** for robust accuracy, widely adopted on RobustBench [9].
- Tool for **cross-attack transfer analysis** measuring how well models generalize to attacks outside the training distribution.
- **Baseline for Stage 2 evaluation:** We compare PGD-based training (Stage 1) against domain-generalization-enhanced training (Stage 2) using AutoAttack as the primary evaluation metric.

By comparing $\text{Acc}_{\text{robust}}^{\text{PGD}}$ (robust accuracy under the training attack) versus $\text{Acc}_{\text{robust}}^{\text{AA}}$

(robust accuracy under AutoAttack), we quantify the **robustness gap** motivating our domain-generalization approach. Models that achieve high $\text{Acc}_{\text{robust}}^{\text{PGD}}$ but low $\text{Acc}_{\text{robust}}^{\text{AA}}$ exhibit overfitting to the training attack—a key problem addressed by treating attack types as domains (Chapter 4).

From attacks to training and domains. The attacks above instantiate the inner maximization used in adversarial training and form the evaluation suite for Stage 1. In Chapter 2.3 we cast training as a min–max problem under this threat model, and in Chapter 2.4 (and Section 4.1) we reuse the same attacks as domains in the attacks-as-domains viewpoint adopted in Stage 2.

2.3 Adversarial Training

Adversarial Training (AT) formulates robustness against adversarial perturbations as a distributionally robust optimization problem. The core idea is to enforce model consistency not only on clean samples but also under worst-case perturbations constrained within a prescribed threat model. This section introduces the min–max formulation underlying AT, outlines the general AT procedure, and reviews the Projected Gradient Descent (PGD)-based adversarial training method popularized by Madry et al. [1].

2.3.1 Min–Max Formulation

Adversarial Training (AT) formulates robustness as a distributionally robust optimization problem. Rather than minimizing loss on the empirical data distribution alone, AT minimizes the worst-case loss over small perturbations of each training example. This section presents the min–max formulation underlying AT, explains the roles of the adversary and learner, and introduces a generic algorithmic framework that will be instantiated in Section 2.3.2.

Min–max objective. The robust training objective is expressed as

$$\min_{\theta} \mathbb{E}_{(x,y) \sim D_{\text{train}}} \left[\max_{\delta \in \mathcal{S}_p(\varepsilon)} \ell(f_{\theta}(x + \delta), y) \right], \quad (2.18)$$

where:

- $f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}^C$ is the classifier with parameters θ ,
- $\ell(\cdot, y)$ is the training loss function (typically cross-entropy),
- $\mathcal{S}_p(\varepsilon) = \{\delta : \|\delta\|_p \leq \varepsilon\}$ is the threat set defining allowable perturbations,
- D_{train} is the training dataset.

This formulation treats adversarial training as a two-player game: the inner

maximization represents an adversary seeking the worst-case perturbation, while the outer minimization represents the learner adapting to defend against it.

Inner maximization (adversary). For each training example (x, y) , the adversary solves

$$\delta^*(x, y, \theta) = \arg \max_{\|\delta\|_p \leq \varepsilon} \ell(f_\theta(x + \delta), y). \quad (2.19)$$

This inner maximization identifies a perturbation δ^* within the threat set that maximizes the loss of the current model on the perturbed input $x + \delta^*$. It represents the worst-case adversarial perturbation under a known threat model with norm p and budget ε .

Outer minimization (learner). Given the adversary’s worst-case perturbations, the learner updates the model parameters θ by minimizing the expected robust loss:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(x, y) \sim \mathcal{D}} [\ell(f_\theta(x + \delta^*(x, y, \theta)), y)].$$

In practice, the outer minimization is carried out using standard gradient-based optimizers (e.g., SGD or Adam) on adversarial examples generated by the inner loop. By repeatedly training on worst-case perturbations, the model is encouraged to learn representations that are stable within the specified threat set.

Key assumptions. The min–max formulation in Equation (2.18) relies on several standard assumptions commonly adopted in adversarial robustness literature:

1. **Known threat model:** The perturbation norm p and budget ε are fixed and will be given. In practice, common choices include ℓ_∞ with $\varepsilon = 8/255$ (CIFAR-10) or ℓ_2 with $\varepsilon = 0.5$.
2. **Label preservation:** The true label y remains semantically correct for all perturbed inputs $x + \delta$ with $\|\delta\|_p \leq \varepsilon$. This assumption justifies treating misclassification as a genuine failure rather than an artifact of invalid perturbations.
3. **Bounded adversary:** The adversary is constrained to $\mathcal{S}_p(\varepsilon)$ and cannot apply arbitrary transformations. This reflects realistic attack scenarios where perturbations must remain imperceptible or limited by physical constraints.
4. **Meaningful gradients:** The model f_θ provides informative gradients $\nabla_x \ell(f_\theta(x), y)$ throughout training, enabling gradient-based attacks to succeed. Defenses that induce gradient masking (e.g., obfuscated gradients [6]) violate this assumption and require adaptive attacks for reliable evaluation.
5. **Strong evaluation protocol:** Reliable robustness assessment requires diverse, parameter-free attacks such as AutoAttack [8] (Section 2.2.6) to avoid overly

optimistic estimates caused by weak attack configurations during evaluation.

Generic adversarial training framework. Algorithm 3 presents a generic instantiation of the min–max objective. At each training step, the algorithm generates adversarial examples by approximately solving the inner maximization, then updates model parameters by minimizing loss on these adversarial examples.

Algorithm 3: Generic Adversarial Training Framework

Input: Training set D ; model f_θ ; attack method $\mathcal{A}$; loss ℓ ; optimizer;
epochs T ; learning rate η_{lr} .

Output: Robust model f_θ

```

1 for  $t = 1$  to  $T$  do
2   foreach mini-batch  $\mathcal{B} \subset D$  do
3     foreach  $(x, y) \in \mathcal{B}$  do
4        $\delta^* \leftarrow \mathcal{A}(x, y, f_\theta, \varepsilon)$            // Inner max: generate perturbation
5        $x' \leftarrow x + \delta^*$ 
6     end
7      $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{(x, y) \in \mathcal{B}} \ell(f_\theta(x'), y)$            // Compute robust loss
8      $\theta \leftarrow \theta - \eta_{lr} \nabla_\theta \mathcal{L}$            // Outer min: update parameters
9   end
10 end
11 return  $f_\theta$ 

```

Remark. This framework is included as an explanatory template; the concrete training pipelines used in Stage 1 are reported in Chapter 5.

Transition to concrete instantiation. The generic framework in Algorithm 3 leaves open the choice of attack method $\mathcal{A}$ for approximating the inner maximization. Section 2.3.2 describes the widely-adopted instantiation proposed by Madry et al. [1], which uses multi-step Projected Gradient Descent (PGD) as the attack and forms the basis of our Stage 1 baseline.

2.3.2 Madry’s PGD Adversarial Training

The min–max formulation in Equation (2.18) was instantiated by Madry et al. [1], who approximated the inner maximization using multi-step Projected Gradient Descent (PGD). This section describes the PGD attack mechanism, explains why it serves as an effective surrogate for the worst-case adversary, and presents the complete Madry PGD-AT algorithm adopted as our Stage 1 baseline.

PGD as the inner-maximization solver. To approximate the inner max in Equation (2.19), Madry et al. employ PGD (Section 2.2.3), which iteratively ascends the loss landscape with respect to the input while projecting perturbations back onto the

threat set:

$$\delta^{(k+1)} = \Pi_{\mathcal{S}_p(\varepsilon)}(\delta^{(k)} + \alpha \cdot \text{sign}(\nabla_x \ell(f_\theta(x + \delta^{(k)}), y))),$$

where $\Pi_{\mathcal{S}_p(\varepsilon)}(\cdot)$ denotes projection onto the ℓ_p -ball of radius ε , α is the step size, and $\text{sign}(\cdot)$ is the element-wise sign function (for ℓ_∞ attacks).

Why PGD? PGD is considered the **universal first-order adversary** for several reasons:

- **Multi-step exploration:** Unlike single-step FGSM, PGD performs multiple gradient ascent steps, allowing it to explore the local loss landscape more thoroughly and find stronger perturbations near local maxima.
- **Random initialization:** Starting from a random point within the threat set (e.g., uniform sampling from $\mathcal{S}_p(\varepsilon)$) helps avoid early convergence to weak local maxima.
- **Projection ensures feasibility:** Each iterate remains within the threat set $\mathcal{S}_p(\varepsilon) \cap [0, 1]^d$, guaranteeing that the final adversarial example satisfies the perturbation budget and pixel-range constraints.
- **Empirical strength:** Extensive evaluations show that PGD with sufficient iterations (e.g., 20–50 steps) reliably approximates the worst-case first-order adversary, making it a strong proxy for Equation (2.19) in practice.

Concrete setup. The Madry PGD-AT configuration for CIFAR-10, following the original implementation and its configuration¹

- **Threat model:** ℓ_∞ with $\varepsilon = 8/255 \approx 0.031$.
- **Training attack:** PGD-10 with $\alpha = 2/255$, random initialization.
- **Evaluation attack:** PGD-20 with $\alpha = 2/255$.
- **Architecture:** WideResNet-28-10 (~ 36.5 M parameters).
- **Data augmentation:** Random horizontal flips and 4-pixel random crops.
- **Optimizer:** Momentum Optimizer (momentum 0.9, weight decay 2×10^{-4}).
- **LR schedule:** Step decay: 0.1 (0–40K steps), 0.01 (40K–60K), 0.001 (60K–80K).
- **Training:** 80,000 steps (≈ 200 epochs), batch size 128.

Pseudo-code. Algorithm 4 presents the complete Madry PGD Adversarial Training procedure. For each mini-batch, the algorithm generates adversarial examples using PGD (lines 5–10), then performs a standard gradient descent step (line 12).

¹Code of original paper at: https://github.com/MadryLab/cifar10_challenge/blob/master/config.json

Algorithm 4: Madry PGD Adversarial Training

Input: Training set $\mathcal{D} = \{(x_i, y_i)\}$; epochs T ; PGD steps K ; step size α ; budget ε .

Output: Robust model parameters θ

```
1 Randomly initialize  $\theta$ 
2 for  $t = 1$  to  $T$  do
3   foreach mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
4     foreach  $(x, y) \in \mathcal{B}$  do
5       Initialize  $\delta^{(0)} \sim \text{Uniform}(\mathcal{S}_\infty(\varepsilon))$  // Random init
6       for  $k = 0$  to  $K - 1$  do
7          $g \leftarrow \nabla_x \ell(f_\theta(x + \delta^{(k)}), y)$ 
8          $\delta^{(k+1)} \leftarrow \delta^{(k)} + \alpha \text{sign}(g)$ 
9          $\delta^{(k+1)} \leftarrow \Pi_{\mathcal{S}_\infty(\varepsilon)}(\delta^{(k+1)})$  // Project to  $\ell_\infty$  ball
10         $\delta^{(k+1)} \leftarrow \text{clip}(\delta^{(k+1)}, -x, 1 - x)$  // Ensure  $x + \delta \in [0, 1]^d$ 
11      end
12       $x' \leftarrow x + \delta^{(K)}$  // Adversarial example for this sample
13    end
14     $\theta \leftarrow \theta - \eta_{\text{lr}} \nabla_{\theta} \frac{1}{|\mathcal{B}|} \sum_{(x, y) \in \mathcal{B}} \ell(f_\theta(x'), y)$  // Update adversarial batch
15  end
16 end
17 return  $\theta$ 
```

Remark. The report’s Stage 1 experiments use standardized implementations and controlled settings documented in Chapter 5.

Computational cost. Generating adversarial examples via PGD- K requires K forward–backward passes per training sample, making Madry AT approximately $(K + 1)$ times more expensive than standard training. For PGD-10, this translates to roughly $11\times$ computational overhead. Despite this cost, Madry AT remains the most widely adopted robust training method due to its simplicity, effectiveness, and strong empirical performance.

Role in this project. Madry PGD-AT serves as the **Stage 1 baseline** in our two-stage training framework (Chapter 4). In Stage 1, we train a robust model using standard PGD-AT on CIFAR-10 under the ℓ_∞ threat model. This model provides the initialization for Stage 2, where we apply domain generalization techniques to improve cross-attack transfer and robustness against unseen attack types. All Stage 1 models are evaluated using PGD-20 (validation) and AutoAttack (final robustness benchmark, Section 2.2.6).

Limitations. While Madry PGD-AT achieves strong robustness against PGD attacks under the training threat model, it exhibits several known limitations:

- **Accuracy-robustness trade-off:** Robust models typically suffer 5–10% drops in clean accuracy compared to standard models.
- **Single-threat-model brittleness:** Models trained on ℓ_∞ PGD may remain vulnerable to ℓ_2 attacks or other attack families (e.g., C&W, DeepFool).
- **Overfitting to training attack:** Robust accuracy often degrades when evaluated with stronger attacks (e.g., AutoAttack) compared to the training attack (PGD-10), indicating incomplete robustness.

These limitations motivate our Stage 2 approach, which treats different attack types as distinct domains and applies domain generalization methods to improve robustness generalization across the attack landscape.

2.3.3 Variants of Adversarial Training

The original Madry PGD adversarial training [1] remains the most widely recognized baseline due to its simplicity and strong empirical performance under the ℓ_∞ threat model. However, it suffers from high computational cost, degraded clean accuracy, and limited robustness against unseen attack types.

Subsequent works have significantly improved upon Madry’s formulation along three principal directions:

- **Improved accuracy–robustness trade-off.** TRADES [12] introduces a regularization term that explicitly balances natural and robust error, consistently outperforming vanilla PGD-AT on both clean and robust accuracy.
- **Training efficiency.** Fast Adversarial Training (Fast AT) [13] demonstrates that single-step FGSM with random initialization, combined with early stopping, can achieve comparable ℓ_∞ robustness to multi-step PGD while reducing training time from $\sim 10\times$ to only $\sim 2\times$ standard training.
- **Better generalization across attacks and norms.** Adversarial Weight Perturbation (AWP) [14] perturbs both inputs and model weights during training, enforcing flatness in the weight-loss landscape. When combined with TRADES, it yields state-of-the-art ℓ_∞ robustness and substantially improves performance against ℓ_2 -bounded attacks (e.g., C&W, DeepFool) and AutoAttack.

Despite these advances, all aforementioned methods train against a *single* attack family (typically ℓ_∞ PGD or FGSM variants) and exhibit robustness overfitting when evaluated on diverse attack distributions. In this project, we retain the original Madry PGD-AT as our Stage 1 baseline to ensure direct comparability with the vast majority of prior work while clearly exposing the cross-attack generalization gap that motivates our Stage 2 domain-generalization approach.

2.4 Domain Shift and Domain Generalization

2.4.1 Problem Setup and Taxonomy

Modern machine learning systems are typically trained under the independent and identically distributed (i.i.d.) assumption: training and test samples are assumed to be drawn from the same distribution P_{XY} [15]. In practice, this assumption is frequently violated. When the joint distribution at test time, denoted $P_{\text{test}}(X, Y)$, differs from that at training time, $P_{\text{train}}(X, Y)$, we say that a *domain shift* occurs, and standard empirical risk minimization may fail to generalize.

Domain Generalization (DG) seeks to learn models that remain reliable under such shifts, even when target-domain data are unavailable during training [15]. Formally, we consider a collection of source domains

$$\mathcal{D}_{\text{source}} = \{\mathcal{D}_i\},$$

each associated with a joint distribution $P_i(X, Y)$. During training, we observe datasets

$$\mathcal{D}_i = \{(x_k^{(i)}, y_k^{(i)})\}$$

sampled from the corresponding P_i . At test time, we evaluate on one or more target domains $\mathcal{D}_{\text{target}}$ with distributions $P_{\text{target}}(X, Y)$ that may differ from every P_i . The goal is to learn parameters θ such that the classifier f_θ achieves low risk on unseen domains

$$\mathbb{E}_{(x,y) \sim P_{\text{target}}} [\ell(f_\theta(x), y)]$$

for a broad family of plausible target distributions P_{target} .

In this report, we follow the taxonomy of DG algorithms proposed in recent surveys [15], which group methods into three main categories:

- **Data manipulation** – alter the input data distribution by augmenting or generating samples that simulate domain shift.
- **Representation learning** – learn domain-invariant or disentangled features $\phi(x)$ such that a classifier on top of $\phi(x)$ generalizes across domains.
- **Learning strategy** – modify the training objective, optimization procedure, or model ensemble to directly optimize for cross-domain performance.

The remainder of this section reviews these three categories with an eye toward treating adversarial attack families as domains in later chapters.

2.4.2 Data Manipulation for Domain Generalization

Data manipulation methods aim to expose the model to a wide variety of domain variations during training by transforming the observed data. Instead of relying solely on the finite set of source domains $\{\mathcal{D}_i\}$, these methods construct additional “virtual domains” that approximate potential target shifts [15]. Below we outline several directions; detailed algorithms, examples, and empirical results can be filled in subsequently.

(a) Data augmentation and domain randomization (outline).

- Interpret classical augmentation operations (random crops, flips, color jittering, Gaussian noise, etc.) as sampling from a family of perturbed distributions $\{P_i(X, Y)\}$ that preserve label semantics.
- Describe *domain randomization* as an extension that aggressively perturbs styles, textures, or backgrounds (possibly producing unrealistic images), with the hypothesis that robustness to diverse low-level distortions improves generalization to real-world target domains.

(b) Adversarial and gradient-based augmentation (outline).

- Introduce augmentation schemes that move inputs along gradient directions of the loss, in analogy to adversarial examples, but not necessarily maximizing the loss exactly.
- Explain how such methods can enlarge feature variance or simulate domain-specific shifts by conditioning perturbations on domain labels or style statistics.
- Position classical adversarial training (e.g., PGD-AT [1]) as a special case in which each attack $\mathcal{A}$ defines a new domain $\mathcal{D}_i$, while noting that standard AT typically concentrates on a *single* attack domain rather than promoting robustness across many attack families.

(c) Synthetic domain generation and style transfer (outline).

- Outline how generative models (GANs, VAEs, etc.) can be used to generate transformed versions x' of a clean sample x that mimic the style of different domains (e.g., paintings, sketches, weather or illumination changes).
- Discuss style-transfer and feature-statistics-based methods (e.g., replacing mean/variance of $\phi(x)$ with those from another domain, as in MixStyle-like approaches) that perturb either pixel space or early-layer feature maps while preserving semantic content.

- Emphasize that these operations effectively sample from synthetic distributions $P_j(X, Y)$ intended to approximate realistic domain shifts in style.

(d) Strengths, limitations, and relevance to adversarial robustness (outline).

- Summarize the practical advantages of data manipulation: easy integration into existing pipelines and no need for target-domain samples or labels at test time.
- Highlight limitations, including the lack of guarantees that synthetic domains adequately cover the true target distribution, and the risk that overly aggressive augmentation may harm performance if generated samples move far outside the realistic support of $P_{\text{target}}(X, Y)$.
- Connect to adversarial robustness by:
 - Viewing adversarial attacks themselves as structured augmentations, where each attack $\mathcal{A}_i$ induces a domain $\mathcal{D}_i$.
 - Proposing hybrid schemes that combine attack-based augmentation with style- or feature-based perturbations (e.g., Mixup, MixStyle) to create a richer spectrum of adversarial domains that stress-test the decision boundary of f_θ .

2.4.3 Representation Learning

Representation learning in Domain Generalization (DG) decomposes the predictive function h into a feature extractor g and classifier f ($h = f \circ g$). The objective is to learn a representation space where domain discrepancies are minimized while preserving task-relevant semantics, enabling generalization to unseen distributions. Existing methods can be grouped into three main lines: domain-invariant learning, disentangled/causal representations, and their implications for cross-attack robustness.

(a) Domain-invariant representation learning. These methods enforce statistical invariance across source domains so the model does not rely on unstable, domain-specific patterns.

Explicit alignment. Distance-based regularizers align the feature distributions of different domains. Typical metrics include Maximum Mean Discrepancy (MMD), which compares distributions in an RKHS, and CORAL, which matches second-order covariance statistics. This encourages g to filter out domain-specific variations.

Adversarial alignment. Inspired by GANs, a domain discriminator competes against the feature extractor. The discriminator predicts domain indices, while the extractor is optimized to confuse it. At equilibrium, features contain minimal domain-identifying information.

Invariant Risk Minimization (IRM). Rather than aligning marginals, IRM seeks a representation for which the optimal classifier is identical across environments. By

penalizing the gradient of the loss w.r.t. the classifier, IRM encourages the model to rely on causal, stable predictive mechanisms and ignore spurious correlations.

(b) Feature disentanglement and causal DG. These approaches explicitly model the data-generation process to isolate invariant semantic factors.

Feature disentanglement. Samples are assumed to arise from domain-invariant (content) and domain-specific (style) latent factors. VAE- or GAN-based frameworks separate these components so the classifier uses only the invariant representations, reducing negative transfer.

Causal DG. Domain shifts are interpreted as interventions on non-causal variables. According to the principle of independent causal mechanisms, the functional relationship between causal features and the label remains invariant even when non-causal factors change. Causal DG therefore aims to recover the true causal parents of the target, yielding theoretically grounded robustness.

(c) Implications for cross-attack robustness. The principles above naturally extend to adversarial robustness when different attacks are viewed as distinct “domains.” Learning domain-invariant features corresponds to learning perturbation-invariant object features, while disentanglement separates clean content from adversarial noise. As a result, DG-inspired representation learning enables models to defend not only against seen attacks but also to generalize to unseen adversaries, reducing overfitting to specific attack signatures.

2.4.4 Learning Strategy

Learning-strategy approaches modify the training objective, optimization dynamics, or model ensemble to directly target cross-domain performance, often on top of standard augmentation or representation-learning techniques.

(a) Distributionally robust optimization: REx and GroupDRO. Risk Extrapolation (REx) and Group Distributionally Robust Optimization (GroupDRO) treat domains as groups and optimize for performance under worst-case or variance-sensitive risk.

Given empirical risks $R_i(\theta)$ on domains $\mathcal{D}_i$, REx penalizes variance

$$\min_{\theta} \sum_i R_i(\theta) + \lambda \text{Var}_i[R_i(\theta)],$$

encouraging solutions whose performance is similar across domains. GroupDRO, on the

other hand, optimizes a worst-case objective

$$\min_{\theta} \max_i R_i(\theta),$$

ensuring that no domain is catastrophically under-served. When domains correspond to attack families, REx and GroupDRO naturally translate into objectives that reduce robust-accuracy variance

$$\text{Acc}_{\text{robust}}^{\mathcal{A}_i}$$

and improve worst-case robustness across attacks.

(b) Flat-minima and weight-space strategies: SWAD, SAM. Another line of work seeks parameter regions where the loss remains low under perturbations of θ , motivated by the link between flat minima and generalization. Sharpness-Aware Minimization (SAM) perturbs parameters within a small ball and optimizes for the worst-case loss in this neighbourhood. Stochastic Weight Averaging in Domain Generalization (SWAD) averages weights along the training trajectory, explicitly targeting flat basins that perform well across domains. When combined with adversarial training, these strategies can produce models whose robust accuracy is less sensitive to small changes in attack configuration or initialization.

(c) Mixup-style strategies across domains. Mixup and MixStyle interpolate either inputs and labels or feature statistics across different domains. In the DG setting, mixing samples from different $\mathcal{D}_i$ encourages f_{θ} to behave smoothly not only within domains but also along the paths between them. For attacks-as-domains, we can generate mixed adversarial examples—e.g., interpolating between PGD and CW perturbations—or mix feature statistics of different attacks in early layers. This has the potential to create a continuum of attack domains, mitigating over-specialization to any single one.

(d) When to use which strategy. Broadly, distributionally robust objectives (REx, GroupDRO) are well-suited when we care about worst-case attack performance; flat-minima methods (SWAD, SAM) are attractive when training stability and broad generalization are priorities; and mixup-style strategies are simple yet powerful regularizers that enhance diversity of attack patterns. Stage 2 will explore a small set of these methods in combination with PGD-AT, trading off complexity and computational cost.

2.4.5 From Classical DG to Attacks-as-Domains

Traditional DG has predominantly focused on distribution shifts such as dataset bias, style variation, or sensor differences between source and target domains [15]. In this report, we extend the DG viewpoint to *adversarial domains*, where each attack $\mathcal{A}_i$ induces a domain $\mathcal{D}_i$ with associated joint distribution $P_i(X, Y)$ over perturbed inputs. Our Stage 1 results (Chapter 5) reveal significant gaps in robust accuracy $\text{Acc}_{\text{robust}}^{\mathcal{A}_i}$ across attacks even for strong PGD-based adversarially trained (PGD-AT) models [1], suggesting that robustness does not automatically generalize across attack domains.

The taxonomy reviewed above provides a toolbox of mechanisms—data manipulation, representation learning, and learning strategies—that can be systematically adapted to the attacks-as-domains setting. In the next chapter, we survey concrete methods in each category and highlight existing attempts to jointly leverage adversarial robustness and domain generalization, as well as gaps where combining these perspectives remains under-explored.

2.5 Chapter Summary

In this chapter we fixed the notation for supervised classification and deep neural networks, formalized adversarial examples and threat models, and described the main attacks used in our evaluation suite. We then presented the min–max formulation of adversarial training and the PGD-based adversarial training procedure that serves as our Stage-1 baseline. Finally, we introduced domain generalization and defined evaluation metrics that treat each attack as a domain, including robust accuracy, robustness gaps, cross-attack transfer matrices, and aggregate robustness measures. This foundation enables the literature review in Chapter 3 and the design and analysis of experiments in later chapters.

Chapter 3

Related Work

This chapter situates our work within the broader literature on adversarial robustness and domain generalization. We first review key developments in adversarial training and robustness evaluation, then survey domain generalization methods in computer vision. Finally, we discuss conceptual and methodological connections between adversarial robustness and DG, and identify the research gap addressed by this report.

3.1 Adversarial Robustness and Adversarial Training

Overview of Adversarial Training. Adversarial training has emerged as one of the most effective defenses against adversarial examples. Early approaches augmented training data with FGSM-based perturbations [2], improving robustness to simple attacks but remaining vulnerable to stronger multi-step attacks. Subsequent work adopted the min–max formulation described in Section 2.3.1, using stronger inner maximization procedures and careful training protocols.

PGD-based adversarial training popularized by Madry et al. [1] has become a standard baseline for robustness. By iteratively updating perturbations using gradient ascent and projection, PGD provides a strong first-order approximation to the inner maximization, and models trained with PGD-AT often define state-of-the-art robustness under the training threat model.

Objective Variants and Robustness–Accuracy Trade-Off. A number of works have explored objective variants, regularization schemes, and alternative losses to mitigate the robustness–accuracy trade-off observed in adversarial training. Examples include margin-based losses, confidence-calibrated losses, and hybrid objectives that mix natural and adversarial examples. These methods often adjust the balance between clean accuracy and robustness, but do not fundamentally change the fact that robustness tends to be specific to the threat model and attack configuration used for training.

Data Scale, Robust Overfitting, and Training Protocol. Subsequent studies have investigated the impact of data scale, training duration, and regularization on adversarial robustness. Phenomena such as *robust overfitting* [16], where robust accuracy on the training attack improves on the training set but degrades on the test set over extended training, highlight the importance of carefully chosen training schedules and early stopping criteria.

In our Stage-1 experiments, we observe diminishing returns in robustness when increasing the number of training epochs beyond a moderate threshold (e.g., 50 epochs), while incurring substantial increases in computational cost. These findings align qualitatively with prior work on robust overfitting and training-time trade-offs.

Multi-Attack Robustness and Evaluation Best Practices. Beyond single-attack robustness, several works emphasize the importance of evaluating models against multiple attacks, including attacks with different norms, different loss functions, and different implementation details [17]. Ensemble attacks such as AutoAttack [8] have been proposed to provide reliable, parameter-free robustness estimates and to reduce the risk of inadvertently underestimating vulnerability due to weak attack configurations.

Our Stage-1 pipeline follows these best practices by including a diverse attack suite and by treating AutoAttack as a primary evaluation metric. The observed gaps between robustness to the training PGD attack and robustness to AutoAttack motivate our attacks-as-domains perspective.

3.2 Domain Generalization

Overview and Taxonomy. Domain generalization aims to train models that perform well on unseen target domains, given access only to one or more source domains during training. In computer vision, domain shifts may arise from changes in style, viewpoint, imaging conditions, or dataset collection procedures. Comprehensive surveys organize DG methods into categories of data manipulation, representation learning, and learning strategy [15]. We adopt this taxonomy and focus on methods that are particularly relevant to our attacks-as-domains framework.

Data Manipulation–Based Domain Generalization. Data manipulation methods modify the training data distribution to expose models to a richer set of domain variations. Techniques include style randomization, domain-specific augmentations, and Mixup-like interpolations. Mixup generates virtual training examples by convexly combining pairs of inputs and labels. This technique has been shown to improve generalization and robustness in a variety of settings. In our context, Mixup suggests cross-domain strategies that interpolate between adversarial examples from different

attack domains, potentially smoothing decision boundaries across attacks.

Representation Learning–Based Methods. Representation-learning-based DG methods seek representations $\phi(x)$ that capture features invariant across domains while discarding spurious or domain-specific factors. IRM (Invariant Risk Minimization) is a prominent example: it aims to find a representation ϕ such that there exists a classifier w that is simultaneously optimal across all environments, enforced via a gradient-based penalty on per-environment risks [18]. Other methods include domain-adversarial training and contrastive approaches that align feature distributions across domains. These techniques highlight the potential of learning domain-invariant representations, a principle we adapt by considering attacks as domains in Stage 2.

Learning Strategy–Based Methods. Learning-strategy methods modify the training objective or model selection criteria to prioritize cross-domain performance. Group Distributionally Robust Optimization (GroupDRO) minimizes the worst-group risk by up-weighting high-loss domains, while Risk Extrapolation (REx) penalizes the variance of risk across domains, encouraging balanced performance.

These methods directly operate at the level of domain-specific risks $R_e(\theta)$, making them particularly well-suited to our formulation where each attack family defines a domain. In Stage 2, we adapt GroupDRO and REx as loss-based DG strategies on top of multi-attack adversarial training.

3.3 Bridging Adversarial Robustness and Domain Generalization

Research on *adversarial robustness*—the resilience of models against intentionally crafted perturbations—and *domain generalization* (DG)—the ability to generalize to unseen target domains—shares deep conceptual connections. Fundamentally, both lines of work aim to construct neural networks that maintain stability under undesirable distribution shifts: adversarial perturbations in the former case, and domain shifts in the latter. However, the relationship between these two objectives is not merely reciprocal. While they share a common goal, existing literature reveals a complex landscape where robustness and generalization can exhibit either synergistic effects or inherent conflicts depending on the method and data assumptions.

Theoretical and empirical analyses of the robustness–generalization relationship.

A primary area of investigation focuses on determining whether the objectives of robustness and generalization are fundamentally aligned or at odds.

From a theoretical perspective, Stutz et al. [19] analyze the geometric nature of adversarial examples relative to the data manifold. They demonstrate that the majority of standard adversarial examples lie off the data manifold, whereas “on-manifold” adversarial examples can be interpreted simply as standard generalization errors. This distinction suggests that the interplay between robustness and generalization is heavily dependent on the underlying manifold assumptions and the type of perturbation applied.

Complementing this theoretical view, recent empirical studies systematically evaluate the trade-offs involved. For instance, Gokhale et al. [20] conduct a comprehensive comparison of preprocessing strategies and data augmentation techniques across both tasks. Their results indicate a disjointed benefit: many techniques that significantly enhance out-of-distribution (OOD) generalization do not yield improvements in adversarial robustness, and vice versa. These findings highlight the difficulty of achieving both objectives simultaneously without a unified framework.

Adversarial training as a mechanism for improving DG. A substantial body of research views adversarial training (AT) not just as a defense mechanism, but as a potent tool for data augmentation to enrich the training distribution, thereby facilitating better transfer to unseen domains.

Early work by Song et al. [21] proposes the Adversarial Training with Domain Adaptation (ATDA) framework. In this approach, adversarial examples are treated as a proxy for a “target domain” within a domain adaptation setting. By enforcing invariance between clean source data and their adversarial counterparts, ATDA aims to simultaneously improve robustness against perturbations and generalization across domains.

Expanding on this, more recent studies integrate AT with structured priors. Wang et al. [22] and Poursaeed et al. [23] explore the combination of adversarial training with generative models or structural constraints. Instead of relying solely on pixel-level noise, these methods generate perturbation spectra at multiple semantic levels. This approach has been shown to enhance clean accuracy and OOD performance while maintaining robustness, suggesting that semantically meaningful perturbations are key to bridging the gap between the two fields (see also [22, 23]).

Applying DG and adaptation techniques to improve adversarial robustness. Conversely, a separate research direction interprets the distribution of adversarial examples as a special case of domain shift, subsequently applying Domain Generalization or Domain Adaptation (DA) techniques to mitigate vulnerability.

The core philosophy here is the learning of domain-invariant features. By treating the clean distribution and the adversarial distribution as different domains, these methods

aim to extract features that are insensitive to the domain shift caused by attacks. Awais et al. [24] demonstrate the efficacy of this approach by integrating Unsupervised Domain Adaptation (UDA) modules directly into the network architecture. Their findings suggest that when the architecture and training objectives are properly aligned, it is possible to maintain a reasonable trade-off between robustness and generalization, effectively using adaptation mechanisms to “align” the adversarial samples back to the natural manifold [24].

Hybrid approaches and controlled generative perturbations. Representing the most recent trend, hybrid approaches seek to combine the benefits of domain simulation with the rigor of adversarial training.

These methods move beyond standard pixel-level adversarial noise, which often lacks semantic interpretability, by utilizing generative models to produce controlled distributional variations. [23] report that leveraging generative models for controlled perturbation synthesis allows for the creation of a broad family of distributional shifts. By training on these synthesized “domains,” models demonstrate consistent improvements in both OOD generalization and robustness against previously unseen attack types. This line of work indicates that the integration of generative modeling with robustness training provides a more comprehensive coverage of the potential risk space than either method in isolation [23].

3.4 Research Gap: Attacks as Domains

Synthesis of Existing Work. Despite extensive work on both adversarial training and domain generalization, the two literatures rarely provide a unified and systematic treatment of adversarial attack families as domains. On the one hand, AT research has produced strong defenses such as PGD-AT, TRADES, and MART, which are usually evaluated against a small number of threat models and reported in terms of per-attack robust accuracy [1, 12, 25]. On the other hand, DG research offers principled frameworks—such as IRM, REx, and DomainBed—for reasoning about performance across natural domains, but these frameworks have been instantiated primarily on style, viewpoint, or dataset shifts rather than adversarial perturbations [18, 26, 27]. Bridging efforts like ATDA, robustness-under-shift studies, and REx-based multi-attack training show that adversarial examples can be interpreted as domains and that DG tools can be useful in robustness contexts, but they each cover only part of the “attacks as domains” picture [21, 28, 29]. In particular, prior work generally lacks a unified metricized view of attack families as domains and a staged framework that separates empirical characterization from method design.

Concrete Research Gap. To the best of our knowledge, there is no existing work that (i) systematically quantifies cross-attack transfer through an explicit attacks-as-domains perspective and (ii) integrates this characterization into a two-stage DG-style training framework. Benchmarks such as MultiRobustBench evaluate and rank models under multiple attacks and perturbation strengths, but they focus on aggregate robustness metrics and leaderboards rather than modeling attack families as domains with associated transfer matrices and gaps [30]. Recent work on REx-based out-of-distribution adversarial robustness treats each attack as a domain and demonstrates gains in robustness to unseen attacks, yet it does not center the analysis on cross-attack transfer matrices, nor does it formalize the problem within a DG taxonomy or articulate a clear separation between a measurement stage and a DG-enhanced training stage [29]. Thus, we identify the following concrete gap:

- There is a lack of systematic, metric-driven characterization of how robustness to one attack family transfers to others, especially in terms of transfer matrices and transfer gaps that treat attacks as domains.
- There is no comprehensive framework that first empirically maps cross-attack behavior and then leverages DG-inspired objectives to improve robustness across attack domains under realistic computational constraints.

From Gap to Research Questions and Framework. This gap directly motivates the research questions and framework of our project. Concretely:

- **Stage 1 (completed).** We adopt PGD-based adversarial training on CIFAR-10 with a ResNet-18 baseline and construct a multi-attack evaluation suite across norms (e.g., ℓ_∞ , ℓ_2) and implementations (TorchAttacks, ART, original AutoAttack). Robustness metrics (clean and per-attack accuracy, transfer matrices, transfer gaps) reveal substantial cross-attack gaps.
- **Stage 2 (planned).** We treat each attack family as a domain and explore DG-inspired training objectives (e.g., REx-style risk variance penalties) on top of adversarial training to reduce transfer gaps while keeping robustness competitive on seen attacks.
- **Framework formalization.** Chapter 4 formalizes the attacks-as-domains perspective, states our research questions, and describes the two-stage framework and model variants; evaluation protocols are fixed to keep Stage 1 and Stage 2 comparable. These elements map directly to RQ1 (diagnosis in Stage 1) and RQ2–RQ3 (planned interventions in Stage 2) in Section 4.2.

Chapter 4

Proposed Framework

Scope and Goals

This chapter presents the *conceptual framework* for Stage 2 of the project. All methods discussed here are *planned* and are not claimed to be implemented or empirically validated at this stage. Completed implementations and experimental results are reported exclusively in Chapter 5 (Stage 1).

The goals of this chapter are fourfold:

1. **Formalize the attacks-as-domains viewpoint.** We reinterpret different adversarial attack families as distinct domains, each inducing its own input–label distribution and associated domain-specific risk. This formulation provides a principled language for analyzing cross-attack robustness gaps observed in Stage 1.
2. **State Stage 2 research questions and testable hypotheses.** Building on the empirical findings of Chapter 5, we articulate concrete research questions concerning whether and how domain generalization (DG) principles can reduce cross-attack robustness gaps, improve worst-case robustness, and enhance generalization to unseen attacks.
3. **Describe the planned Stage 2 training workflow at a high level.** We outline, in future tense, the intended two-stage workflow in which DG-inspired objectives are integrated on top of adversarial training. This description focuses on conceptual structure and dataflow rather than algorithmic derivations or implementation details.
4. **Summarize candidate DG-inspired objectives at an overview level.** We review and categorize several DG-motivated training strategies (e.g., multi-attack ERM, REx-style variance penalties, GroupDRO-style worst-case optimization, and mixup-style cross-domain regularization), emphasizing their conceptual role and expected behavior rather than precise formulations.

In addition, this chapter serves two supporting purposes:

- **Define validation scenarios for Stage 2.** We specify a set of planned evaluation scenarios—such as leave-one-attack-out training, held-out AutoAttack evaluation, and cross-norm testing—that operationalize the attacks-as-domains perspective and allow systematic assessment of out-of-domain robustness.
- **Clarify expected trade-offs and limitations.** We discuss anticipated trade-offs between clean accuracy, robustness to seen attacks, robustness to unseen attacks, and computational cost. These expectations are stated explicitly to guide interpretation of future results and to avoid overclaiming improvements in any single dimension.

We intentionally avoid algorithmic derivations, implementation details, and performance claims in this chapter. As a result, Chapter 4 should be read as a *conceptual bridge* between the completed Stage 1 empirical analysis and the prospective Stage 2 methodological development, rather than as a report of new experimental results.

4.1 Attacks as Domains

In Stage-2, we treat each adversarial attack family—defined by its family, norm, budget, and optimization variant—as a distinct *domain*. Under this view, an attack procedure induces a domain-specific distribution over perturbed inputs, and robustness is discussed as a per-domain performance quantity.

Formally, let $\mathcal{E} = \{1, \dots, E\}$ index the selected attack domains and let $\mathcal{A}_e$ denote the attack procedure for domain e . For a classifier f_θ and loss ℓ , we define the domain-induced adversarial risk

$$R_e(\theta) = \mathbb{E}_{(x,y) \sim P_{XY}} \left[\ell(f_\theta(\mathcal{A}_e(x, y, f_\theta)), y) \right].$$

In Stage 2, $R_e(\theta)$ will be approximated by the average loss on domain-partitioned mini-batches, where each partition is attacked using its corresponding $\mathcal{A}_e$. This definition provides a common measurement unit to express both objectives and evaluation metrics in later chapters.

We distinguish *source* attack domains (used during training) from *target / held-out* domains (reserved for evaluation). The number and diversity of source domains will be treated as a practical design choice; Stage 2 will document compute–coverage trade-offs explicitly in the evaluation plan.

4.2 Research Questions and Hypotheses

To systematically address the limitations of current adversarial training paradigms identified in Stage 1, this study establishes a structured investigation framework. The logic of our inquiry proceeds in three sequential phases: *diagnosis*, *intervention*, and

evaluation. First, we must rigorously quantify the extent to which current single-attack defenses fail to generalize (Diagnosis). Second, we must determine if re-framing the problem through the lens of Domain Generalization provides a statistically superior defense against unseen attacks compared to strong baselines (Intervention). Finally, we must assess whether the computational and performance costs of this intervention are viable for practical deployment (Evaluation).

Accordingly, this research is guided by the following three questions.

RQ1 (Quantifying the Transfer Gap – Stage 1 Completed). To what extent does adversarial robustness learned from a single attack domain fail to transfer to heterogeneous unseen attack domains?

RQ2 (Efficacy of DG-Integrated Training – Stage 2 Planned). Does integrating domain-generalization objectives into adversarial training yield superior worst-case robustness on held-out attacks compared to multi-attack empirical risk minimization?

RQ3 (Practical Trade-offs – Stage 2 Planned). What are the empirical trade-offs between clean accuracy, worst-case robustness, and training computational costs when adopting domain-generalization strategies?

Drawing from the *Attacks-as-Domains* formulation as we defined earlier, we derive the following testable hypotheses to guide the Stage-2 experiments:

- **H1 (Stage 1 expectation).** Models trained via standard PGD on a single norm will exhibit significant performance degradation when evaluated against attacks with different optimization objectives (such as Carlini-Wagner) or different norms, even when perturbation budgets are perceptually comparable. This confirms that without domain-aware intervention, robustness remains brittle and domain-specific.
- **H2 (Stage 2 expectation).** Multi-attack ERM (training on several attacks without domain-aware regularization) will reduce cross-attack transfer gaps relative to single-attack training, but may retain a weak domain that limits worst-case robustness.
- **H3 (Stage 2 expectation).** Domain-aware objectives that explicitly control per-domain risk dispersion or emphasize weak domains (e.g., variance penalties or worst-domain weighting) will improve worst-domain and leave-one-attack-out robustness for reasonable hyperparameter settings, with a modest reduction in clean accuracy.
- **H4 (Stage 2 expectation).** Alignment-oriented mechanisms (e.g., cross-domain Mixup or output-consistency penalties) will reduce boundary discrepancies between domains and thereby reduce transfer gaps without prohibitive additional training cost.

4.3 Framework Overview

This section describes what the proposed framework is and how it is structured. We focus on the overall structure rather than technical details. Stage 2 is planned only and has not been implemented within the scope of this report. Chapter 4 proposes a research framework that connects our Stage 1 empirical findings Stage 1 is completed and provides baseline results and a reliable evaluation procedure. Stage 2 is proposed and is treated as a design blueprint for future work. Accordingly, this section summarizes the framework at a high level and does not describe training steps, loss functions, or specific DG algorithms.

4.3.1 Two-phase structure.

The framework is organized into a completed *foundation phase* (Stage 1) and a planned *extension phase* (Stage 2).

Foundation phase (Stage 1, completed). The foundation phase includes components that are already completed in Stage 1. It is not only a benchmark baseline, but also the infrastructure needed to measure cross-attack robustness gaps in a consistent way. The foundation consists of:

- **Baseline adversarial training.** A standard PGD-based adversarial training pipeline provides the main Stage 1 baseline and a reference point for quantifying robustness changes under different evaluation attacks.
- **Unified attack harness.** A single evaluation harness generates adversarial examples from multiple attack families under a shared interface, enabling cross-attack evaluations with consistent settings and controlled budgets.
- **Logging and reporting pipeline.** A reproducible workflow for experiment tracking and reporting (configuration control, checkpointing, and standardized evaluation outputs).

The main outcome of this phase is a *measurement-ready* setup: it allows us to evaluate the same model under different attacks and quantify cross-attack robustness gaps without changing the evaluation tooling.

Extension phase (Stage 2, planned). Building on the Stage 1 foundation, the planned extension introduces DG as a mechanism to reduce cross-attack robustness gaps. Under the attacks-as-domains viewpoint, each adversarial setting is treated as a separate **domain**. Stage 2 is intended to build around this idea:

- **Domain-aware adversarial training.** Training will be extended to consider multiple attack domains as *source domains*. The goal is to improve robustness not only on attacks seen during training, but also on attacks outside the training set

(treated as held-out evaluation domains). Candidate DG strategies and integration options are outlined conceptually in Section 4.4.

- **Domain-based evaluation.** Evaluation will be organized around domain splits such as seen versus held-out attacks to directly target the Stage 1 issue where robustness can be strong on the training attack but weaker on other attacks. The validation scenarios are specified in Section 4.6.

4.3.2 Pipeline overview.

The framework uses two strictly separated pipelines: (i) a **training pipeline** that produces a trained checkpoint, and (ii) an **evaluation pipeline** that produces robustness metrics. Stage 1 and Stage 2 share the same evaluation pipeline so that differences across stages (if observed later) can be attributed to training design rather than evaluation changes.

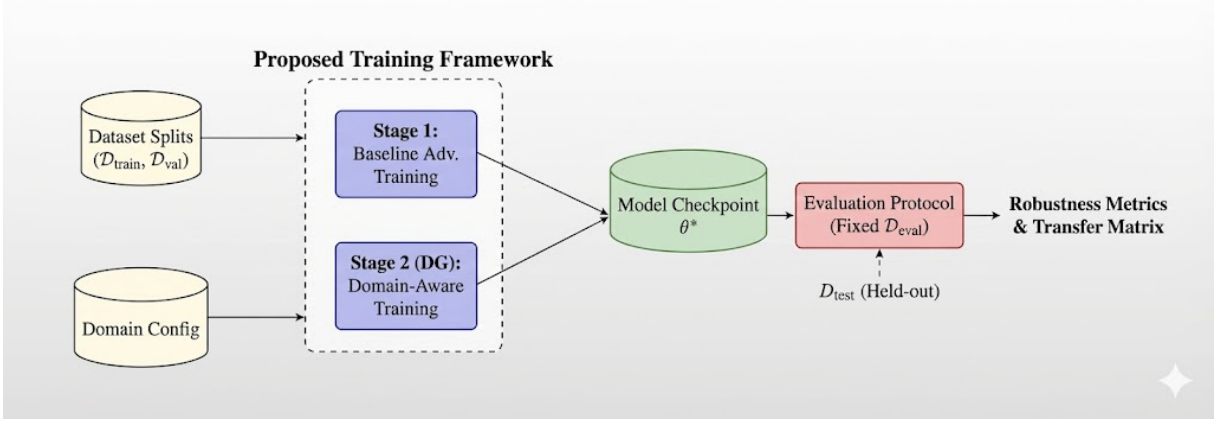


Figure 4.1: High-level training and evaluation pipeline. Stage 1 (completed) provides the foundation; Stage 2 (planned) reuses the same evaluation while extending training with domain-aware objectives.

Key components and deliverables. We define the main inputs and outputs used throughout the framework:

- **Dataset splits.** Starting from a dataset D , we follow a fixed split policy. When a validation split is used, we create $(D_{\text{train}}, D_{\text{val}}, D_{\text{test}})$ with a fixed seed and a fixed preprocessing/normalization setup. These splits are reused across all model variants.
- **Attack-domain specification.** We define a finite set of attack domains $\mathcal{D} = \{d_1, \dots, d_M\}$, where each domain is a pair of an attack operator and its configuration:

$$d \mapsto (\mathcal{A}_d, \delta_d), \quad \delta_d = (\text{norm}, \epsilon, \text{steps}, \text{restarts}, \text{targeting}).$$

The full domain set $\mathcal{D}$ (including evaluation budgets) is stored in a single versioned

config file and reused unchanged across stages.

- **Checkpoint and logs.** The output of training is a checkpoint θ^* , together with logs that record run settings (seeds, split policy/seed, domain configs, training schedule, runtime) to support reproducibility.

Training pipeline. The training pipeline takes D_{train} and produces a checkpoint θ^* .

- **Stage 1 (baseline adversarial training).** For each minibatch $(x, y) \sim D_{\text{train}}$, we generate adversarial examples on-the-fly using a fixed training domain $d_{\text{train}} = (\mathcal{A}_{\text{train}}, \delta_{\text{train}})$ (e.g., PGD under a chosen norm and budget), and update the model using the adversarially perturbed inputs. The output is a trained checkpoint θ^* and training logs.
- **Stage 2 (planned DG-integrated training).** Stage 2 will keep the same training-loop structure but will extend it to consider multiple training domains (e.g., a set of $\{d_k\}_{k=1}^K$). The specific DG objectives and integration options are treated as design choices and are outlined conceptually in Section 4.4. Importantly, Stage 2 remains planned in this report, and this section only specifies the dataflow and the definition of training inputs and outputs.

Evaluation pipeline. The evaluation pipeline takes a trained checkpoint θ^* and computes robustness metrics on D_{test} using a *fixed* evaluation domain set $\mathcal{D}_{\text{eval}}$.

- **Inputs:** checkpoint θ^* , D_{test} , and the frozen domain config $\mathcal{D}_{\text{eval}}$.
- **Per-domain evaluation:** for each domain $d = (\mathcal{A}_d, \delta_d)$, generate adversarial examples $x_i^{(d)} = \mathcal{A}_d(x_i, y_i, f_\theta; \delta_d)$ (with consistent input constraints such as clamping to valid range) and compute the corresponding robust accuracy.
- **Outputs:** a per-domain results table, transfer summaries, and aggregate robustness metrics (defined in Section 4.5).

When applicable, AutoAttack is included as a standardized evaluation suite with fixed parameters and no manual tuning. All evaluation settings are kept consistent across model families.

Pipeline requirements. To keep training and evaluation reproducible and comparable across stages, the pipelines adhere to:

- **Versioned configs:** attack budgets (norm, ε , steps, restarts, loss), seeds, and data splits are stored in a single config file reused across runs.
- **Logging discipline:** every run logs start/end timestamps, GPU model/count, learning-rate schedule, and the exact attack domains used for inner/outer loops.
- **Frozen evaluation:** evaluation code and attack hyperparameters are immutable between Stage 1 and planned Stage 2 to avoid selection bias.

- **Reproducible data handling:** fixed preprocessing (normalization, augmentation) and deterministic split seeds for $(D_{\text{train}}, D_{\text{val}}, D_{\text{test}})$.

No-leakage and consistency rules. To ensure a fair comparison and avoid evaluation leakage:

- **Frozen evaluation config.** The evaluation domain set $\mathcal{D}_{\text{eval}}$ and its budgets are fixed and reused unchanged across Stage 1 and planned Stage 2.
- **Held-out domains are not used for selection.** Domains reserved for held-out evaluation are not used for model selection, hyperparameter tuning, or early stopping in the planned Stage 2 study.
- **Reproducibility.** We log split seeds/policy, domain configs, and evaluation settings so the same evaluation can be rerun exactly on any checkpoint.

This specification makes the stage-to-stage comparison well-defined: the training pipeline may change in Stage 2, but the evaluation pipeline and its domain definitions remain fixed (Section 4.5).

4.4 Design Space and Prioritization of DG Objectives

To systematically bridge the gap between adversarial robustness and domain generalization, we adopt the taxonomy proposed by Wang et al. [15], which categorizes approaches into *Learning Strategy*, *Data Manipulation*, and *Representation Learning*. Within this framework, we identify four representative candidates: Risk Extrapolation (REx) [26], Group Distributionally Robust Optimization (GroupDRO) [31], Cross-Domain Mixup [32], and Invariant Risk Minimization (IRM) [18].

However, adversarial training is computationally intensive, requiring iterative gradient steps for every batch. Therefore, we do not treat these candidates equally. We prioritize their implementation based on three practical criteria: **computational overhead** (avoiding expensive auxiliary networks), **optimization stability** (compatibility with min-max dynamics), and **adversarial relevance** (direct addressing of attack shifts). The following subsections detail our instantiation of these methods, ordered by their strategic priority.

The Baseline: Multi-attack Empirical Risk Minimization. Before introducing specialized generalization objectives, we establish a strong control baseline. We adopt Multi-attack Empirical Risk Minimization (ERM), which trains the model on the combined set of all source attack domains by minimizing the average risk. Formally, for

a set of attack domains $\mathcal{E}$, the objective is given by

$$\min_{\theta} \frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} R_e(\theta)$$

where $R_e(\theta)$ represents the risk associated with attack domain e . This baseline assumes that simply exposing the model to diverse attacks is sufficient for generalization. It serves as the primary benchmark to verify whether domain-aware constraints provide statistically significant benefits beyond mere data augmentation.

Pillar 1: Learning Strategy (Risk Profile Optimization). Standard ERM often fails in adversarial settings because attack domains are highly heterogeneous. Optimizing the average allows the model to overfit to easy attacks while neglecting harder ones. We propose two mechanisms to reshape this risk profile, prioritizing methods with low computational overhead.

- **Variance Reduction via Risk Extrapolation (REx).** Adversarial training often exhibits "robustness oscillation," where improving resistance to one attack causes a sudden drop in another. REx addresses this instability by penalizing the variance of risks across domains [26]. We instantiate *V*-REx by computing the variance of losses across the attack dimension within each mini-batch. Given that this operation incurs negligible computational cost ($O(1)$) relative to the backward pass and forces the optimizer to find a "flat" minimum consistent across attacks, we consider REx a **high-priority candidate** for immediate integration.
- **Worst-Case Optimization via GroupDRO.** From a security perspective, a system is only as strong as its weakest link. A model with high average robustness but near-zero resistance to a specific attack family is effectively broken. GroupDRO aligns directly with our worst-case robustness metric by dynamically up-weighting the domain with the highest loss [31]. We treat each attack family as a distinct group. Although GroupDRO can exhibit instability with small batch sizes, its ability to theoretically guarantee a lower bound on performance makes it a **critical priority** for ensuring comprehensive defense.

Pillar 2: Data Manipulation (Inter-Domain Interpolation). Adversarial examples typically reside on the boundary of defined norm balls, leaving the space between distinct geometries (e.g., l_{∞} versus l_2) unexplored during standard training.

- **Manifold Smoothing via Cross-Domain Mixup.** The shift between different perturbation norms represents a geometric discontinuity. Standard training leaves the decision boundary undefined in the transition region. To address

this, we propose *Adversarial Interpolation* using Mixup [32]. Unlike standard applications mixing clean and perturbed images, we generate training samples by linearly interpolating between adversarial examples from different domains (e.g., combining a PGD perturbation with a DeepFool perturbation). This forces the model to maintain linear behavior in the latent space connecting distinct threat models. Since this relies on linear combinations of already-generated attacks, it adds minimal overhead and serves as a **complementary high-priority strategy** to the loss-based methods above.

Pillar 3: Representation Learning (Deep Invariance). This pillar focuses on the internal feature space, aiming to map input images to consistent feature clusters regardless of the applied perturbation.

- **Deep Feature Invariance via IRM (Exploratory Candidate).** Invariant Risk Minimization (IRM) theoretically ensures that the learned predictor is optimal across all environments, offering a stronger condition than simple output consistency [18]. However, IRM requires computing gradients with respect to classifiers for each environment, which significantly increases the training cost per iteration. Consequently, we classify IRM as a **secondary, exploratory candidate**. We will investigate this method primarily to establish a theoretical upper bound for feature invariance, provided that the computational budget permits after the evaluation of the higher-priority methods.

Table 4.1: Qualitative evaluation of candidate DG objectives based on three selection criteria: **Comp.**, **Cost**, **Stability**, and **Relevance**. Priorities are assigned based on the trade-off between feasibility and theoretical value.

Method	Comp. Cost	Stability	Adv. Relevance	Priority
Multi-attack ERM	Low (Baseline)	High	Baseline Coverage	<i>Control</i>
REx (Variance)	Negligible ($O(1)$)	High	Anti-Oscillation	High
GroupDRO	Low (Re-weight)	Moderate	Worst-Case Opt.	High
Cross-Domain Mixup	Low (Linear)	High	Geometric Bridge	High
IRM (Invariant)	High (Grad-norm)	Low	Feature Invariance	Secondary

4.5 Evaluation Protocol

This section defines the unified evaluation protocol used to report completed Stage 1 results and to support the planned Stage 2 study. The protocol specifies *what* is measured, *how* metrics are computed and reported, and *how* comparisons are kept fair across training designs. No Stage 2 result or performance claim is made in this section.

Role of the evaluation protocol. The evaluation protocol serves two purposes. First, it provides a consistent and reproducible mechanism for reporting Stage 1 baselines. Second, it defines a fixed measurement plan for Stage 2 so that future experiments can be compared against Stage 1 under identical evaluation conditions.

This section specifies the evaluation *mechanism* only. Concrete validation scenarios (e.g., in-distribution evaluation, held-out attack families, LOAO, AutoAttack stress test) are defined separately in Section 4.6.

Evaluation objectives. The protocol is aligned with the goals of this project and targets three measurement objectives:

1. **Cross-attack robustness gaps:** quantify how robust accuracy varies when evaluation attacks differ from the attacks used during training.
2. **Aggregate robustness across domains:** summarize robustness over a fixed set of evaluation domains, with emphasis on worst-domain behavior.
3. **Trade-offs and cost:** characterize clean–robust trade-offs and report basic compute indicators under a realistic evaluation budget.

Models under comparison. To reduce evaluation bias, all compared models share the same architecture, dataset splits, and evaluation pipeline. We consider the following model families:

- **Stage 1 baselines (completed).** Single-domain adversarial training under a standard PGD-based threat model. These models serve as fixed reference points for robustness gaps and compute trade-offs.
- **Multi-domain ERM baselines (planned control).** Training on multiple attack domains using a standard average-risk objective. These baselines isolate the benefit of training-domain diversity from the effect of DG-specific structure.
- **DG-integrated adversarial training (planned).** Training under the attacks-as-domains formulation using DG-inspired objectives. Candidate strategies are discussed in Section 4.4.

Across all model families, domain definitions and the evaluation pipeline remain unchanged.

Notation. Let $D_{\text{test}} = \{(x_i, y_i)\}_{i=1}^{n_{\text{test}}}$ denote the test dataset and $f_{\theta}(\cdot)$ a trained model with parameters θ . An evaluation domain is defined as $d = (\mathcal{A}_d, \delta_d)$, where $\mathcal{A}_d$ is an attack operator and δ_d specifies its configuration (norm, budget, and other hyperparameters).

For a given evaluation domain $d \in \mathcal{D}_{\text{eval}}$, we denote the adversarial example

generated from (x_i, y_i) as

$$x_i^{(d)} = \mathcal{A}_d(x_i, y_i, f_\theta; \delta_d).$$

Metric definitions. We define the following metrics to capture the evaluation objectives.

Clean accuracy. Clean accuracy measures performance on unperturbed inputs:

$$\text{Acc}_{\text{clean}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbb{1}[f_\theta(x_i) = y_i]. \quad (4.1)$$

This metric provides context for robustness trade-offs.

Per-domain robust accuracy. For a given evaluation domain $d \in \mathcal{D}_{\text{eval}}$, robust accuracy is defined as:

$$\text{Acc}_{\text{robust}}^{(d)} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbb{1}[f_\theta(x_i^{(d)}) = y_i]. \quad (4.2)$$

Reporting per-domain robustness exposes cross-attack asymmetries.

AutoAttack robustness (primary/hard metric). AutoAttack robustness is defined as the robust accuracy evaluated under the AutoAttack suite in its standard, parameter-free configuration. Let $d_{\text{AA}} = (\mathcal{A}_{\text{AA}}, \delta_{\text{AA}})$ denote the AutoAttack domain. We define

$$\text{Acc}_{\text{AA}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbb{1}[f_\theta(x_i^{(d_{\text{AA}})}) = y_i]. \quad (4.3)$$

AutoAttack is treated as the primary (hard) robustness metric and, under the attacks-as-domains viewpoint, is interpreted as a target-domain evaluation. It is used strictly for evaluation and is never employed for model selection, early stopping, or hyperparameter tuning.

Clean-robustness gap. For each evaluation domain d , we quantify the drop from clean to adversarial performance:

$$\Delta_{\text{clean-robust}}^{(d)} = \text{Acc}_{\text{clean}} - \text{Acc}_{\text{robust}}^{(d)}. \quad (4.4)$$

Cross-attack transfer gap (scenario-dependent). When a training-domain reference is well-defined by the scenario (e.g., single-domain training or a designated source set),

we define the transfer gap from a reference domain d_{ref} to an evaluation domain d as:

$$\Delta_{\text{transfer}}^{(d_{\text{ref}} \rightarrow d)} = \text{Acc}_{\text{robust}}^{(d_{\text{ref}})} - \text{Acc}_{\text{robust}}^{(d)}. \quad (4.5)$$

This quantity is used to summarize cross-attack generalization only in scenarios where d_{ref} is unambiguous (see Section 4.6).

Aggregate robustness across domains. For a fixed evaluation domain set $\mathcal{D}_{\text{eval}}$, we summarize robustness using:

$$\text{Acc}_{\text{robust}}^{\text{worst}} = \min_{d \in \mathcal{D}_{\text{eval}}} \text{Acc}_{\text{robust}}^{(d)}, \quad (4.6)$$

$$\text{Acc}_{\text{robust}}^{\text{avg}} = \frac{1}{|\mathcal{D}_{\text{eval}}|} \sum_{d \in \mathcal{D}_{\text{eval}}} \text{Acc}_{\text{robust}}^{(d)}. \quad (4.7)$$

Reporting rules and fairness constraints. To ensure fair and reproducible comparisons, we apply the following rules:

- **Frozen evaluation pipeline.** The evaluation code, attack implementations, and evaluation domain set $\mathcal{D}_{\text{eval}}$ are fixed across models.
- **Fixed attack budgets.** Each evaluation domain uses a fixed budget (e.g., iteration count and restarts), applied consistently across models.
- **No tuning on held-out domains.** Domains reserved for held-out evaluation are not used for model selection, hyperparameter tuning, or early stopping.

Compute and statistical reporting. We report basic evaluation cost indicators (e.g., runtime) alongside accuracy metrics. When feasible, results are averaged over multiple random seeds and dispersion (e.g., mean and standard deviation) is reported. We avoid strong conclusions when results are unstable across seeds.

4.6 Validation Scenarios

To distinguish genuine cross-attack generalization from mere overfitting to the training attacks, a single evaluation setting is insufficient. We therefore design a structured validation protocol composed of four complementary scenarios, each targeting a distinct aspect of robustness under the *attacks-as-domains* perspective. All scenarios are applied using identical model architectures, training budgets, and evaluation pipelines for both DG-inspired methods and the corresponding multi-attack ERM baselines, ensuring fair and controlled comparison.

Across all scenarios, we report per-attack robust accuracy together with aggregate summary metrics computed over the evaluated attack set, focusing on worst-case and

average robustness. Attacks designated as held-out targets are used strictly for evaluation and are never employed for model selection, early stopping, or hyperparameter tuning.

Scenario 1: Seen-Domain Evaluation (In-Distribution Control). This scenario evaluates models on the same attack families used during training, i.e., the source domains.

- **Objective.** The purpose of this scenario is to assess in-domain optimization stability and quantify the cost of generalization. DG-inspired objectives often introduce additional constraints that may reduce performance on known domains. By comparing against the multi-attack ERM baseline, this scenario determines whether DG methods preserve competitive robustness on seen attacks or incur excessive in-distribution degradation (i.e., catastrophic forgetting).

Scenario 2: Held-Out Attack Evaluation (Illustrative Zero-Shot Transfer). In this scenario, models are evaluated on one or more attack families that are explicitly excluded from the training set. For example, a model trained on gradient-based ℓ_∞ attacks may be evaluated on optimization-based ℓ_2 attacks such as Carlini–Wagner or DeepFool.

- **Objective.** This scenario provides a clear and interpretable illustration of zero-shot transfer to unseen attack domains. Rather than aiming for exhaustive coverage, it serves as a focused case study that highlights large cross-family robustness gaps that are invisible under seen-domain evaluation alone. Performance in this scenario reflects the ability of a model to generalize across qualitatively different attack mechanisms.

Scenario 3: Leave-One-Attack-Out (LOAO) Protocol. To eliminate selection bias in the choice of source and target domains, we adopt a systematic Leave-One-Attack-Out (LOAO) protocol. In each fold, one attack family is designated as the held-out target domain, while all remaining families constitute the source domains used for training.

- **Objective.** The LOAO protocol assesses whether observed robustness gains are consistent properties of the training objective rather than artifacts of a particular target-domain choice. By rotating the held-out attack, this scenario yields a distribution of transfer performance and robustness gaps, enabling analysis of both mean behavior and variability across attack domains. This provides a principled measure of systematic cross-attack generalization.

Scenario 4: Standardized Stress Test (AutoAttack). Finally, all models are evaluated under AutoAttack, an ensemble of parameter-free attacks widely regarded as a standardized benchmark for adversarial robustness.

- **Objective.** AutoAttack serves as the primary robustness metric and is interpreted as a target-domain evaluation under the attacks-as-domains framing. Its role is to verify end-to-end robustness, rule out gradient masking through its gradient-free components, and provide a single, comparable reference metric for validating whether DG-inspired objectives improve robustness beyond Stage 1 baselines. AutoAttack is used strictly for evaluation and is never employed during training or model selection.

4.7 Expected Trade-offs

Stage 2 (planned) explores DG-inspired objectives within adversarial training under the *attacks-as-domains* viewpoint. While these objectives are intended to promote more transferable robustness across attack domains, they may introduce systematic trade-offs in clean accuracy, in-domain robustness, training stability, and computational cost. This section summarizes the main trade-offs that the Stage 2 validation scenarios (Section 4.6) are designed to probe.

Trade-off 1: Clean accuracy versus adversarial robustness. A well-established tension in adversarial training is that improving robustness under a given threat model may reduce clean accuracy on unperturbed inputs. In Stage 2, this trade-off may become more pronounced because training may incorporate multiple attack domains and additional DG-inspired regularization terms. For example, objectives that emphasize worst-case or balanced performance across domains (e.g., GroupDRO-style reweighting or REx-style risk dispersion control) may reduce the model’s capacity to optimize purely for clean accuracy.

- **How it will be checked.** This trade-off is primarily assessed via Scenario 1 (seen-domain control) by tracking clean accuracy alongside per-attack robust accuracy on the training domains, and via Scenario 4 (AutoAttack) to ensure that any robustness gains under the hard metric are not accompanied by disproportionate clean-accuracy degradation.

Trade-off 2: In-domain robustness versus cross-domain robustness. Standard single-attack adversarial training can yield strong robustness on the training attack but substantially weaker robustness on mismatched attacks (different norms, losses, or optimization dynamics), i.e., large cross-attack robustness gaps. DG-inspired training may reduce this specialization by explicitly discouraging high variance in domain-specific risk. However, improving robustness on weaker or unseen domains may come at the cost of slightly reduced robustness on some seen domains, reflecting a shift from maximizing a single-domain score to improving robustness and transferability.

- **How it will be checked.** Scenario 1 measures robustness on training (source) domains, while Scenario 2 and Scenario 3 directly probe cross-domain transfer to held-out domains. We will quantify this trade-off using per-attack robust accuracy and robustness-gap summaries (e.g., worst-case robustness over evaluated domains and transfer-gap statistics in LOAO folds).

Trade-off 3: Attack-domain coverage versus computational cost. Increasing the diversity of attack domains used during training can broaden exposure to different adversarial mechanisms, but it also increases compute overhead because each domain typically requires generating adversarial examples (often iteratively). Therefore, there is a practical trade-off between (i) the number of training domains included (attack families and norms), (ii) the strength of the inner maximization budget (e.g., PGD steps), and (iii) feasible training time under a fixed compute budget.

- **How it will be checked.** All scenarios report computational cost (e.g., wall-clock time) alongside robustness metrics. Comparisons against multi-attack ERM baselines are performed under matched training budgets and identical evaluation pipelines to ensure that robustness differences are not simply explained by increased compute.

Trade-off 4: Strength of DG constraints versus training stability. Some DG objectives that enforce invariances or reduce domain-to-domain risk dispersion can be sensitive to hyperparameters and may be challenging to optimize when combined with adversarial training, which is already expensive and nontrivial to stabilize. Overly strong penalties may lead to unstable training dynamics, degraded convergence, or inconsistent results across random seeds.

- **How it will be checked.** Scenario 1 acts as an optimization sanity check by monitoring training curves, validation robustness on seen domains, and the consistency of outcomes under controlled settings. In addition, Stage 2 proceeds in a modular manner: multi-attack ERM serves as the baseline, and DG constraints are introduced incrementally so that their effects on robustness, stability, and compute can be isolated.

Role of AutoAttack in trade-off interpretation. Across scenarios, AutoAttack robust accuracy is treated as the primary (hard) robustness metric and interpreted as a target-domain evaluation under the attacks-as-domains framing. As a result, improvements on weaker or seen-domain metrics are considered meaningful only if they do not collapse under AutoAttack, which also serves as a safeguard against spurious robustness and gradient masking effects.

Chapter 5

Initial Implementation

This chapter describes the Stage-1 implementation on CIFAR-10 and presents empirical results on adversarial robustness under multiple attacks. We first describe the dataset, model architecture, training configurations, and attack harness. We then report clean and robust accuracies for several models, analyze cross-attack robustness gaps, and discuss the implications for Stage 2.

5.1 Baseline Setup

5.1.1 Dataset

All experiments in Stage 1 are conducted on the CIFAR-10 dataset [33], which contains 60,000 32×32 color images from 10 classes, with 50,000 training and 10,000 test images.

Why only CIFAR-10?

- We restrict Stage 1 to CIFAR-10 for three reasons: (i) it is the standard benchmark for PGD-based adversarial training, enabling direct comparison with prior work; (ii) the 32×32 resolution keeps per-epoch adversarial training time manageable under our compute budget; and (iii) the chosen ResNet-18 backbone and attack harness (TorchAttacks/ART/AutoAttack) are well tested on CIFAR-10, minimizing engineering risk. Larger datasets (CIFAR-100, ImageNet) are deferred to future work due to compute and time constraints.
- We do not include MNIST in this study. Although MNIST is historically important, it is no longer considered a challenging benchmark for modern adversarial robustness research. Models typically achieve near-saturated performance on MNIST, and robustness trends observed on MNIST often fail to transfer to more complex natural-image datasets. Since the goal of Stage 1 is to study cross-attack robustness gaps under realistic image statistics, we restrict our experiments to CIFAR-10, which provides a more meaningful and widely accepted testbed.

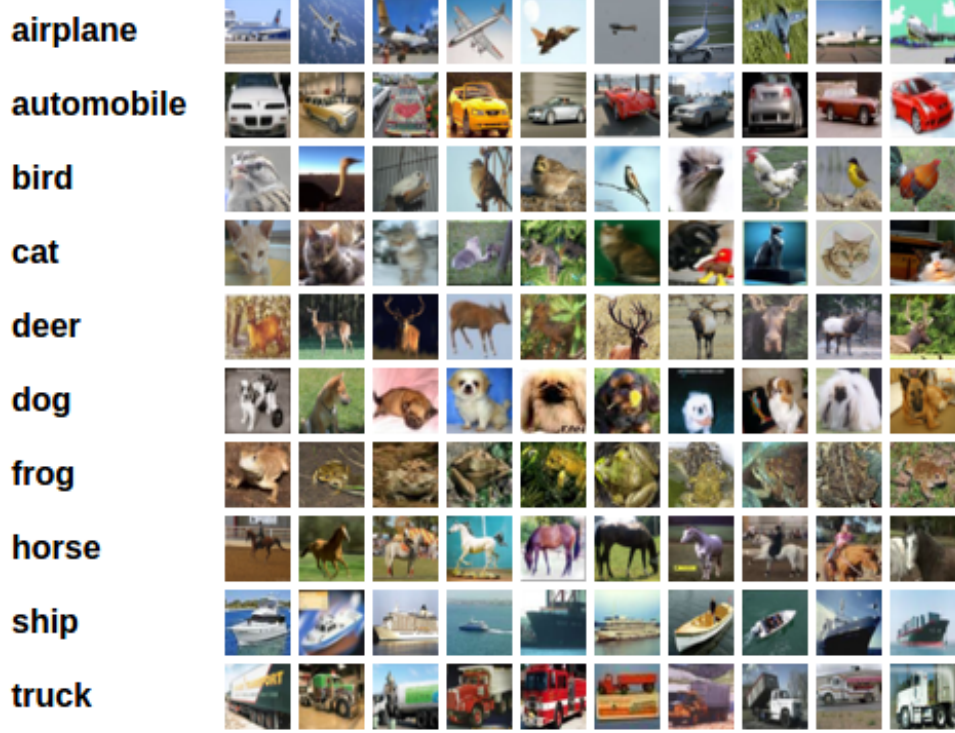


Figure 5.1: CIFAR-10 sample images.

5.1.2 Model Architecture

As the Stage 1 backbone we adopt a CIFAR-style ResNet-18. The network follows the standard 18-layer residual architecture with BasicBlock units, but with a modified “CIFAR stem”: the first convolution is a 3×3 layer with stride 1 and no initial max-pooling, which is more appropriate for 32×32 inputs. Batch normalization is used after each convolution, and we initialize the second batch-normalization layer in each residual block with zero weight to stabilize training and encourage identity mappings at the start of optimization. Compute environment details are provided in Appendix A.7.

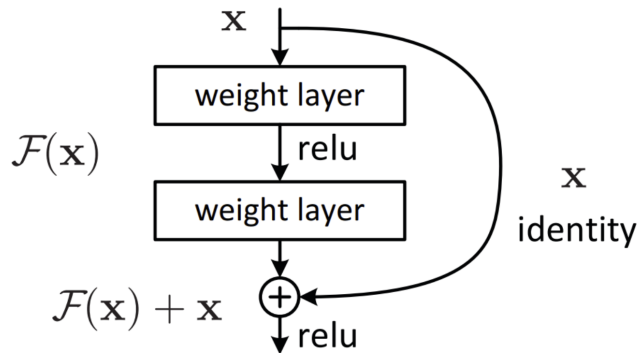


Figure 5.2: BasicBlock of residual learning used in ResNet-18. Downsampling blocks use stride $s = 2$ in the first 3×3 convolution and a 1×1 projection on the skip path [34].

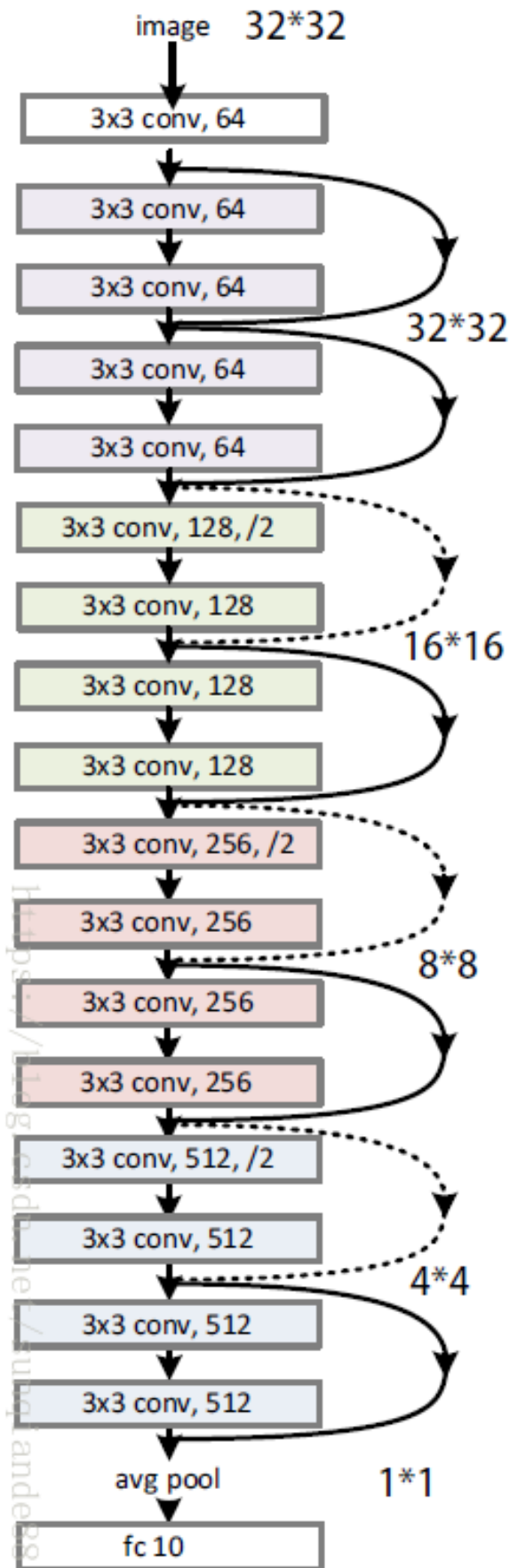


Figure 5.3: ResNet-18 with CIFAR stem (no maxpool), two BasicBlocks per stage (rotated for vertical layout). Source: adapted from https://blog.csdn.net/lyq_12/article/details/89483601.

5.1.3 Attack Suite

A central deliverable of Stage 1 is a unified adversarial evaluation harness that applies a diverse set of white-box attacks to any PyTorch classifier through a consistent interface. We treat each attack as inducing a distinct “adversarial domain”—i.e., a characteristic distribution of adversarial examples for the same underlying data distribution.

Evaluation scope and assumptions. Unless otherwise stated, all attacks are white-box and untargeted, and operate under either an ℓ_∞ or ℓ_2 constraint. Detailed threat-model parameters (e.g., ε , step sizes, and iteration budgets) are fixed a priori and specified in Section 5.1.4.

Normalized vs. pixel-space inputs. Models are evaluated on normalized inputs, while some libraries (e.g., ART) operate natively in pixel space. To ensure consistency across implementations, the harness includes a lightweight bridge that (i) de-normalizes inputs to $[0, 1]$, (ii) runs the attack in pixel space, and (iii) re-normalizes the resulting adversarial examples before forwarding them through the model. This design avoids silent mismatches in threat models due to differing input conventions.

Attack families. The suite covers representative attacks from the major adversarial families:

- **Gradient-based attacks.**
 - *FGSM* (ℓ_∞) – a single-step attack using the sign of the loss gradient.
 - *PGD* (ℓ_∞ and ℓ_2) – multi-step projected gradient descent with random initialization inside the threat set. We implement PGD in both TorchAttacks and ART to cross-check consistency under matched hyperparameters.
- **Optimization-based attacks.**
 - *Carlini–Wagner* (*CW*, ℓ_2) – a margin-based attack formulated as an unconstrained optimization problem with a distortion penalty, solved by iterative optimization.
- **Decision-boundary attacks.**
 - *DeepFool* (ℓ_2) – iteratively linearizes the classifier and updates toward the nearest decision boundary.
- **Ensemble evaluation: AutoAttack.** We integrate both (i) the original AutoAttack implementation, consisting of APGD-CE, APGD-DLR, FAB, and Square, and (ii) the TorchAttacks wrapper, using the standard configuration. AutoAttack is treated as a strong, parameter-light robustness benchmark.

Implementation interface. The harness is centered around the function `make_attack_suite_for_model`, which takes a model, loss function, and a set of requested backends (TorchAttacks / ART / original AutoAttack), and returns a dictionary mapping descriptive names (e.g., ```Torch PGD (ℓ_∞) ''`) to instantiated attack objects. This abstraction allows the same evaluation code to be reused across models and attack subsets, and it later serves as the basis for defining per-attack “domains” in Stage 2.

5.1.4 Threat Model and Attack Configuration

Unless stated otherwise, all attacks are white-box and untargeted, and all intermediate adversarial iterates are projected back to the valid pixel range after each step.

- **Dataset and splits.** CIFAR-10. We hold out 2% of the training set for validation and use the official test set only for final evaluation. All splits and deterministic sampling procedures use `seed = 42`.
- **Training-time inner maximization (PGD- ℓ_∞).** $\varepsilon = 8/255$, $K = 10$ steps, step size $\alpha = 2/255$, random start within the ℓ_∞ ball, no restarts.
- **Validation attack.** PGD- ℓ_∞ with $\varepsilon = 8/255$, $K = 20$ steps, $\alpha = 2/255$, one random start (restarts = 1).
- **Evaluation attacks (test set).**
 - **FGSM- ℓ_∞ :** $\varepsilon = 8/255$, $K = 1$ step.
 - **PGD- ℓ_∞ (TorchAttacks/ART):** $\varepsilon = 8/255$, $K = 20$ steps, $\alpha = 2/255$, restarts = 5, random start.
 - **PGD- ℓ_2 (TorchAttacks/ART):** $\varepsilon = 0.5$, $K = 20$ steps, $\alpha = 0.1$, restarts = 5, random start.
 - **CW- ℓ_2 :** learning rate 0.01, up to 200 iterations, untargeted.
 - **DeepFool- ℓ_2 :** up to 50 iterations, untargeted.
 - **AutoAttack (original suite):** APGD-CE, APGD-DLR, FAB, Square; default (parameter-free) settings; one run per model.
- **Diagnostics.** We report (i) a PGD step-sweep with $K \in \{1, 2, 5, 10, 20, 50, 100\}$ under the same ε and step sizes as above to validate attack strength, and (ii) a PGD- ℓ_∞ ε -sweep with $\varepsilon \in \{0, 5/255, \dots, 30/255\}$ using $K = 20$ steps and 5 restarts.

5.2 Adversarial Training and Evaluation Workflow

5.2.1 PGD-AT Training Pipeline

Stage 1 uses standard adversarial training with projected gradient descent (PGD-AT) as the inner maximization. We instantiate the generic min–max formulation

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\max_{\|\delta\|_p \leq \varepsilon} \ell(f_{\theta}(x + \delta), y) \right]$$

with an ℓ_∞ PGD attacker of radius $\varepsilon = 8/255$.

For training we apply standard data augmentation: random horizontal flips and random crops with 4-pixel padding. All images are then normalized using the standard CIFAR-10 statistics $\mu = (0.4914, 0.4822, 0.4465)$ and $\sigma = (0.2470, 0.2435, 0.2616)$. A stratified split reserves 2% of the training set as a validation set used for hyperparameter tuning and early monitoring, while the full 10k test set is kept untouched for final robustness evaluation.

Concretely, for each mini-batch we:

1. Generate adversarial examples x' from clean images x using a 10-step TorchAttacks PGD- ℓ_∞ attack for training, with step size $\alpha = 2/255$ and random initialization inside the ℓ_∞ ball.
2. Perform a forward and backward pass on (x', y) under the cross-entropy loss and update parameters using SGD with momentum 0.9, weight decay 5×10^{-4} , and an initial learning rate of 0.1.
3. Apply a cosine-annealing learning-rate scheduler over the total number of epochs.

Validation is performed at the end of each epoch using a 20-step PGD attack that is stronger than the inner attack. This helps detect under-fitting of the inner maximization without excessively increasing training cost.

We train three adversarially robust models under the same configuration (architecture, optimizer, schedule, threat model), varying only the number of epochs and the inner-attack library:

- **TA-50**: PGD-AT with TorchAttacks PGD- ℓ_∞ , trained for 50 epochs.
- **TA-100**: same as TA-50 but trained for 100 epochs.
- **ART-100**: PGD-AT where adversarial examples are generated by ART’s PGD- ℓ_∞ implementation for 100 epochs.

In addition we keep a *clean* baseline model trained with standard ERM (no adversarial augmentation) on the same data.

5.2.2 Metrics and Logging

All experiments are conducted with a fixed random seed (seed = 42) to ensure reproducibility of the training–validation split, model initialization, and adversarial example generation. The main evaluation metrics follow the definitions in the background chapter:

- **Clean accuracy** $\text{Acc}_{\text{clean}}$: accuracy on unperturbed test images.
- **Per-attack robust accuracy** $\text{Acc}_{\text{robust}}^{\mathcal{A}}$: accuracy on adversarial examples generated by a specific attack $\mathcal{A}$ under its configured norm and radius.
- **AutoAttack robust accuracy** Acc_{AA} : robust accuracy under AutoAttack in its standard configuration. We treat Acc_{AA} as the primary robustness metric in Stage 1.

In later stages we will also compute aggregate robustness metrics across attack families (e.g., average and worst-case robust accuracy), but Stage 1 focuses on per-attack robustness in order to expose cross-attack asymmetries.

Metrics, logging, and diagnostics. To ensure reproducibility and consistent reporting, the experimental pipeline records and outputs the following:

- **Training logs (per epoch).** Training and validation loss/accuracy computed on adversarial examples; optional clean validation loss/accuracy; current learning rate; and a UTC timestamp. These statistics are stored in a JSON history file and automatically plotted into a `training_curves.png` summary.
- **Evaluation outputs.** During evaluation, the Evaluator class records loss and per-attack robust accuracy in a structured dictionary, saves the results to `evaluation_results.json`, and prints a formatted summary table to the console.
- **Attack-strength diagnostics.** Step-sweep and ε -sweep analyses are run at evaluation time (Sections 5.3.3 and 5.3.4) to verify attack strength and assess robustness sensitivity to the optimization budget and perturbation radius.
- **Training-time reporting.** Total wall-clock training time for each run is estimated from the timestamps of the first and last validation events recorded in the training logs and is reported alongside robustness results.

5.3 Robustness Results

5.3.1 Summary Table of Robust Accuracies

Table 5.1 summarizes the Stage 1 robustness results under a diverse set of evaluation-time attacks on CIFAR-10. For each model, we report clean accuracy and robust accuracy (top-1, %) against gradient-based, optimization-based, and decision-boundary attacks under both ℓ_∞ and ℓ_2 threat models.

To reduce evaluation bias, we include multiple implementations of the same attack family when available. In particular, PGD is evaluated using both TorchAttacks and ART under matched hyperparameters, while robustness under strong, parameter-light attacks is assessed using AutoAttack in both its original implementation and the TorchAttacks wrapper. This allows us to verify that the reported robustness trends are not artifacts of a specific library or implementation detail.

All results are computed on the CIFAR-10 test set using the fixed threat models defined in Section 5.1.4. Clean accuracy is included as a reference point for robustness–accuracy trade-off induced by adversarial training. Training times are reported to give a indication of the computational cost associated with each adversarial training configuration.

Table 5.1: Clean and robust accuracies (%) on CIFAR-10 for the naturally trained baseline (“Clean model”) and three PGD-AT models. ℓ_∞ attacks use $\varepsilon = 8/255$; ℓ_2 attacks use $\varepsilon \approx 0.5$. Training times (approximate wall-clock hours) are: TA-50: 4.5 h; TA-100: 6.9 h; ART-100: 10.4 h.

Attack	Clean model	TA-50	TA-100	ART-100
Clean (No Attack)	92.00	82.93	82.51	76.60
FGSM (ℓ_∞)	16.70	55.87	54.92	49.57
PGD (ART, ℓ_∞)	0.00	48.45	48.20	44.71
PGD (ART, ℓ_2)	0.03	60.53	59.71	58.51
PGD (Torch, ℓ_∞)	0.00	48.29	48.25	44.72
PGD (Torch, ℓ_2)	0.03	60.51	59.63	58.52
AutoAttack (orig)	0.00	45.11	44.88	40.84
AutoAttack (Torch std.)	0.00	45.62	45.47	41.33
CW (ℓ_2)	0.54	22.31	26.51	40.03
DeepFool (ℓ_2)	31.52	27.54	29.35	26.65

5.3.2 Key Findings

We highlight three main observations from Table 5.1:

(1) Adversarial training drastically improves robustness. The naturally trained baseline collapses under most strong attacks, with robust accuracy close to 0% under PGD and AutoAttack, despite achieving 92% clean accuracy. In contrast, PGD-AT models attain around 48% robust accuracy under PGD- ℓ_∞ and about 45% under AutoAttack, while keeping clean accuracy in the 76–83% range. This is consistent with the expected clean–robustness trade-off for CIFAR-10 under PGD-based adversarial training.

(2) Diminishing returns beyond 50 epochs. TA-50 and TA-100 have almost identical performance: differences in clean, PGD- ℓ_∞ , and AutoAttack accuracies are below 0.5 percentage points, whereas TA-100 requires roughly $1.5\times$ longer training time. This suggests that, for our configuration, about 50 epochs of PGD-AT are sufficient to reach a robustness plateau, and further epochs mostly increase compute cost without delivering clear gains.

(3) Large cross-attack robustness gaps. Despite being trained under an ℓ_∞ threat model, PGD-AT models show substantial variation across attack families:

- For TA-50, robust accuracy ranges from $\sim 60\%$ under ℓ_2 PGD down to $\sim 22\%$ under CW- ℓ_2 , a gap of almost 40 percentage points.
- AutoAttack remains consistently a bit stronger than PGD- ℓ_∞ , lowering accuracy by about 3 percentage points compared to Torch PGD.
- ART-100 is weaker than TA-50/TA-100 by 3–6 points on most PGD and AutoAttack evaluations, but notably stronger on CW- ℓ_2 (40.0% vs. 22–27%), indicating a shift in which attack family becomes the worst case.

These discrepancies confirm the existence of a pronounced *cross-attack robustness gap* even for models trained with a strong PGD-AT baseline, and they directly motivate the DG-based methods in Stage 2.

5.3.3 PGD Step-Sweep Analysis

To assess whether our PGD evaluations are sufficiently strong, we perform a *step-sweep* by varying the PGD iteration budget $K \in \{1, 2, 5, 10, 20, 50, 100\}$ at evaluation time and recording the resulting robust accuracy. Small- K PGD can substantially overestimate robustness (especially for naturally trained models), while robust models should stabilize once K is sufficiently large.

Figures 5.4 and 5.5 summarize the PGD step-sweep results under both norms; dashed lines mark the corresponding clean accuracies for reference.

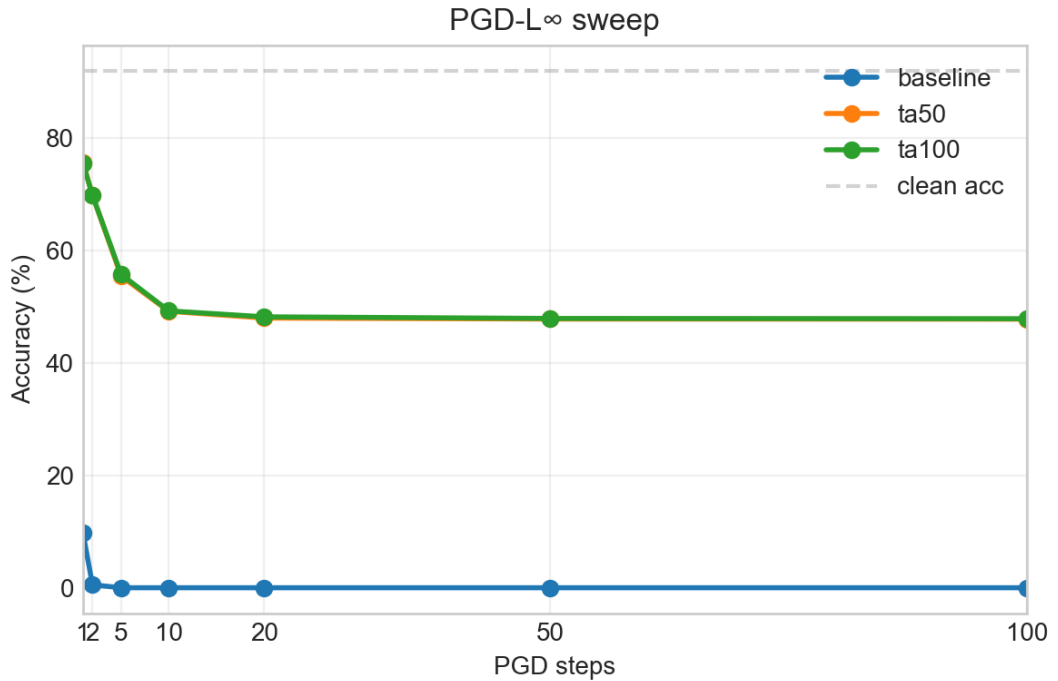


Figure 5.4: PGD- ℓ_∞ step-sweep. Robust accuracy (%) versus evaluation steps K for baseline, TA-50, and TA-100. Dashed horizontal lines indicate each model’s clean accuracy.

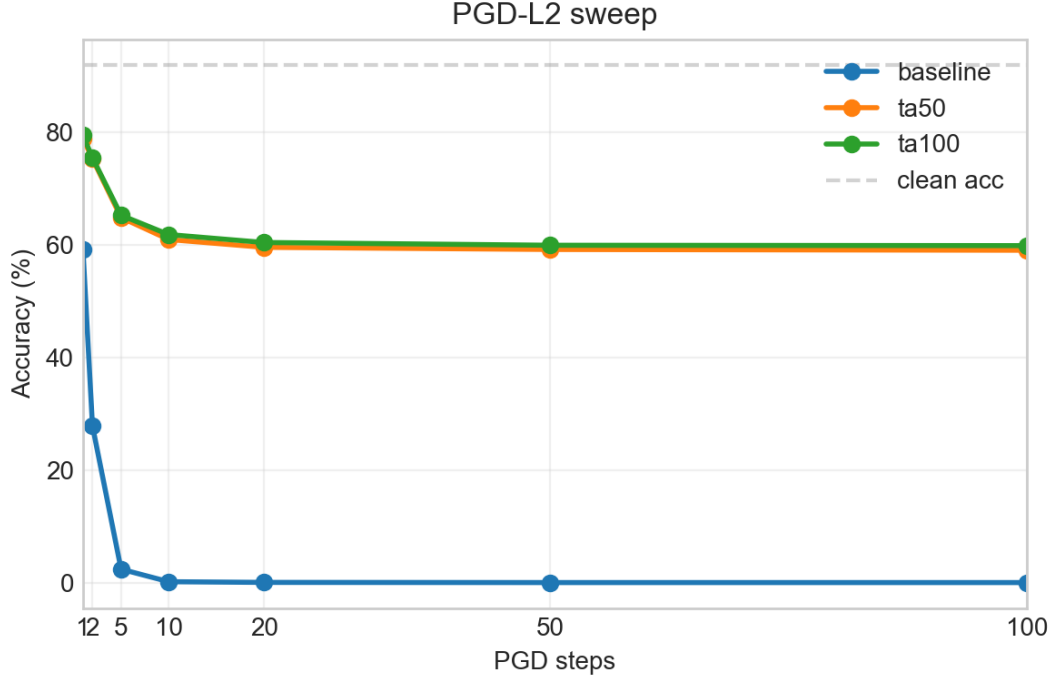


Figure 5.5: PGD- ℓ_2 step-sweep. Robust accuracy (%) versus evaluation steps K for baseline, TA-50, and TA-100. Dashed horizontal lines indicate each model’s clean accuracy.

Interpretation. Two consistent trends emerge. The naturally trained baseline exhibits severe overestimation at small K and drops to near-zero accuracy under sufficiently strong multi-step PGD. In contrast, for adversarially trained checkpoints, robust accuracy decreases as K increases but becomes essentially stable beyond $K \approx 20$.

Practical takeaway. We adopt $K = 20$ (with the fixed ε and restarts in this section) as the default PGD evaluation budget because it avoids overestimation on the baseline while yielding stable robustness estimates for the adversarially trained models.

5.3.4 PGD ℓ_∞ ε -Sweep

We further evaluate sensitivity to the perturbation budget by performing an ε -sweep under the ℓ_∞ threat model. We vary $\varepsilon \in \{0, 5/255, 10/255, 15/255, 20/255, 25/255, 30/255\}$ and report robust accuracy under a fixed PGD- ℓ_∞ configuration (20 steps, 5 restarts), holding all other evaluation settings constant.

Figure 5.6 summarizes the results for the baseline, TA-50, and TA-100. As expected, accuracy decreases monotonically as ε increases, and the adversarially trained models maintain substantially higher robustness across the sweep. Detailed numeric values can be found in Appendix C.1.

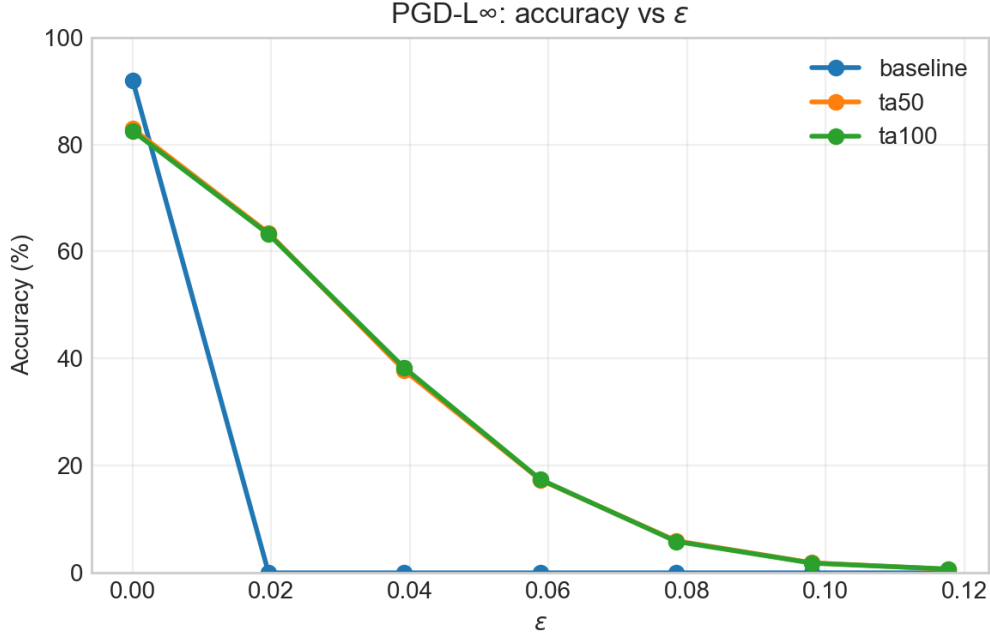


Figure 5.6: PGD- ℓ_∞ ϵ -sweep. Robust accuracy (%) as a function of the perturbation budget ϵ for the naturally trained baseline, TA-50, and TA-100, evaluated with a fixed 20-step, 5-restart PGD- ℓ_∞ attack.

5.3.5 Qualitative Attack Diagnostics

To complement the quantitative results, we provide qualitative visualizations of adversarial examples. These figures are used for intuition and sanity checks only (not as evidence of robustness). We visualize a fixed set of $N = 8$ test images selected deterministically (first correctly classified samples under a fixed seed) and apply the same evaluation-time attack settings described in Sections 5.3.3 and 5.3.4. In all grids, Δ denotes the signed pixel-space difference ($x_{\text{adv}} - x_{\text{clean}}$) shown as a red/blue heatmap (scaled for visibility; no other post-processing is applied).

Norm comparison. Figure 5.7 contrasts PGD- ℓ_∞ and PGD- ℓ_2 perturbations on the same inputs. In comparison, ℓ_∞ perturbations tend to appear as higher-frequency pixel-level noise, whereas ℓ_2 perturbations are often smoother and more spatial.

To facilitate interpretation, we briefly describe the layout of the qualitative grids. For each input, the clean image is shown once, followed by paired blocks corresponding to different threat models or attacks. Within each block, the perturbation $\Delta = x_{\text{adv}} - x_{\text{clean}}$ is visualized as a signed red/blue heatmap, and the resulting adversarial example is shown alongside. Perturbation maps are independently scaled for visibility only. Colored borders indicate whether the model’s prediction is correct (green) or incorrect (red).

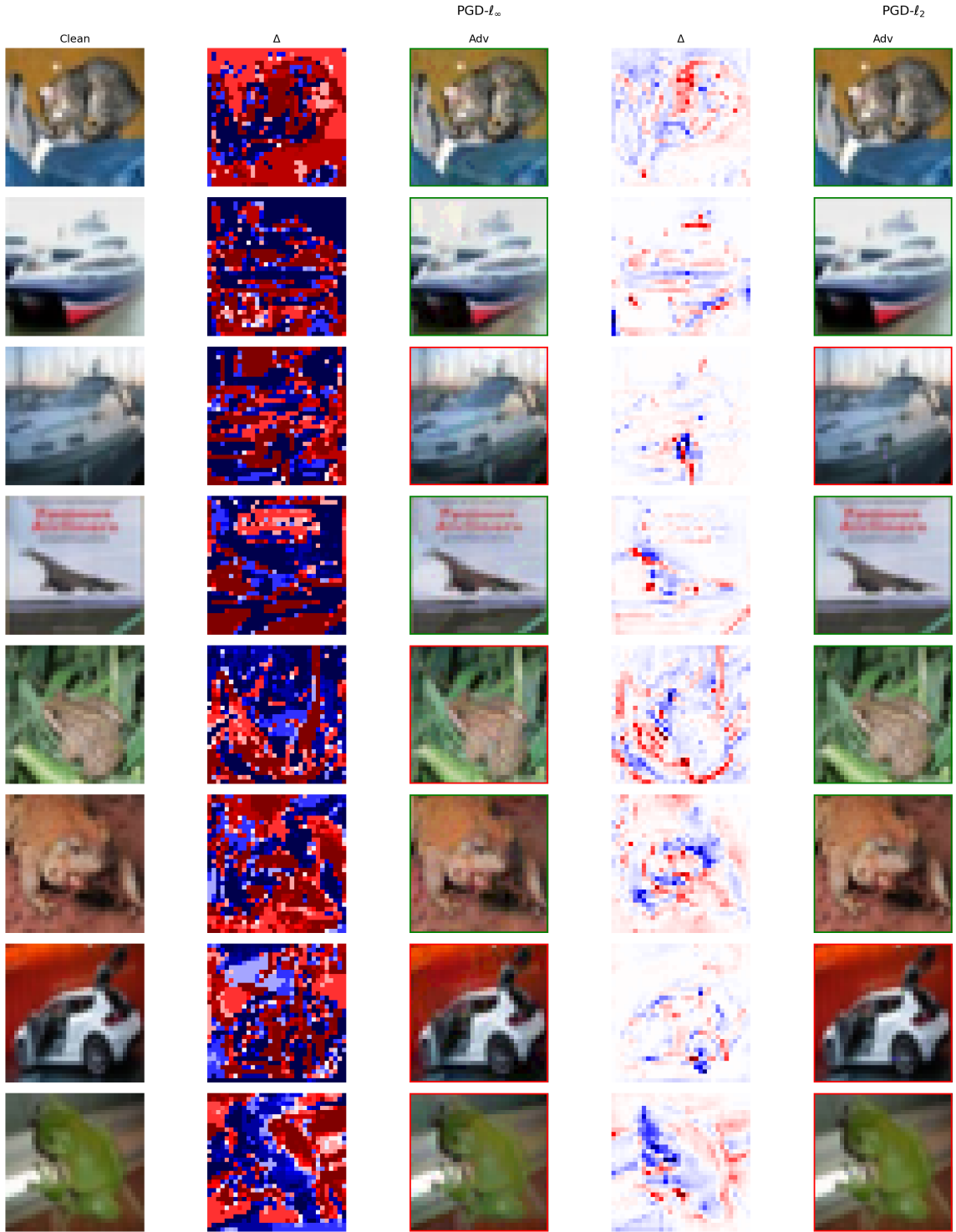


Figure 5.7: Qualitative norm comparison on a fixed set of CIFAR-10 test images. Columns show the clean image, perturbation visualization and the adversarial example.

Attack-family comparison. Figure 5.8 compares perturbation patterns across different attack families on the same inputs. PGD- ℓ_∞ and AutoAttack often produce visually similar, small-amplitude perturbations, whereas CW- ℓ_2 tends to yield more structured perturbations in pixel space. This helps interpret the cross-attack robustness gaps reported in Table 5.1. Color border indication is the same as in Figure 5.7.

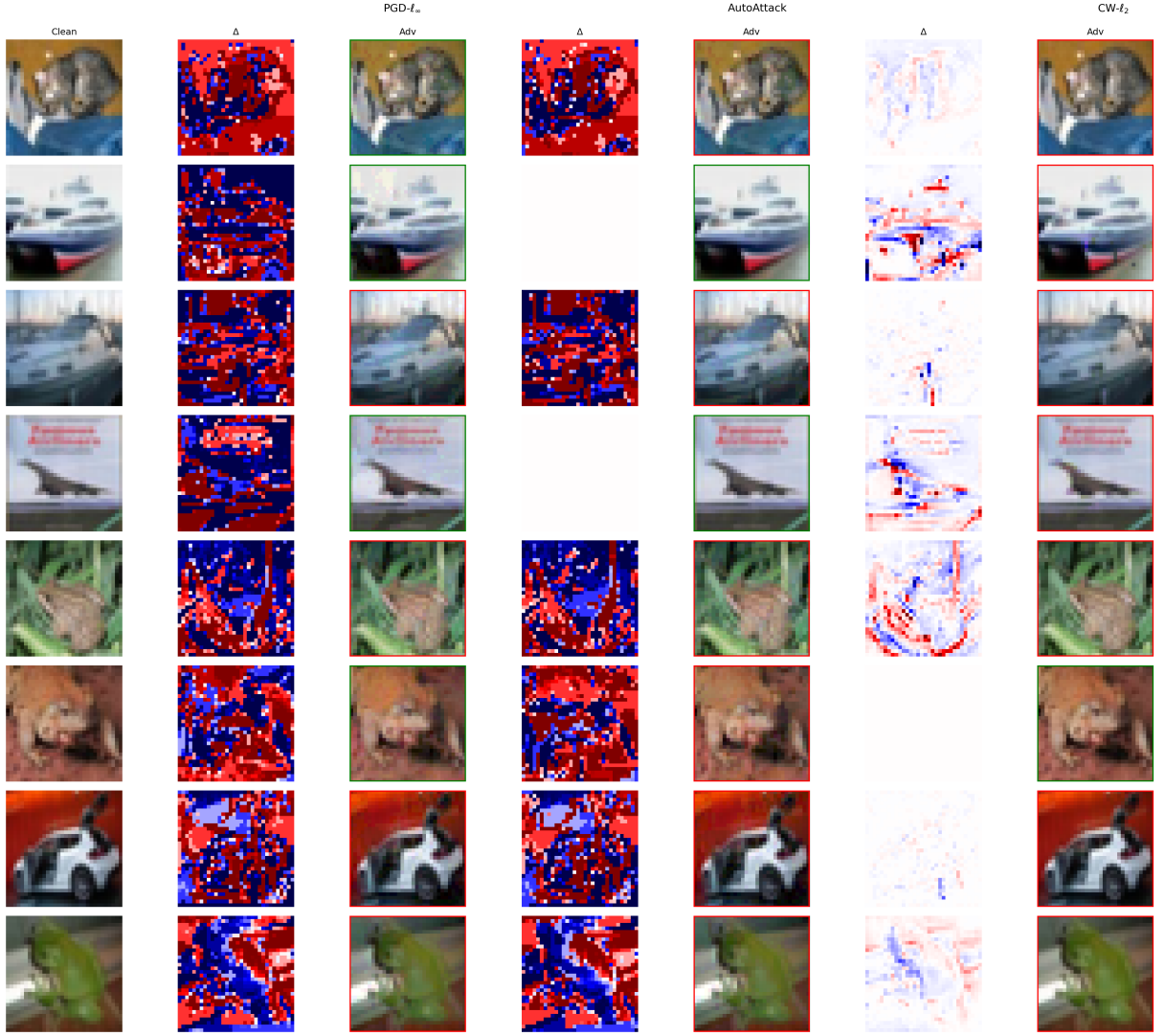


Figure 5.8: Qualitative comparison across attack families on the same fixed set of test images. For each attack, we show the clean image, perturbation visualization ($\Delta = x_{\text{adv}} - x_{\text{clean}}$), and adversarial example. Columns correspond to PGD- ℓ_∞ , AutoAttack, and CW- ℓ_2 .

5.3.6 Explained Insights

The grids provide intuition for the cross-attack gaps observed in Table 5.1:

- **PGD- ℓ_∞ vs. PGD- ℓ_2 .** Visually, ℓ_∞ perturbations appear as high-frequency noise that slightly alters many pixels, while ℓ_2 perturbations tend to be smoother and more global. Although both are nearly imperceptible, the model achieves higher accuracy against ℓ_2 PGD ($\sim 60\%$) than against ℓ_∞ PGD ($\sim 48\%$), suggesting that the ℓ_∞ threat model is stricter in terms of decision-boundary margins. This matches the expectation that robustness certified for ℓ_∞ transfers partially but not symmetrically to ℓ_2 .
- **AutoAttack vs. PGD- ℓ_∞ .** AutoAttack often finds perturbations that are qualitatively similar to PGD- ℓ_∞ but that alter the image in a more targeted way

(e.g., focusing on discriminative regions). This is consistent with AutoAttack lowering robust accuracy by a few extra points compared to a standard PGD- ℓ_∞ evaluation attack under the same threat model.

- **CW- ℓ_2 and DeepFool.** CW- ℓ_2 and DeepFool generate perturbations that look less noisy but significantly alter the classifier’s logits. For TA-50 and TA-100, CW- ℓ_2 is the strongest attack by a large margin (around 22–27% robust accuracy), whereas ART-100 is much more resilient to CW- ℓ_2 but slightly weaker to PGD and AutoAttack. This qualitative difference supports the view that each attack family acts as a distinct domain with its own decision-boundary geometry.

Overall, the visualizations reinforce that small, almost invisible perturbations can exploit model vulnerabilities in substantially different ways depending on the attack algorithm and norm.

5.4 Main Limitations and Takeaways

While Stage 1 successfully delivers a working PGD-AT baseline and a multi-attack evaluation harness, several limitations remain:

- **Single dataset and backbone.** All experiments are conducted on CIFAR-10 with a single ResNet-18 architecture. This is sufficient to study cross-attack gaps qualitatively, but it limits conclusions about robustness trends on larger-scale datasets or higher-capacity models (e.g., WideResNet-28–10).
- **Single inner attack per run.** Each adversarially trained model uses a single training attack (PGD- ℓ_∞ via TorchAttacks or ART). We do not yet mix attack families during training, nor do we use domain-balanced sampling across attacks. As a result, models tend to specialize to the inner attack and show large performance gaps on other attack families.
- **Partial metrics.** Stage 1 focuses on per-attack robust accuracy but does not yet compute average or worst-case robustness across all attacks, nor confidence intervals. These aggregate metrics are crucial for fair comparison in Stage 2.
- **Compute cost.** Adversarial training remains expensive: TA-50 already requires several hours of GPU time, and ART-100 is the slowest yet not the most robust model. This highlights the need for training strategies that either reduce cost or allocate compute more intelligently across attack domains.

The key takeaway is that standard PGD-AT produces models that are reasonably robust to the attack used during training but still vulnerable to other attacks, especially those with different optimization objectives (e.g., CW) or norms. This behavior fits naturally into a domain-generalization perspective: each attack domain induces its own loss distribution, and training on one domain does not automatically guarantee good performance on others.

5.5 Summary and Transition to Stage 2

In Stage 1 we have:

- Implemented a reproducible PGD-AT pipeline on CIFAR-10 with a ResNet-18 backbone, including mixed-precision training, checkpointing, and experiment logging.
- Built a unified attack harness that integrates FGSM, PGD, CW, DeepFool, and AutoAttack across both TorchAttacks and ART, and used it to perform systematic robustness evaluation.
- Quantified substantial cross-attack robustness gaps for standard PGD-AT models, both across norms (ℓ_∞ vs. ℓ_2) and algorithms (PGD/AutoAttack vs. CW/DeepFool), despite comparable perturbation budgets.

These findings motivate the design of Stage 2. If each attack family is interpreted as a separate domain, then the cross-attack robustness gap can be rephrased as an *out-of-domain generalization* problem: training on some attack domains (e.g., PGD- ℓ_∞) should yield models that generalize to unseen or under-represented domains (e.g., CW, DeepFool, new AutoAttack variants).

The next chapter concludes the report by summarizing the Stage 1 findings and outlining the planned Stage 2 directions under the attacks-as-domains perspective, including the evaluation protocol used to assess whether the proposed DG-enhanced methods reduce the robustness gaps observed in Stage 1.

Chapter 6

Conclusion and Future Work

This chapter consolidates the completed Stage 1 outcomes, interprets them under the *attacks-as-domains* viewpoint, and delineates a Stage 2 roadmap that is explicitly *planned* (not implemented in this report). The Stage 2 plan follows the framework, DG objective prioritization, and evaluation design established in Chapter 4.

6.1 Summary of Stage 1 Outcomes

Stage 1 delivers a reproducible adversarial training and evaluation pipeline on CIFAR-10 with a CIFAR-style ResNet-18 backbone. A key engineering outcome is a unified attack harness covering: (i) gradient-based attacks (FGSM; PGD- ℓ_∞ and PGD- ℓ_2 via TorchAttacks and ART), (ii) optimization-based attacks (CW- ℓ_2), (iii) boundary-based attacks (DeepFool- ℓ_2), and (iv) standardized evaluation with AutoAttack. In addition, Stage 1 establishes logging/reporting conventions that make cross-attack comparisons repeatable.

Table 5.1 summarizes the main empirical outcomes:

- **Adversarial training yields large robustness gains over clean ERM.** The naturally trained baseline achieves 92.00% clean accuracy, but collapses to near-zero robust accuracy under strong multi-step attacks and AutoAttack. In contrast, PGD-AT models achieve about 48% robust accuracy under PGD- ℓ_∞ and about 45% under AutoAttack, while retaining clean accuracy in the 76–83% range.
- **Robustness does not transfer uniformly across attack families.** Even for PGD-AT models, robust accuracy varies substantially across attacks. For example, TA-50 attains about 60.5% under PGD- ℓ_2 but drops to 22.31% under CW- ℓ_2 , indicating pronounced attack-specific sensitivity despite comparable perturbation budgets.
- **Diminishing returns beyond 50 epochs under the current setup.** TA-50 and TA-100 are nearly identical on core benchmarks such as PGD- ℓ_∞ and AutoAttack

(differences < 0.5 percentage points), while TA-100 requires substantially more compute.

Overall, Stage 1 supports the central motivation of this project—robustness obtained from a single training attack (or a single threat model) does not automatically generalize to diverse adversarial attacks encountered at evaluation time.

Lessons learned from Stage 1.

- Attack-specific overfitting is visible: PGD-AT holds up on PGD- ℓ_∞ but drops sharply on CW- ℓ_2 and AutoAttack.
- AutoAttack is the hardest and most informative metric; it should anchor Stage 2 model selection.
- Compute scales quickly with epochs; gains plateau after ~ 50 epochs under current settings, so Stage 2 should prioritize method changes over longer training.

6.2 Implications Under the Attacks-as-Domains View

The Stage 1 findings imply several points that directly motivate the Stage 2 design.

- **Attack-specific overfitting is empirically visible.** Strong performance against the training-style threat model does not guarantee robustness under attacks with different objectives or search behaviors (e.g., CW, AutoAttack).
- **Attack shifts create multiple axes of distribution change.** Differences in norm constraints (ℓ_∞ vs. ℓ_2), attack implementations (TorchAttacks vs. ART), and attack families (PGD vs. CW vs. AutoAttack) can be operationalized as distinct *domains*. This provides a concrete basis for importing DG-style objectives into adversarial training.
- **AutoAttack provides a stringent target metric.** In Stage 1, AutoAttack behaves as a consistently stronger evaluation than standard PGD- ℓ_∞ and is treated as a hard robustness target for validating cross-attack generalization improvements.
- **Compute-aware choices are necessary.** Training time scales sharply with adversarial training configurations (e.g., TA-50 vs. TA-100 vs. ART-100), while robustness improvements on the main benchmarks can plateau early. This motivates Stage 2 objectives that improve generalization *without* relying primarily on longer training.

6.3 Future Work: Stage 2 Plan

Stage 2 extends the Stage 1 foundation while keeping the evaluation pipeline fixed, following the framework and evaluation design specified in Chapter 4.

- **DG-integrated adversarial training (planned).** Implement multi-attack adversarial training and integrate DG-inspired objectives prioritized in Chapter 4,

with an initial emphasis on methods that are (i) low-overhead, (ii) stable under min—max optimization, and (iii) directly relevant to attack shifts. Candidate objectives include variance-control penalties (REx-style), worst-domain emphasis (GroupDRO-style), and low-overhead alignment regularizers (e.g., cross-domain Mixup / output-consistency). Higher-cost invariance constraints (IRM-style) are reserved as exploratory candidates if stability and budget allow.

- **Cross-attack evaluation with fixed, stage-invariant settings (planned).** Report per-attack robust accuracy and use AutoAttack as the primary hard metric for model selection under the attacks-as-domains viewpoint. Additional aggregate summaries (worst/average across attacks) will be added to quantify robustness dispersion across domains.
- **Ablations and sensitivity studies (planned).** Study sensitivity to the choice and number of source attack domains, and to DG regularization strengths, and explicitly report robustness—accuracy—compute trade-offs.
- **Optional scope extensions (planned).** If resources permit, extend beyond CIFAR-10 (e.g., CIFAR-100) and/or evaluate robustness beyond adversarial domains (e.g., corruption robustness) to test whether DG-integrated adversarial training improves broader generalization.

6.4 Closing Remarks

This project links adversarial robustness and domain generalization by treating attack families as domains and adapting DG-inspired objectives for adversarial training. Stage 1 establishes a measurement-ready pipeline and demonstrates pronounced cross-attack robustness gaps even under strong PGD-AT baselines. Stage 2 will test whether DG-integrated training can reduce these gaps and yield robustness that generalizes more reliably across adversarial domains, with AutoAttack serving as the primary hard validation target.

Bibliography

- [1] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” *International Conference on Learning Representations*, 2019.
- [2] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” *International Conference on Learning Representations*, 2015.
- [3] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, “Intriguing properties of neural networks,” 2014. [Online]. Available: <https://arxiv.org/abs/1312.6199>
- [4] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, “Distillation as a defense to adversarial perturbations against deep neural networks,” 2016. [Online]. Available: <https://arxiv.org/abs/1511.04508>
- [5] N. Carlini and D. Wagner, “Towards evaluating the robustness of neural networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1608.04644>
- [6] A. Athalye, N. Carlini, and D. A. Wagner, “Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples,” *CoRR*, vol. abs/1802.00420, 2018. [Online]. Available: <http://arxiv.org/abs/1802.00420>
- [7] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, “Deepfool: a simple and accurate method to fool deep neural networks,” 2016. [Online]. Available: <https://arxiv.org/abs/1511.04599>
- [8] F. Croce and M. Hein, “Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks,” *CoRR*, vol. abs/2003.01690, 2020. [Online]. Available: <https://arxiv.org/abs/2003.01690>
- [9] F. Croce, M. Andriushchenko, V. Sehwag, N. Flammarion, M. Chiang, P. Mittal, and M. Hein, “Robustbench: a standardized adversarial robustness benchmark,” *CoRR*, vol. abs/2010.09670, 2020. [Online]. Available: <https://arxiv.org/abs/2010.09670>

- [10] F. Croce and M. Hein, “Minimally distorted adversarial examples with a fast adaptive boundary attack,” *CoRR*, vol. abs/1907.02044, 2019. [Online]. Available: <http://arxiv.org/abs/1907.02044>
- [11] M. Andriushchenko, F. Croce, N. Flammarion, and M. Hein, “Square attack: a query-efficient black-box adversarial attack via random search,” *CoRR*, 2019. [Online]. Available: <https://arxiv.org/abs/1912.00049>
- [12] H. Zhang, Y. Yu, J. Jiao, E. P. Xing, L. E. Ghaoui, and M. I. Jordan, “Theoretically principled trade-off between robustness and accuracy,” *CoRR*, vol. abs/1901.08573, 2019. [Online]. Available: <http://arxiv.org/abs/1901.08573>
- [13] E. Wong, L. Rice, and J. Z. Kolter, “Fast is better than free: Revisiting adversarial training,” *CoRR*, vol. abs/2001.03994, 2020. [Online]. Available: <https://arxiv.org/abs/2001.03994>
- [14] D. Wu, S. tao Xia, and Y. Wang, “Adversarial weight perturbation helps robust generalization,” 2020. [Online]. Available: <https://arxiv.org/abs/2004.05884>
- [15] J. Wang, C. Lan, C. Liu, Y. Ouyang, T. Qin, Y. Ma, H. Chen, and P. Zhao, “Generalizing to unseen domains: A survey on domain generalization,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- [16] L. Rice, E. Wong, and J. Z. Kolter, “Overfitting in adversarially robust deep learning,” in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 8093–8104. [Online]. Available: <https://proceedings.mlr.press/v119/rice20a.html>
- [17] S. Dai, S. Mahloujifar, C. Xiang, V. Sehwag, P.-Y. Chen, and P. Mittal, “Multirobustbench: Benchmarking robustness against multiple attacks,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 6760–6785. [Online]. Available: <https://proceedings.mlr.press/v202/dai23c.html>
- [18] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz, “Invariant risk minimization,” *arXiv preprint arXiv:1907.02893*, 2019. [Online]. Available: <https://arxiv.org/abs/1907.02893>
- [19] D. Stutz, M. Hein, and B. Schiele, “Disentangling adversarial robustness and generalization,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 6976–6987. [Online]. Available:

https://openaccess.thecvf.com/content_CVPR_2019/html/Stutz_Disentangling_Adversarial_Robustness_and_Generalization_CVPR_2019_paper.html

- [20] T. Gokhale, S. Mishra, M. Luo, B. Sachdeva, and C. Baral, “Generalized but not robust? comparing the effects of data modification methods on out-of-domain generalization and adversarial robustness,” in *Findings of the Association for Computational Linguistics: ACL 2022*. Association for Computational Linguistics, 2022, pp. 2705–2718. [Online]. Available: <https://aclanthology.org/2022.findings-acl.213>
- [21] C. Song, X. He, Y. Wang, K. He, and J. E. Hopcroft, “Improving the generalization of adversarial training with domain adaptation,” in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://arxiv.org/abs/1810.00740>
- [22] Y. Wang, J.-Y. Wu, and K. Xu, “Exact feature distribution matching for arbitrary style transfer and domain generalization,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 7243–7252. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2022/html/Wang_Exact_Feature_Distribution_Matching_for_Arbitrary_Style_Transfer_and_Domain_CVPR_2022_paper.html
- [23] O. Poursaeed, T. Jiang, H. Yang, S. J. Belongie, and S. Lim, “Robustness and generalization via generative adversarial training,” *CoRR*, vol. abs/2109.02765, 2021. [Online]. Available: <https://arxiv.org/abs/2109.02765>
- [24] M. Awais, F. Zhou, H. Xu, Z. Li, Y. Hong, and S.-H. Bae, “Adversarial robustness for unsupervised domain adaptation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 8568–8577. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2021/html/Awais_Adversarial_Robustness_for_Unsupervised_Domain_Adaptation_ICCV_2021_paper.html
- [25] Y. Wang, D. Zou, J. Yi, J. Bailey, X. Ma, and Q. Gu, “Improving adversarial robustness requires revisiting misclassified examples,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=rklOg6EFwS>
- [26] D. Krueger, E. Caballero, J. Jacobsen, A. Zhang, J. Binas, R. L. Priol, and A. C. Courville, “Out-of-distribution generalization via risk extrapolation (rex),” *CoRR*, vol. abs/2003.00688, 2020. [Online]. Available: <https://arxiv.org/abs/2003.00688>

- [27] I. Gulrajani and D. Lopez-Paz, “In search of lost domain generalization,” in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=lQdXeXDoWtI>
- [28] K. Alhamoud, H. Abed Al Kader Hammoud, M. Alfarra, and B. Ghanem, “Generalizability of adversarial robustness under distribution shifts,” *Transactions on Machine Learning Research*, 2023. [Online]. Available: <https://openreview.net/forum?id=XNFo3dQiCJ>
- [29] A. Ibrahim, C. Guille-Escuret, I. Mitliagkas, I. Rish, D. Krueger, and P. Bashivan, “Towards out-of-distribution adversarial robustness,” *arXiv preprint arXiv:2210.03150*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.03150>
- [30] S. Dai, S. Mahlouljifar, C. Xiang, V. Sehwag, P.-Y. Chen, and P. Mittal, “Multirobustbench: Benchmarking robustness against multiple attacks,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.10980>
- [31] S. Sagawa, P. W. Koh, T. B. Hashimoto, and P. Liang, “Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization,” *arXiv preprint arXiv:1911.08731*, 2019. [Online]. Available: <https://arxiv.org/abs/1911.08731>
- [32] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond empirical risk minimization,” 2018. [Online]. Available: <https://arxiv.org/abs/1710.09412>
- [33] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Technical Report, 2009.
- [34] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>

Appendix A

Pseudo-code and Flowcharts

This appendix collects illustrative pseudo-code and flowcharts for key adversarial attacks and the training/evaluation pipelines introduced in Chapter 2. The purpose is conceptual clarity and documentation of algorithmic structure at a high level. These materials are not intended to represent implementation-level details, optimized code, or exact hyperparameter settings. All empirical results reported in the main chapters are obtained using the actual implementations and experimental pipeline described in Chapter 5.

Organization. We first summarize individual attack procedures (FGSM, PGD, CW- ℓ_2 , DeepFool, and AutoAttack’s Auto-PGD component), followed by generic adversarial-training formulations used for exposition. Flowcharts for the training and evaluation pipelines are included at the end of this appendix.

A.1 FGSM Attack

The following pseudo-code summarizes the high-level procedure of the FGSM attack, as described in Section 2.2.2.

Algorithm 5: FGSM ℓ_∞ Attack (Targeted)

Input: classifier f_θ , loss ℓ , input x , target label y_{target} , perturbation budget ε

Output: adversarial example x'

- 1 $g \leftarrow \nabla_x \ell(f_\theta(x), y_{\text{target}})$
 - 2 $\delta \leftarrow \varepsilon \cdot \text{sign}(g)$
 - 3 $x' \leftarrow \text{clip}(x - \delta, 0, 1)$
 - 4 **return** x'
-

Remark. The pseudo-code above is illustrative and focuses on the core update rule under an ℓ_∞ constraint. Experimental runs use the standardized implementations documented in Chapter 5.

A.2 PGD Attack

The following pseudo-code summarizes the high-level procedure of the PGD attack, as described in Section 2.2.3.

Algorithm 6: PGD ℓ_∞ Attack (Targeted)

Input: classifier f_θ , loss $\ell(\cdot, y)$, clean input x , target label y_{target} , budget $\varepsilon > 0$, step size $\alpha > 0$, iterations N .

Output: adversarial example x' .

```

1 Initialize  $x^{(0)} \leftarrow x + \delta^{(0)}$  where  $\delta^{(0)} \sim \text{Uniform}(\mathcal{S}_\infty(\varepsilon))$ 
2 for  $t = 0$  to  $N - 1$  do
3    $g^{(t)} \leftarrow \nabla_x \ell(f_\theta(x^{(t)}), y_{\text{target}})$ 
4    $\hat{x}^{(t+1)} \leftarrow x^{(t)} - \alpha \cdot \text{sign}(g^{(t)})$ 
5    $\delta^{(t+1)} \leftarrow \text{clip}(\hat{x}^{(t+1)} - x, -\varepsilon, +\varepsilon)$ 
6    $x^{(t+1)} \leftarrow \text{clip}(x + \delta^{(t+1)}, 0, 1)$ 
7 end
8 return  $x' \leftarrow x^{(N)}$ 

```

Remark. This presentation emphasizes the iterative projected update under an ℓ_∞ constraint.

A.3 Carlini & Wagner Attack

ℓ_2 attack – canonical formulation (illustrative)

Algorithm 7: Carlini & Wagner ℓ_2 attack (illustrative; based on the original formulation)

Input: Image $x_0 \in [0, 1]^d$, classifier logits $Z(\cdot)$, true label y , optional target label $t \neq y$, confidence $\kappa \geq 0$

Output: Adversarial example x_{adv} with small ℓ_2 distortion

```

1 Initialize  $w \leftarrow \text{arctanh}(2x_0 - 1)$ ,  $c \leftarrow 10^{-3}$  (lower bound)
2 while not converged do
3    $c \leftarrow$  binary search to minimal value such that the attack succeeds
4   Initialize Adam optimizer on  $w$ 
5   for a fixed number of iterations do
6      $\delta_i \leftarrow \frac{1}{2}(\tanh(w_i) + 1) - x_{0,i}$ 
7      $x' \leftarrow x_0 + \delta$ 
8      $\text{loss} \leftarrow \|\delta\|_2^2 + c \cdot \max(\max_{i \neq t} Z(x')_i - Z(x')_t, -\kappa)$ 
9     Update  $w$  by applying Adam step on loss
10    if  $Z(x')_t \geq \max_{i \neq t} Z(x')_i + \kappa$  then
11      break
12    end
13  end
14 end
15  $x_{\text{adv}} \leftarrow x_0 + \delta$ 
16 return  $x_{\text{adv}}$ 

```

Remark. This pseudo-code is illustrative. Unless otherwise stated, experiments use the

untargeted configuration consistent with Chapter 5.

A.4 DeepFool Attack

The following pseudo-code is adapted (with minor notational modernization) from the DeepFool paper [7].

Algorithm 8: DeepFool – Binary classification case (theoretical foundation)

Input: Image x_0 , differentiable binary classifier $f: \mathbb{R}^d \rightarrow \mathbb{R}$
Output: Minimal perturbation $\hat{r}$

```

1  $i \leftarrow 0$ 
2 while  $\text{sign}(f(x_i)) = \text{sign}(f(x_0))$  do
3    $r_i \leftarrow -\frac{f(x_i)}{\|\nabla f(x_i)\|_2^2} \nabla f(x_i)$ 
4    $x_{i+1} \leftarrow x_i + r_i$ 
5    $i \leftarrow i + 1$ 
6 end
7  $\hat{r} \leftarrow \sum_{j=0}^{i-1} r_j$ 
8 return  $\hat{r}$ 

```

Algorithm 9: DeepFool – Multi-class case (Algorithm 2 in the original paper – used in all experiments)

Input: Image $x_0 \in [0, 1]^d$, classifier $f(x) = [f_1(x), \dots, f_C(x)]$,
predicted label $\hat{k}(x) = \arg \max_k f_k(x)$
Output: Minimal adversarial perturbation $\hat{r}$

```

1 Initialize  $x_0 \leftarrow x$ ,  $i \leftarrow 0$ 
2 while  $\hat{k}(x_i) = \hat{k}(x_0)$  do
3   for  $k \neq \hat{k}(x_i)$  do
4      $w_k \leftarrow \nabla f_k(x_i) - \nabla f_{\hat{k}(x_i)}(x_i)$ 
5      $f'_k \leftarrow f_k(x_i) - f_{\hat{k}(x_i)}(x_i)$ 
6   end
7    $\hat{l} \leftarrow \arg \min_{k \neq \hat{k}(x_i)} \frac{|f'_k|}{\|w_k\|_2}$ 
8    $r_i \leftarrow -\frac{|f'_{\hat{l}}|}{\|w_{\hat{l}}\|_2^2} w_{\hat{l}}$ 
9    $x_{i+1} \leftarrow x_i + r_i$ 
10   $i \leftarrow i + 1$ 
11 end
12  $\hat{r} \leftarrow \sum_j r_j$ 
13 return  $\hat{r}$ 

```

Remark. The pseudo-code above is included for conceptual completeness; the report’s empirical results use standardized implementations and configurations defined in Chapter 5.

A.5 Auto-PGD attack

Flowchart. The overall workflow is illustrated in Figure A.1.

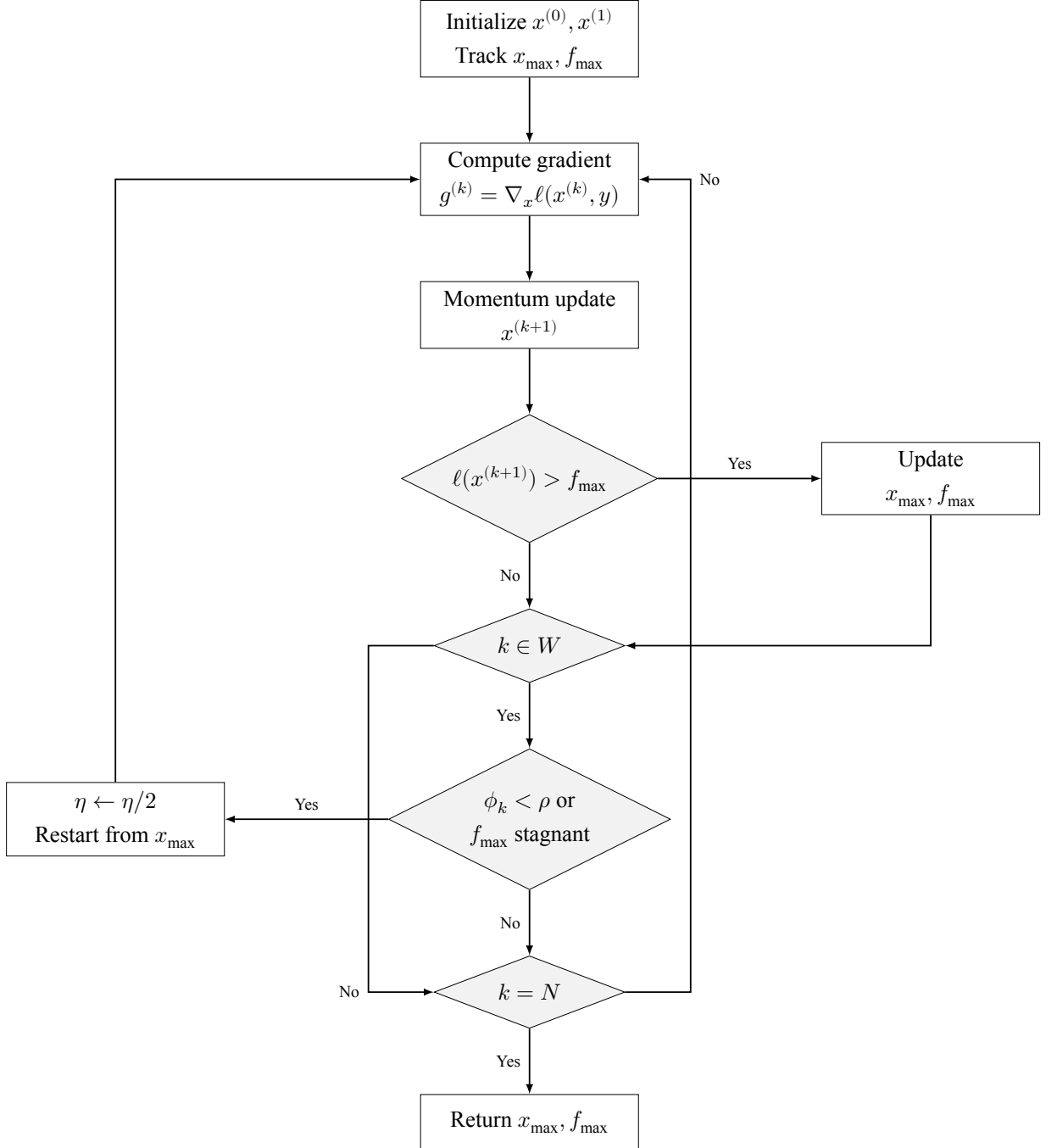


Figure A.1: APGD algorithm workflow with adaptive step-size control.

Figure A.1 illustrates the high-level control flow of Auto-PGD. Unlike standard PGD with a fixed step size, APGD dynamically adapts the step size based on optimization progress. The algorithm tracks the best adversarial iterate observed so far and periodically checks whether recent updates are effective. If progress stalls, the step size is reduced and the attack is restarted from the current best point, improving robustness against poor initializations and non-smooth loss landscapes.

Pseudocode. Algorithm 10 presents the complete APGD procedure, which forms the core of AutoAttack’s first two components. We have $S = (x + \mathcal{S}_p(\varepsilon)) \cap [0, 1]^d$ denotes the feasible adversarial region, $\Pi_S(\cdot)$ denotes projection onto S , and $\text{sign}(\cdot)$ denotes the element-wise sign function. For ℓ_∞ attacks, $\text{sign}(g)$ directly gives the ascent direction; for ℓ_2 attacks, the gradient would be normalized: $g/\|g\|_2$.

Algorithm 10: Auto-PGD (APGD) with Adaptive Step-Size Scheduling

Input: classifier f_θ , loss $\ell(\cdot, y)$, input x , label y , threat set S , step size η , iterations N , momentum α , checkpoints W , threshold ρ

Output: adversarial example $x_{\max}$, maximum loss $f_{\max}$

```

1 Initialize  $x^{(0)} \leftarrow x + \delta^{(0)}$  where  $\delta^{(0)} \sim \text{Uniform}(\mathcal{S}_p(\varepsilon))$ 
2  $x_{\max} \leftarrow x^{(0)}$ ,  $f_{\max} \leftarrow \ell(x^{(0)}, y)$ 
3  $x^{(1)} \leftarrow \Pi_S(x^{(0)} + \eta \text{sign}(\nabla_x \ell(x^{(0)}, y)))$ 
4 if  $\ell(x^{(1)}, y) > f_{\max}$  then  $x_{\max} \leftarrow x^{(1)}$ ,  $f_{\max} \leftarrow \ell(x^{(1)}, y)$ 
5 for  $k = 1$  to  $N - 1$  do
6    $g^{(k)} \leftarrow \nabla_x \ell(x^{(k)}, y)$ 
7    $z^{(k+1)} \leftarrow \Pi_S(x^{(k)} + \eta \text{sign}(g^{(k)}))$ 
8    $x^{(k+1)} \leftarrow \Pi_S(x^{(k)} + \alpha(z^{(k+1)} - x^{(k)}) + (1 - \alpha)(x^{(k)} - x^{(k-1)}))$ 
9    $f^{(k+1)} \leftarrow \ell(x^{(k+1)}, y)$ 
10  if  $f^{(k+1)} > f_{\max}$  then  $x_{\max} \leftarrow x^{(k+1)}$ ,  $f_{\max} \leftarrow f^{(k+1)}$ 
11  if  $k \in W$  then
12     $I_k \leftarrow$  indices since last checkpoint
13     $n_{\text{succ}} \leftarrow |\{t \in I_k : f^{(t+1)} > f^{(t)}\}|$ 
14     $\phi_k \leftarrow n_{\text{succ}}/|I_k|$ 
15    if  $\phi_k < \rho$  or  $f_{\max}$  unchanged since last  $\eta$  reduction then
16       $\eta \leftarrow \eta/2$ 
17       $x^{(k+1)} \leftarrow x_{\max}$ ,  $f^{(k+1)} \leftarrow f_{\max}$ 
18    end
19  end
20 end
21 return  $x_{\max}, f_{\max}$ 

```

Remark. AutoAttack is used strictly as an evaluation suite in the main experiments; the appendix includes APGD for algorithmic context.

Algorithm 10 formalizes APGD as an adaptive first-order attack that combines momentum, checkpoint-based progress tests, and automatic step-size scheduling. These mechanisms remove the need for manual tuning of the step size and number of restarts, making APGD a strong and reliable adversary for robustness evaluation. In AutoAttack, APGD serves as the core gradient-based component for both ℓ_∞ and ℓ_2 threat models and is designed to reduce the risk of gradient masking during evaluation.

A.6 Training Pipeline Flowcharts - ERM vs Adversarial Training

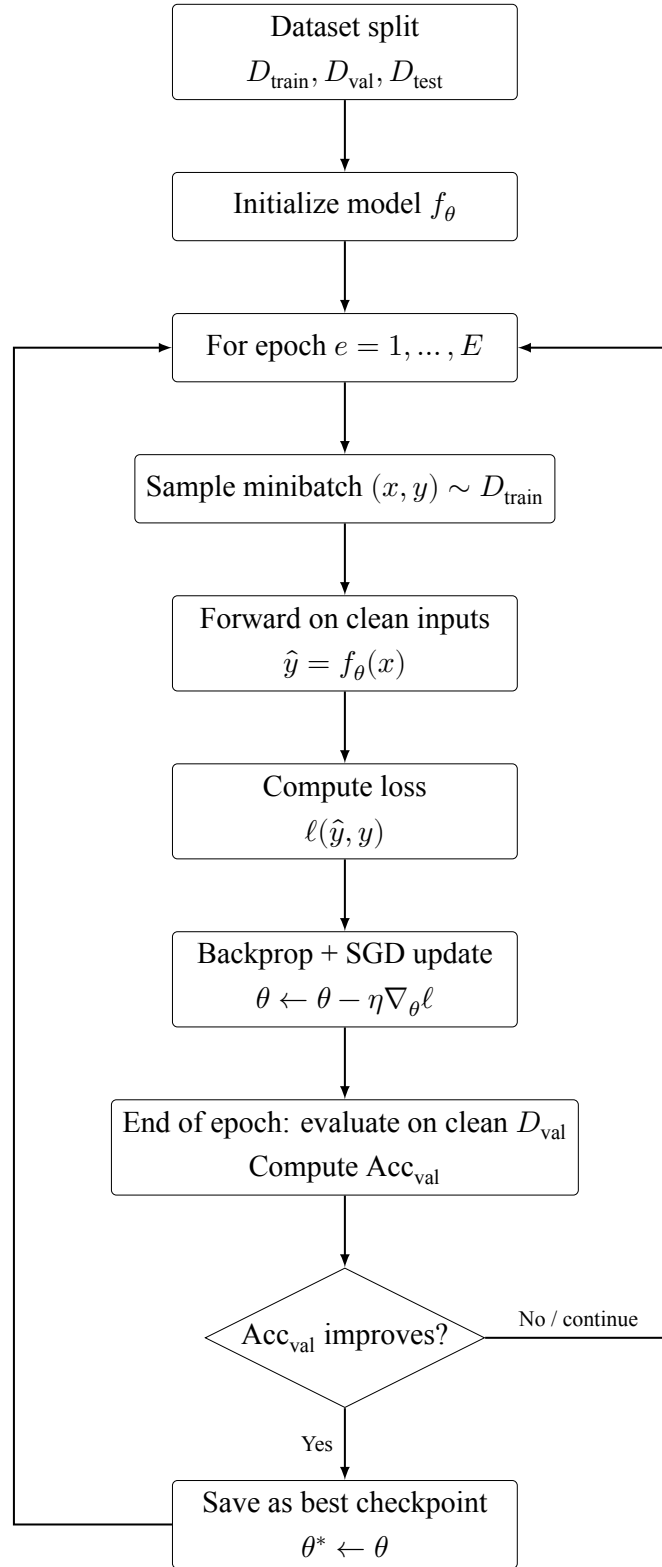


Figure A.2: ERM pipeline with best-checkpoint selection on the validation set. Validation is used only for model selection; the final reported result is obtained by evaluating the selected checkpoint θ^* on D_{test} .

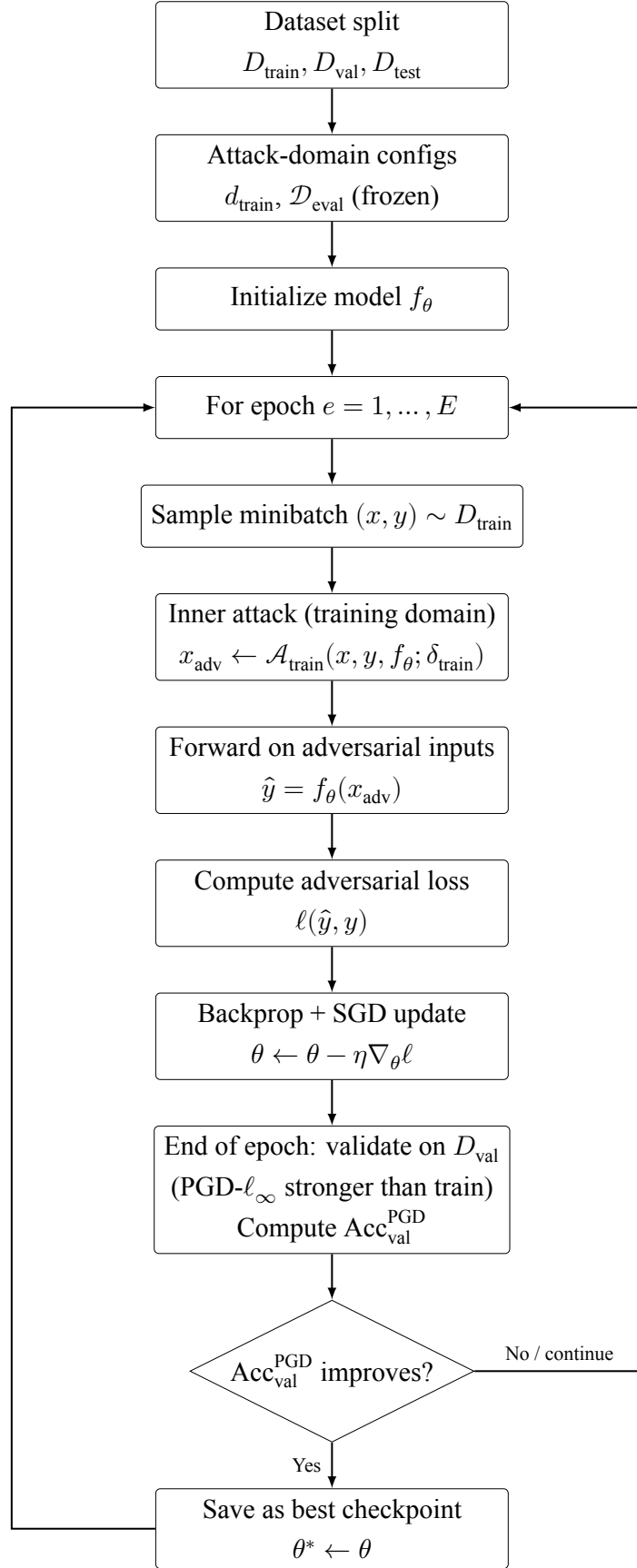


Figure A.3: Adversarial-training pipeline (Stage 1 PGD-AT) with best-checkpoint selection on the validation set and stage-invariant evaluation. Training uses a fixed training domain d_{train} to generate adversarial examples on-the-fly; evaluation reports clean accuracy and per-domain robust accuracies over a frozen domain set $\mathcal{D}_{\text{eval}}$.

A.7 Compute Environment (Reproducibility)

- **Platform:** Google Colab GPU runtime (GPU type L4; OS Linux-6.6.105+).
- **Python & core libs:**
 - Python 3.12.12
 - NumPy 2.0.2
 - pandas 2.2.2
 - matplotlib 3.10.0
- **Framework:**
 - PyTorch 2.9.0+cu126
 - torchvision 0.24.0+cu126
 - Device: cuda
- **Attacks/robustness libs:**
 - TorchAttacks 3.5.1
 - Adversarial Robustness Toolbox (ART) 1.20.1
 - AutoAttack 0.1 (installed from fra31/auto-attack, commit a39220048b3c9f2cca9a4d3a54604793c68eca7e)
- **Utilities:**
 - torchinfo
 - torchviz/graphviz
 - tqdm 4.67.1

Appendix B

Notation Tables

B.1 Notation Summary

This section summarizes the main notation used throughout the report. We group symbols into tables for data and models, adversarial examples, training hyperparameters, robustness metrics, and domain generalization.

Table B.1: General symbols used for data and models.

Symbol	Definition
$\mathcal{X} = \mathbb{R}^d$	Input space (image space)
$\mathcal{Y} = \{1, \dots, C\}$	Label space
$x \in \mathbb{R}^d$	Clean input image (example)
$y \in \{1, \dots, C\}$	Ground-truth label
$\hat{y}$	Predicted label, $\hat{y} = \arg \max_i f_\theta(x)_i$
C	Number of classes
d	Input dimension (number of pixels)
$f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^C$	Neural network classifier
θ	Model parameters
$z(x)$	Logit vector, $z(x) = f_\theta(x)$
$p_\theta(y x)$	Predicted class probability (softmax output)
D_{train}	Training set $\{(x_i, y_i)\}_{i=1}^{n_{\text{train}}}$
D_{val}	Validation set (held-out split from training data)
D_{test}	Test set $\{(x_i, y_i)\}_{i=1}^{n_{\text{test}}}$
P_{XY}	Joint data distribution over (X, Y)
$\mathcal{B}$	Mini-batch of training examples
n_{train}	Number of training examples
n_{test}	Number of test examples

Table B.2: Loss functions in adversarial training and attacks.

Symbol	Definition
$\ell(\cdot, y)$	Generic loss function for label y
$\ell_{\text{CE}}(\cdot, y)$	Cross-entropy loss for label y
$\ell_{\text{DLR}}(\cdot, y)$	Difference-of-Logits-Ratio loss (AutoAttack)

Table B.3: Notation for adversarial examples and perturbations.

Symbol	Definition
$\delta \in \mathbb{R}^d$	Adversarial perturbation
x'	Adversarial example, $x' = x + \delta$
$x^{(t)}$	Adversarial iterate at step t (e.g., PGD / APGD)
$g^{(t)}$	Gradient at step t , $g^{(t)} = \nabla_x \ell(f_\theta(x^{(t)}), y)$
y_{target}	Target label for targeted attacks
ε	Perturbation budget (attack radius)
$\ \cdot\ _p$	ℓ_p -norm, typically $p \in \{2, \infty\}$
$\mathcal{S}_p(\varepsilon)$	Threat set, $\{\delta : \ \delta\ _p \leq \varepsilon\}$
$[0, 1]^d$	Valid pixel domain for images (after normalization)
$\Pi_{x+\mathcal{S}_p(\varepsilon)}(\cdot)$	Projection onto $x + \mathcal{S}_p(\varepsilon)$

Table B.4: Training and attack hyperparameters (global notation).

Symbol	Definition
K	Number of gradient steps in an iterative attack (e.g., PGD)
T	Number of training epochs
α	Step size for PGD inner maximization (training/evaluation)
η_{lr}	Learning rate of the outer optimizer (e.g., SGD)
η_{GDRO}	Step size or temperature parameter in GroupDRO reweighting
B	Mini-batch size during training

Table B.5: Evaluation metrics used for adversarial robustness.

Symbol	Definition
M	Number of attack types in the evaluation suite
$\mathcal{A}$	Attack method (e.g., FGSM, PGD, CW, AutoAttack)
$\text{Acc}_{\text{clean}}$	Clean accuracy on unperturbed test data
$\text{Acc}_{\text{robust}}^{\mathcal{A}}$	Robust accuracy under attack $\mathcal{A}$
$\text{Acc}_{\text{robust}}^{\text{AA}}$	Robust accuracy under AutoAttack
$\Delta_{\text{clean-robust}}^{\mathcal{A}}$	Clean-versus-robust gap for attack $\mathcal{A}$
$\Delta_{\text{transfer}}^{\mathcal{A}_i \rightarrow \mathcal{A}_j}$	Cross-attack transfer gap from $\mathcal{A}_i$ to $\mathcal{A}_j$
$\bar{\Delta}_{\text{transfer}}^{\mathcal{A}_i}$	Average transfer gap for models trained on attack $\mathcal{A}_i$
$R[i, j]$	Entry (i, j) of the transfer matrix: robust accuracy when training on attack i and evaluating on attack j

Table B.6: Domain generalization notation.

Symbol	Definition
$\mathcal{E}$	Index set of domains (e.g., attack domains), $\mathcal{E} = \{1, \dots, E\}$
E	Number of domains in $\mathcal{E}$
$e \in \mathcal{E}$	Domain (attack) index
$\mathcal{D}_e$	Domain e (e.g., attack family, perturbation norm)
$P_e(X, Y)$	Joint distribution over (X, Y) in domain e
$R_e(\theta)$	Empirical risk on domain $\mathcal{D}_e$ for parameters θ
$\mathcal{D}_{\text{source}}$	Set of source domains used for training
$\mathcal{D}_{\text{target}}$	Target domain(s) used only for evaluation
$\phi(x)$	Feature representation (backbone mapping)
q_e	Domain weight for GroupDRO over domain e
λ_{REx}	Regularization strength for REx penalty
λ_{pair}	Weight of logit-pairing loss in RAMP-style training
λ_{IRM}	Regularization strength for IRM penalty

Appendix C

Additional Tables and Visualizations

This appendix collects supplementary results and sensitivity analyses that support the main empirical conclusions, but are not essential to the primary narrative flow. In particular, we report a PGD step-sweep to verify evaluation strength and to characterize the sensitivity of robust accuracy to the PGD iteration budget.

C.1 PGD Step-Sweep Sensitivity

Purpose. Robustness evaluations based on iterative attacks depend on the inner maximization budget (e.g., number of PGD steps). A too-small budget can overestimate robustness, especially for naturally trained models. Therefore, we report a step-sweep analysis to (i) validate that the evaluation is sufficiently strong, and (ii) identify a practical regime where robustness metrics stabilize for adversarially trained models.

Protocol. For each checkpoint, we evaluate clean accuracy and robustness under TorchAttacks PGD for ℓ_∞ and ℓ_2 using the same ε and step-size settings as the standard evaluation protocol, varying only the number of PGD steps $K \in \{1, 2, 5, 10, 20, 50, 100\}$. All results are reported as top-1 accuracy (%).

Naturally Trained Baseline (ERM)

Table C.1 shows that the naturally trained baseline collapses rapidly as the PGD budget increases. In particular, robustness under PGD- ℓ_∞ drops to near-zero within a few steps, and PGD- ℓ_2 robustness also vanishes as the attack becomes sufficiently strong. This supports the use of multi-step PGD as a strong evaluation for Stage 1 comparisons. Reported values are accuracy (%) and round to 2 decimal places for clarity.

Table C.1: PGD step-sweep on the naturally trained baseline (ERM).

PGD steps K	Clean	PGD- ℓ_∞	PGD- ℓ_2
1	92.00	9.78	59.24
2	92.00	0.52	27.84
5	92.00	0.00	2.38
10	92.00	0.00	0.15
20	92.00	0.00	0.03
50	92.00	0.00	0.00
100	92.00	0.00	0.00

Adversarially Trained Models: TA-100 and Robust Checkpoint

Tables C.2 and C.3 report the same analysis for two adversarially trained checkpoints. In contrast to the baseline, robust accuracies decrease as K increases, but stabilize once the PGD budget is sufficiently large (typically by $K \approx 20$ in these runs).

Table C.2: PGD step-sweep on TA-100. Clean accuracy of best checkpoint is 82.51%.

PGD steps K	PGD- ℓ_∞	PGD- ℓ_2
1	75.72	78.95
2	69.85	75.34
5	55.47	64.89
10	49.22	60.99
20	48.01	59.60
50	47.84	59.22
100	47.82	59.08

Table C.3: PGD step-sweep on TA50. Clean accuracy of best checkpoint is 82.93%.

PGD steps K	PGD- ℓ_∞	PGD- ℓ_2
1	75.61	79.52
2	69.87	75.53
5	55.79	65.30
10	49.29	61.87
20	48.23	60.46
50	47.94	59.97
100	47.89	59.90

Takeaway. The baseline ERM model exhibits severe overestimation at small PGD budgets and collapses to near-zero under sufficiently strong multi-step PGD. For adversarially trained checkpoints, robust accuracy decreases with increasing PGD steps but becomes stable once the evaluation budget is large enough (in these runs, approximately $K \geq 20$), supporting the use of a fixed multi-step PGD configuration in the main evaluation protocol.

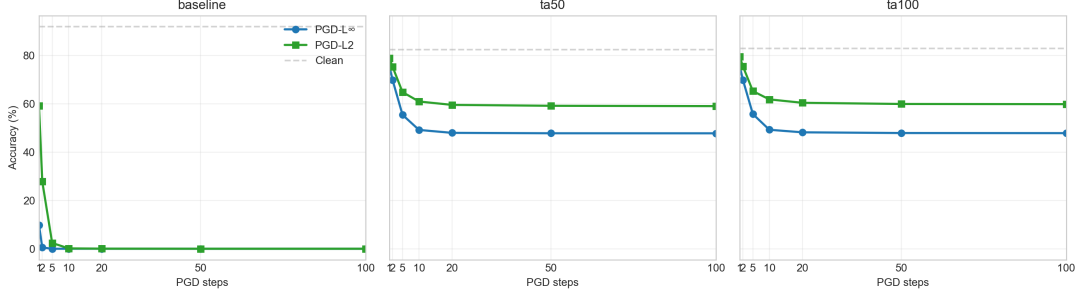


Figure C.1: Per-model PGD step-sweep sensitivity. For each checkpoint (baseline, TA-50, TA-100), we compare PGD- ℓ_∞ and PGD- ℓ_2 robust accuracy as a function of the iteration budget K .

Qualitative Adversarial Examples

This appendix provides supplementary qualitative visualizations of adversarial examples. These grids are intended for intuition and implementation sanity checks only and should not be interpreted as evidence of robustness.

Sample selection. We select a fixed set of N CIFAR-10 test images using a deterministic rule: the first N samples correctly classified by the reference checkpoint under clean evaluation (seed = 42).

Attacks and settings. Adversarial examples are generated using the same as the way we do evaluation in Chapter 5. We include FGSM- ℓ_∞ ; PGD- ℓ_∞ ($\epsilon = 8/255$, $\alpha = 2/255$, $K = 20$); PGD- ℓ_2 ($\epsilon \approx 0.5$, $K = 20$); DeepFool- ℓ_2 (50 iterations); CW- ℓ_2 (200 iterations); and AutoAttack (original). For AutoAttack, we additionally visualize each sub-attack separately (APGD-CE, APGD-DLR, FAB, Square).

Visualization. Each row corresponds to one input image and shows three columns: (i) the clean image x_{clean} , (ii) the signed perturbation $\Delta = x_{\text{adv}} - x_{\text{clean}}$ rendered as a red/blue heatmap, and (iii) the adversarial image x_{adv} .

Prediction markers. Adversarial images are outlined in green and perturbation images will be white if the prediction remains correct, otherwise red for misclassification.

PGD- ℓ_∞ | clip=0.031373 (~8.00/255)

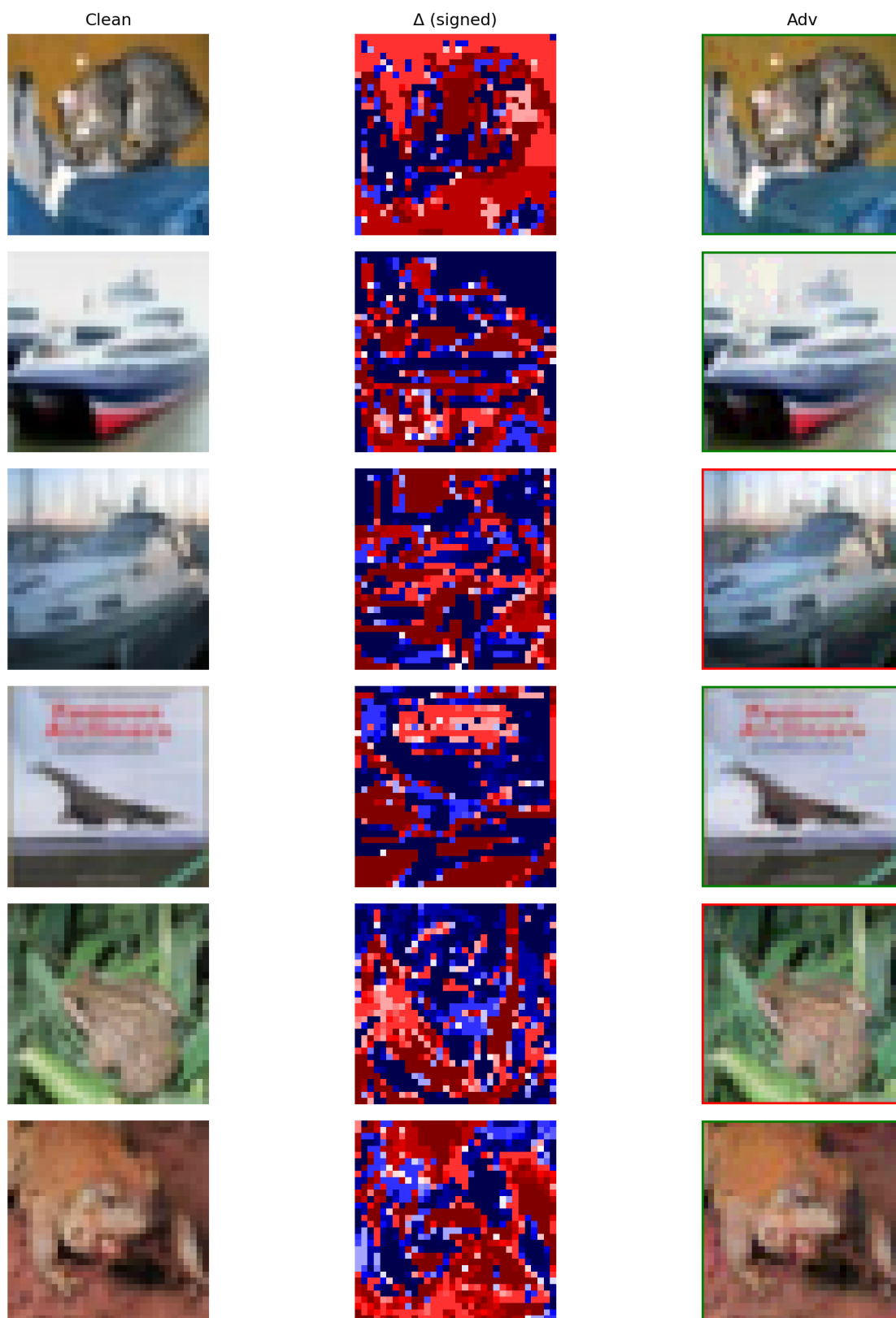


Figure C.2: PGD- ℓ_∞ qualitative examples (clean, Δ , adversarial).

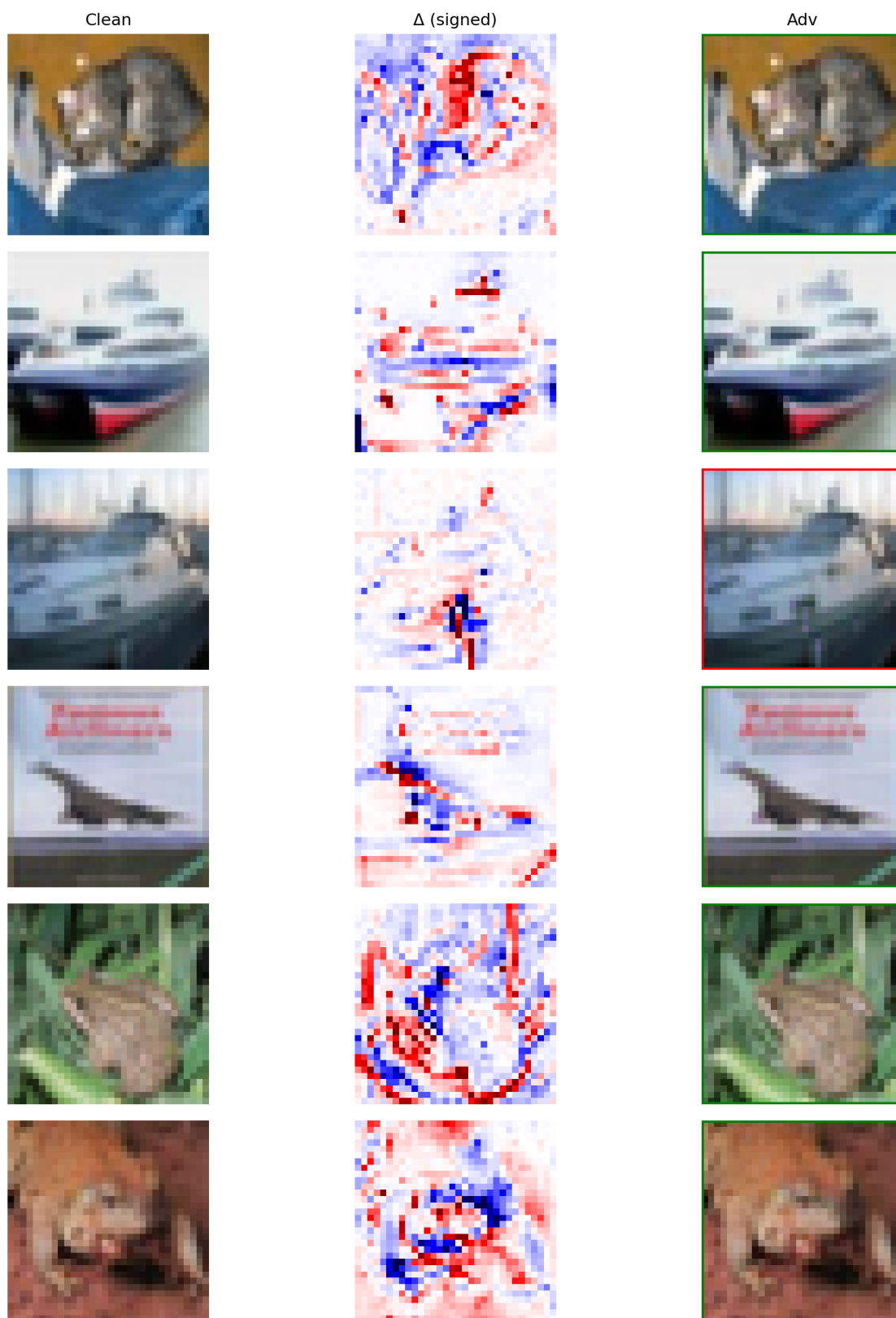


Figure C.3: PGD- ℓ_2 qualitative examples (clean, Δ , adversarial).



Figure C.4: CW- ℓ_2 qualitative examples (clean, Δ , adversarial).

AutoAttack (apgd-ce) | clip=0.031373 (~8.00/255)

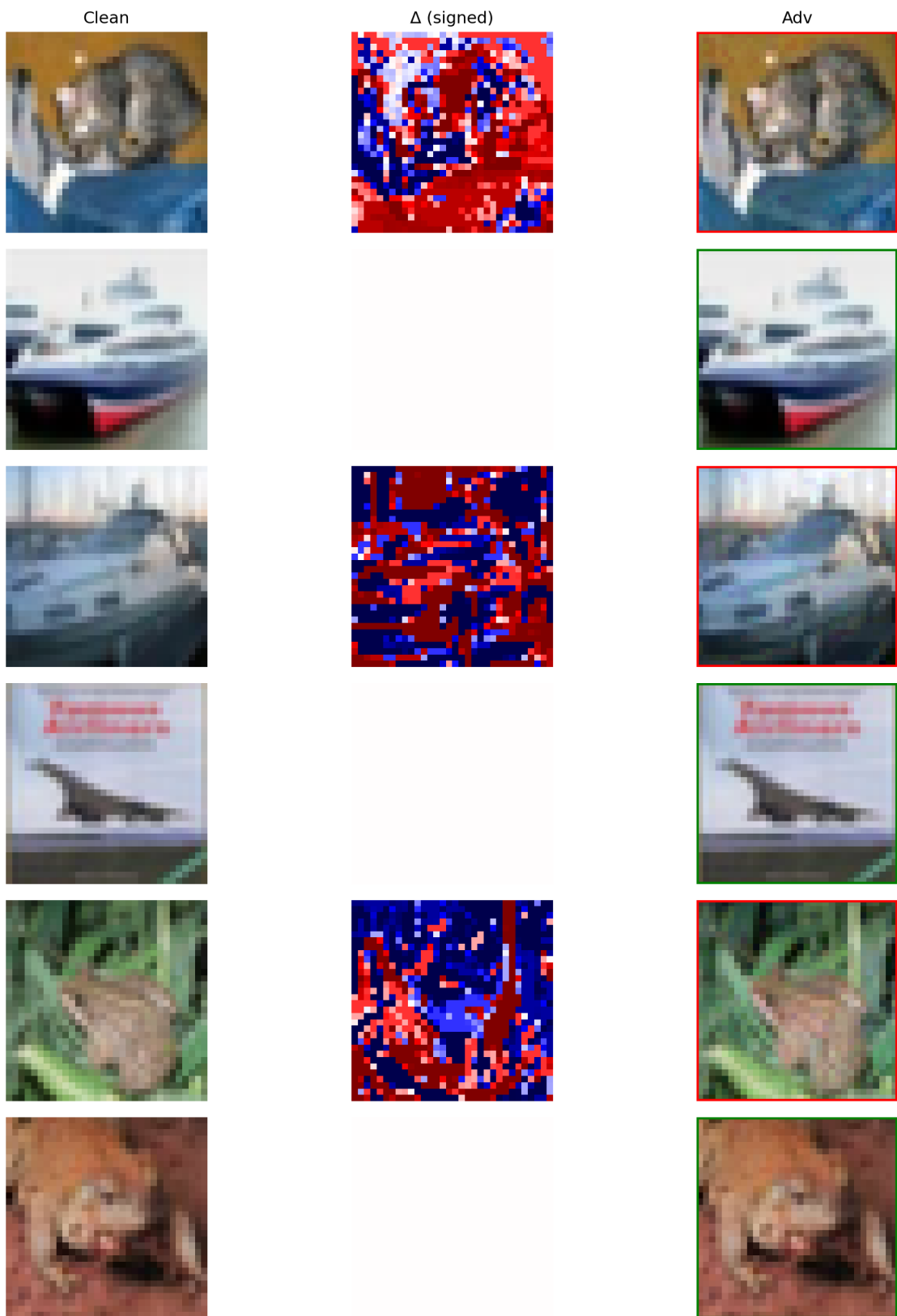


Figure C.5: AutoAttack sub-attack: APGD-CE (clean, Δ , adversarial).

AutoAttack (apgd-dlr) | clip=0.031373 (~8.00/255)

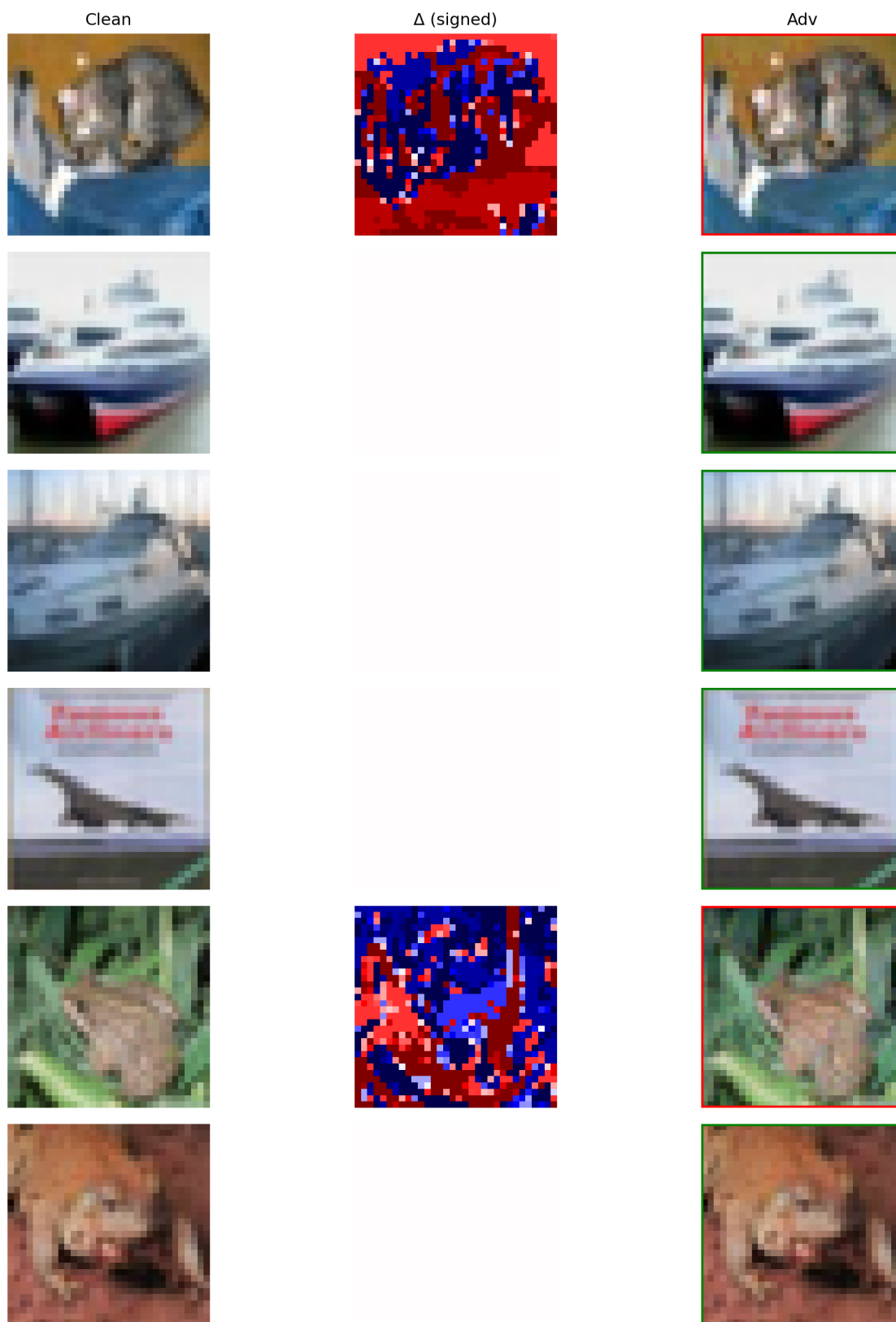


Figure C.6: AutoAttack sub-attack: APGD-DLR (clean, Δ , adversarial).

AutoAttack (fab) | clip=0.031373 (~8.00/255)

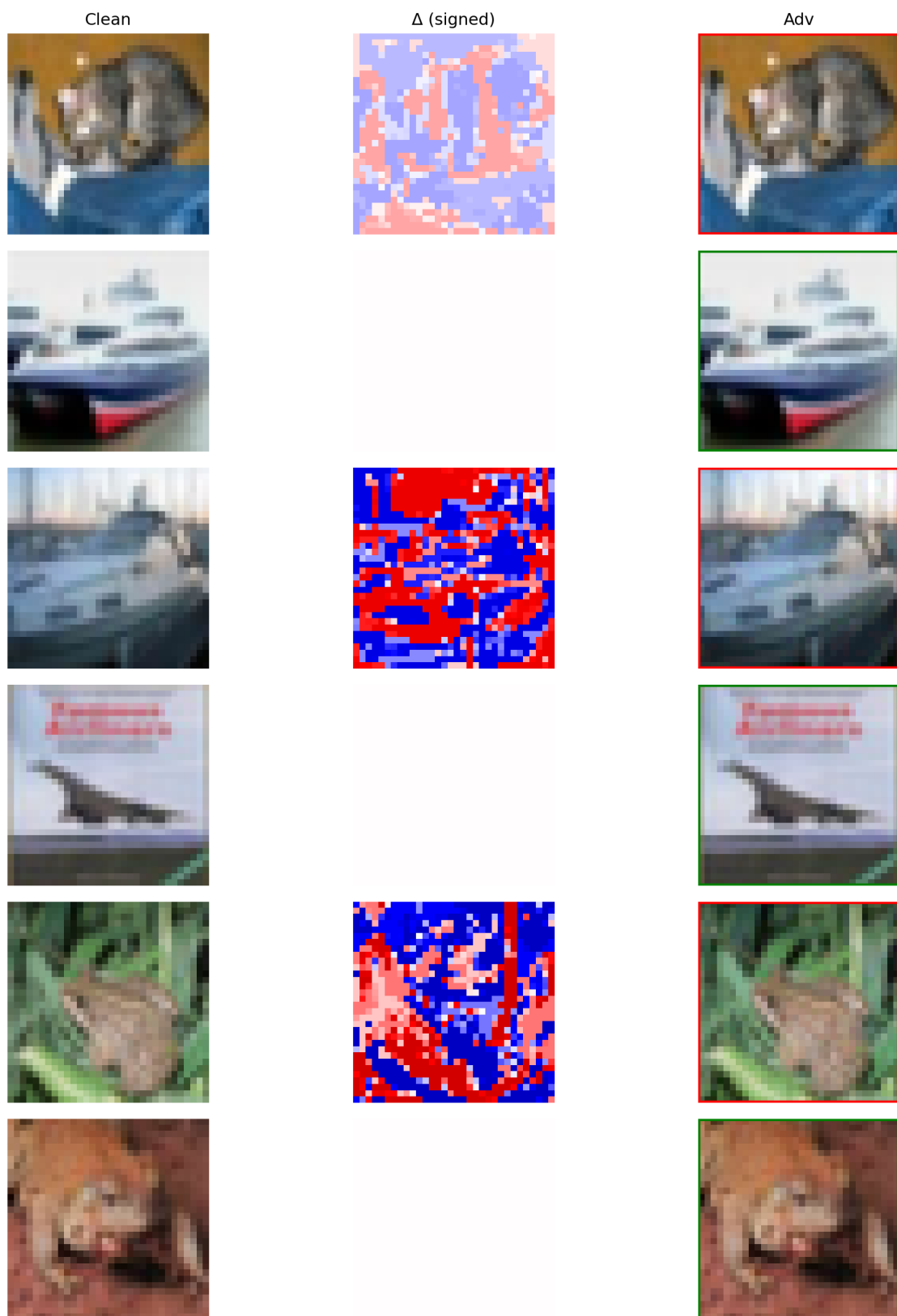


Figure C.7: AutoAttack sub-attack: FAB (clean, Δ , adversarial).

AutoAttack (square) | clip=0.031373 (~8.00/255)

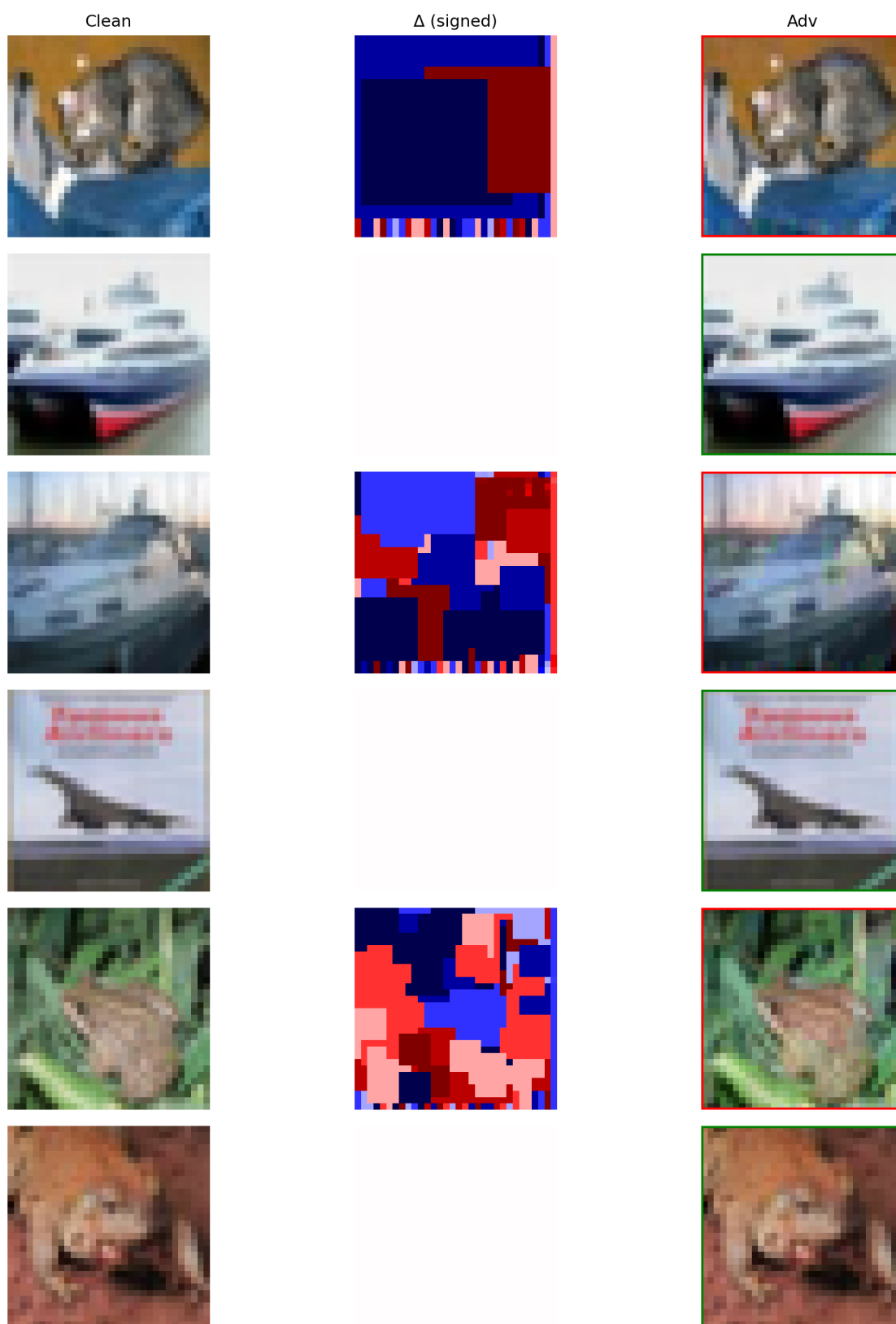


Figure C.8: AutoAttack sub-attack: Square Attack (clean, Δ , adversarial).